

Přírodou inspirované počítání

Učební text k okruhu státní závěrečné zkoušky

SZZ Umělá inteligence, MFF UK

červenec 2026

Text pokrývá okruh *Přírodou inspirované počítání* v rozsahu přednášek NAIL025 (Evoluční algoritmy I) a NAIL086 (Evoluční algoritmy II), doplněný o standardní látku z učebnic Eiben & Smith: *Introduction to Evolutionary Computing*, Mitchell: *An Introduction to Genetic Algorithms*, Michalewicz: *Genetic Algorithms + Data Structures = Evolution Programs* (odtud zejména reprezentace pro SAT a kombinatorické úlohy) a Poli, Langdon & McPhee: *A Field Guide to Genetic Programming*. Je psán tak, aby byl samonosný: obsahuje definice, pseudokódy, odvození, číselné příklady a srovnávací tabulky. Každá kapitola končí shrnutím nejdůležitějších bodů pro ústní zkoušku.

Mapování okruhu na kapitoly

Položka oficiálního okruhu	Kapitola v tomto textu
Genetické algoritmy	1 Obecné schéma EA, 2 Genetické algoritmy, 3 Reprezentace a operátory
Genetické a evoluční programování	8 Genetické a evoluční programování, gramatická evoluce
Teorie schémat	4 Teorie schémat
Pravděpodobnostní modely jednoduchého genetického algoritmu	5 Pravděpodobnostní modely SGA (Voseho model, Markovovy řetězce, EDA)
Evoluční strategie	6 Evoluční strategie
Diferenciální evoluce	7 Diferenciální evoluce
Koevoluce, otevřená evoluce	9 Koevoluce a otevřená evoluce
Rojové optimalizační algoritmy	10 Rojové algoritmy (PSO, ACO, ABC, FA)
Memetické algoritmy, hill climbing, simulované žíhání	11 Lokální prohledávání a memetické algoritmy
Aplikace: evoluce expertních systémů	12 Evoluce pravidlových a expertních systémů (LCS)
Aplikace: neuroevoluce	13 Neuroevoluce
Aplikace: řešení kombinatorických úloh	14 Kombinatorické úlohy a práce s omezeními
Aplikace: vícekritériální optimalizace	15 Vícekritériální optimalizace
(souhrnné opakování, srovnání metod)	16 Souhrnné srovnání a typické zkuškové otázky

Obsah

Mapování okruhu na kapitoly	1
1 Úvod: evoluční algoritmy a jejich obecné schéma	2
1.1 Zařazení oboru	2
1.2 Biologická inspirace	2
1.3 Obecné schéma evolučního algoritmu	2
1.4 Složky evolučního algoritmu	3
1.5 Seleční mechanismy	4
1.5.1 Ruletová (fitness-proporcionální) selekce	4
1.5.2 Stochastické univerzální vzorkování (SUS)	4

1.5.3	Turnajová selekce	4
1.5.4	Pořadová selekce	5
1.5.5	Boltzmannova selekce	5
1.5.6	Environmentální selekce, elitismus, modely populace	5
1.6	Teoretické meze: No Free Lunch	5
1.7	Historické větve evolučních algoritmů	6
1.8	Shrnutí k zkoušce	6
2	Genetické algoritmy	6
2.1	Jednoduchý genetický algoritmus (SGA)	6
2.2	Binární reprezentace	7
2.3	Operátory křížení	7
2.4	Mutace a inverze	8
2.5	Škálování fitness	8
2.6	Shrnutí k zkoušce	9
3	Reprezentace jedince a genetické operátory	9
3.1	Celočíselná reprezentace	9
3.2	Reálná reprezentace	9
3.3	Permutační reprezentace	10
3.3.1	PMX — částečně mapované křížení	10
3.3.2	OX — křížení zachovávající pořadí	11
3.3.3	CX — cyklické křížení	11
3.3.4	ER — rekombinace hran	11
3.3.5	Mutace permutací	12
3.4	Další reprezentace	12
3.5	Shrnutí k zkoušce	12
4	Teorie schémat	12
4.1	Schéma, jeho řád a definující délka	12
4.2	Hollandova věta o schématech	13
4.2.1	Odvození	13
4.2.2	Číselný příklad	14
4.3	Hypotéza stavebních bloků	14
4.4	Implicitní paralelismus a vícerozvětvený bandita	14
4.5	Kritika teorie schémat	15
4.5.1	Kdy GA selhává — deceptivní úlohy	15
4.5.2	Statické průměrné fitness — statická BBH	15
4.5.3	Souhrn slabin	15
4.6	Shrnutí k zkoušce	15
5	Pravděpodobnostní modely jednoduchého GA	16
5.1	Zjednodušený SGA	16
5.2	Formalizace populace	16
5.3	Operátor $\mathcal{G}$	16
5.4	Výsledky pro nekonečné populace	17
5.5	Konečné populace: Markovův řetězec	17
5.6	Algoritmy odhadu rozdělení (EDA)	18
5.7	Další teoretické nástroje (přehled)	19
5.8	Shrnutí k zkoušce	19
6	Evoluční strategie	19
6.1	Motivace a základní rysy	19
6.2	$(1 + 1)$ -ES a pravidlo $1/5$	20

6.3	Samoadaptace strategických parametrů	21
6.4	Rekombinace v ES	21
6.5	CMA-ES	21
6.6	Shrnutí k zkoušce	22
7	Diferenciální evoluce	23
7.1	Motivace	23
7.2	Algoritmus DE/rand/1/bin	23
7.3	Číselný příklad jednoho kroku	24
7.4	Varianty mutace	24
7.5	Srovnání DE s ES a GA	24
7.6	Shrnutí k zkoušce	24
8	Genetické a evoluční programování, gramatická evoluce	25
8.1	Historie	25
8.2	Stromové GP	25
8.3	Bloat	26
8.4	ADF a typované GP	27
8.5	Lineární a grafové GP	27
8.6	Gramatická evoluce	28
8.7	Evoluční programování	28
8.8	Je křížení k něčemu? Experiment s bezhlavým kuřetem	29
8.9	Shrnutí k zkoušce	29
9	Koevoluce a otevřená evoluce	29
9.1	Kontextová fitness	29
9.2	Kompetitivní koevoluce	30
9.3	Patologie koevoluce	30
9.4	Kooperativní koevoluce	31
9.5	Evoluce kooperace a věžnovo dilema	31
9.6	Diverzita, druhy a niky	32
9.7	Dynamické fitness krajiny	33
9.8	Otevřená evoluce	33
9.9	Interaktivní evoluce	35
9.10	Shrnutí k zkoušce	35
10	Rojové optimalizační algoritmy	35
10.1	Optimalizace rojem částic (PSO)	35
10.2	Optimalizace mravenčí kolonií (ACO)	37
10.3	Další rojové algoritmy	38
10.4	Shrnutí k zkoušce	39
11	Lokální prohledávání a memetické algoritmy	39
11.1	Lokální prohledávání a horolezecký algoritmus	39
11.2	Simulované žíhání	40
11.3	Tabu prohledávání	41
11.4	Scatter search	41
11.5	Memetické algoritmy	41
11.6	Ladění a řízení parametrů	42
11.7	Shrnutí k zkoušce	43
12	Evoluce pravidlových a expertních systémů	43
12.1	Michigan vs. Pittsburgh	43
12.2	Learning classifier systems (Michigan)	43

12.3	Pittsburghský přístup a systém GIL	45
12.4	Připomenutí: metriky klasifikace	45
12.5	Shrnutí k zkoušce	45
13	Neuroevoluce	46
13.1	Evoluce vah	46
13.2	Evoluce architektury	46
13.3	NEAT	47
13.4	HyperNEAT a CPPN	47
13.5	Neuroevoluce v éře hlubokých sítí	47
13.6	Shrnutí k zkoušce	48
14	Kombinatorické úlohy a práce s omezeními	48
14.1	Problém batohu	48
14.2	Tři obecné strategie pro omezení	49
14.3	Problém obchodního cestujícího	49
14.4	SAT	49
14.5	Rozvrhování a plánování výroby	50
14.6	Shrnutí k zkoušce	50
15	Vícekritériální optimalizace	51
15.1	Dominance a Pareto fronta	51
15.2	Skalarizace — jednoduchá řešení	51
15.3	Historické algoritmy	51
15.4	NSGA-II	52
15.5	Další významné algoritmy	53
15.6	Metriky kvality aproximace	53
15.7	Souhrnné srovnání MOEA	54
15.8	Shrnutí k zkoušce	54
16	Souhrnné srovnání a typické zkouškové otázky	54
16.1	Mapa oboru	54
16.2	Souhrnná srovnávací tabulka algoritmů	55
16.3	Průřezová témata — co se ptá napříč otázkami	55
16.4	Typické zkouškové otázky	55
16.5	Závěrečné shrnutí	56

1 Úvod: evoluční algoritmy a jejich obecné schéma

1.1 Zařazení oboru

Přírodou inspirované počítání (*nature-inspired computing*) je souhrnné označení pro metaheuristické algoritmy, které přebírají principy pozorované v živé (a někdy i neživé) přírodě. Tradiční členění je následující:

- **Výpočetní inteligence** (*computational intelligence*, dříve *soft computing*) zahrnuje neuronové sítě, fuzzy systémy a evoluční výpočty. Stojí proti „tvrdé“, logicky a symbolicky orientované AI.
- **Evoluční výpočty** (*evolutionary computation*, EC) jsou nadmnožinou zahrnující i metody, které nejsou evoluční v úzkém smyslu (rojové algoritmy, memetické algoritmy, tabu prohledávání).
- **Evoluční algoritmy** (*evolutionary algorithms*, EA) jsou podmnožina EC využívající základní principy přírodní evoluce: populaci kandidátních řešení, stochastické operátory (křížení, mutace) a selekci řízenou účelovou funkcí.

EA jsou **populační stochastické prohledávací algoritmy**. Používají se tam, kde nemáme k dispozici gradient nebo analytický tvar účelové funkce (tzv. *black-box optimization*), kde je prostor řešení diskrétní, strukturovaný (stromy, grafy, permutace) nebo kde hledáme více vzájemně nedominovaných řešení najednou.

1.2 Biologická inspirace

- **Darwin (1859, O původu druhů)**. Prostředí má omezené zdroje, reprodukce je klíčem k přežití, lépe přizpůsobení jedinci mají větší šanci se rozmnožit. Úspěšné znaky fenotypu se reprodukují, modifikují a rekombinují.
- **Mendel (1866)**. Gen jako základní jednotka dědičnosti; alely se předávají nezávisle. Realita je složitější: *polygenie* (více genů ovlivňuje jeden znak), *pleiotropie* (jeden gen ovlivňuje více znaků), mitochondriální DNA, epigenetika.
- **Watson a Crick (1953)**. Struktura DNA; molekulárně-biologický pohled na ukládání a dědění informace. Kodon (trojice nukleotidů) kóduje jednu z aminokyselin, kód je redundantní.
- **Genotyp → fenotyp**. Zobrazení je jednosměrné a složité (transkripce DNA→RNA, translace RNA→protein). *Lamarckismus* naopak předpokládá existenci inverzního zobrazení, tedy dědičnost získaných vlastností. V biologii je odmítnut, v memetických algoritmech se však jako technika používá (kap. 11).

Slovník: příroda vs. algoritmus

prostředí	optimalizační úloha
jedinec / chromozom	kandidátní řešení, bod prohledávaného prostoru
gen, alela	složka řešení a její hodnota
genotyp	vnitřní reprezentace (kódování) jedince
fenotyp	„vnějšek“ jedince, na němž se hodnotí kvalita
fitness	míra kvality řešení (účelová funkce)
populace	multimnožina kandidátních řešení
generace	jedna iterace algoritmu

1.3 Obecné schéma evolučního algoritmu

Evoluční algoritmus

Evoluční algoritmus je iterativní stochastický proces, který udržuje populaci $P(t)$ kandidátních řešení a z ní opakovaně vytváří populaci $P(t+1)$ pomocí *rodičovské selekce*, *rekombinace*, *mutace* a *environmentální selekce*, dokud není splněna ukončovací podmínka.

Algoritmus 1 Obecné schéma evolučního algoritmu

Vstup: reprezentace, fitness f , operátory, parametry

```
1:  $t \leftarrow 0$ ; inicializuj náhodně populaci  $P(0)$ 
2: ohodnoť všechny jedince v  $P(0)$  funkcí  $f$ 
3: while není splněna ukončovací podmínka do
4:   rodičovská selekce: vyber z  $P(t)$  množinu rodičů (pravděpodobnostně, dle fitness)
5:   rekombinace: z dvojic (n-tic) rodičů vytvoř potomky s pravděpodobností  $p_C$ 
6:   mutace: každého potomka náhodně poruš s pravděpodobností  $p_M$ 
7:   ohodnoť potomky  $P'(t+1)$ 
8:   environmentální selekce: z  $P(t) \cup P'(t+1)$  (nebo jen z  $P'(t+1)$ ) vyber novou populaci  $P(t+1)$ 
9:    $t \leftarrow t+1$ 
10: end while
```

Výstup: nejlepší nalezený jedinec

Tento cyklus je společný všem EA; jednotlivé varianty (GA, ES, EP, GP, DE) se liší volbou reprezentace, důrazem na jednotlivé operátory a typem selekce. Klíčovou roli hrají dvě protichůdné síly:

- **Variace** (křížení a mutace) vytváří novou rozmanitost — zajišťuje *explorace* (*exploration*) prostoru.
- **Selekce** rozmanitost odebírá a směřuje hledání ke slibným oblastem — zajišťuje *exploataci* (*exploitation*).

Rovnováha mezi nimi je centrální problém návrhu EA. Samotná explorační degeneruje na náhodnou procházku; samotná exploatace vede k uvíznutí v lokálním extrému a *předčasné konvergenzi* (*premature convergence*).

1.4 Složky evolučního algoritmu

Reprezentace. Volba kódování je nejdůležitější návrhové rozhodnutí. Klasické možnosti: binární řetězec, vektor celých čísel, vektor reálných čísel, permutace, strom, graf, matice, konečný automat, neuronová síť. Reprezentace určuje, jaké operátory dávají smysl.

Fitness funkce. Zobrazení $f : \text{prostor fenotypů} \rightarrow \mathbb{R}$, které měří kvalitu. Pro minimalizační úlohy se obvykle převádí na maximalizaci (např. $f' = C_{\max} - f$ nebo $f' = 1/(1 + f)$). Fitness může být *deterministická*, *zašuměná* (simulace), *dynamická* (mění se v čase, kap. 9) nebo *kontextová* (závisí na ostatních jedincích — koevoluce).

Populace. Multimnožina jedinců pevné (obvykle) velikosti n . Populace je nositelem *diverzity*; ta se měří např. průměrnou Hammingovou vzdáleností, entropií alel na pozicích nebo rozptylem fitness.

Inicializace. Obvykle náhodná uniformní. Alternativy: *seeding* (vložení heuristických nebo expertních řešení — riziko předčasné konvergenze), konstrukční heuristiky (např. nejbližší soused pro TSP), latinské hyperkrychle pro reálné vektory.

Ukončovací podmínka. Maximální počet generací či vyhodnocení fitness, dosažení cílové kvality, stagnace nejlepší fitness po k generací, ztráta diverzity populace.

1.5 Selekční mechanismy

1.5.1 Ruletová (fitness-proporcionální) selekce

Ruletová selekce (*roulette wheel selection*)

Jedinec x_i je vybrán s pravděpodobností

$$p_i = \frac{f(x_i)}{\sum_{j=1}^n f(x_j)} = \frac{f(x_i)}{n \bar{f}},$$

kde $\bar{f}$ je průměrná fitness populace. Očekávaný počet výběrů jedince při n tazích je $np_i = f(x_i)/\bar{f}$. Jde o výběr *s vrácením* — jeden jedinec může být vybrán vícekrát.

Příklad (klasický Goldbergův). Populace čtyř 5-bitových řetězců, fitness $f(x) = x^2$ (dekódováno jako celé číslo):

i	řetězec	x	$f = x^2$	$p_i = f / \sum f$	oček. počet $f/\bar{f}$
1	01101	13	169	0,144	0,58
2	11000	24	576	0,492	1,97
3	01000	8	64	0,055	0,22
4	10011	19	361	0,309	1,23
součet			1170	1,000	4,00
průměr $\bar{f}$			292,5		

Ruleta rozdělí obvod kola na výseče o velikostech p_i a roztočí se n -krát. Jedinec 2 obsadí zhruba dvě místa v mezipopulaci, jedinec 3 pravděpodobně vypadne.

Nevýhody ruletové selekce:

- *Předčasná dominance* „supermana“: jedinec s extrémní fitness ovládne populaci a diverzita zmizí.
- *Ztráta selekčního tlaku* na konci běhu: když jsou všechny fitness podobné, selekce se blíží náhodnému výběru.
- Vyžaduje kladné fitness a je citlivá na afinní transformace (f a $f+c$ dávají jiné pravděpodobnosti). Řeší se *škálováním* (odst. 2.5).
- Vysoký rozptyl výběrů — skutečné počty se mohou od očekávaných značně lišit.

1.5.2 Stochastické univerzální vzorkování (SUS)

Stochastic universal sampling používá jediné roztočení kola s n rovnoměrně rozmístěnými ukazateli vzdálenými o $1/n$. Očekávané počty výběrů zůstávají stejné jako u rulety, ale rozptyl je minimální: jedinec s očekávaným počtem e_i je vybrán $\lfloor e_i \rfloor$ nebo $\lceil e_i \rceil$ -krát. Je to tedy „spravedlivější ruleta“.

1.5.3 Turnajová selekce

k -turnajová selekce (*tournament selection*)

Vyber náhodně (uniformně, s vrácením) k jedinců a vrať toho nejlepšího. Parametr k (typicky 2, 3, 5) přímo řídí selekční tlak: pravděpodobnost, že bude vybrán i -tý nejlepší jedinec z n (zde $i = 1$ značí **nejlepšího**), je $((n-i+1)^k - (n-i)^k)/n^k$ — je to pravděpodobnost, že všech k losovaných padne mezi i -tého a horší, mínus pravděpodobnost, že všech k padne ostře pod i -tého. Deterministický turnaj lze změkčit: lepší vyhraje s pravděpodobností $p < 1$.

Výhody: nevyžaduje globální informaci (žádné sčítání fitness), je invariantní vůči monotónním transformacím fitness, snadno paralelizovatelná, funguje i tam, kde fitness není explicitně dána a porovnání se získává *souhrou* (hraná partie, simulace, koevoluce).

1.5.4 Pořadová selekce

Rank-based selection nahradí fitness pořadím jedince. Lineární pořadová selekce přiřadí jedinci s pořadím i (pozor, zde opačně než u turnaje: nejhorší $i = 1$, nejlepší $i = n$) pravděpodobnost

$$p_i = \frac{1}{n} \left(s^- + (s^+ - s^-) \frac{i-1}{n-1} \right), \quad 1 \leq s^+ \leq 2, \quad s^- = 2 - s^+,$$

kde s^+ je očekávaný počet kopií nejlepšího jedince. Exponenciální varianta používá $p_i \propto (1-c)c^{n-i}$. Pořadová selekce udržuje konstantní selekční tlak během celého běhu a je odolná vůči „supermanům“ i vůči změně měřítka fitness.

1.5.5 Boltzmannova selekce

Pravděpodobnost výběru $p_i \propto e^{f(x_i)/T}$ s teplotou T , která klesá v čase: na začátku (vysoké T) je selekce téměř uniformní (explorace), na konci (nízké T) téměř deterministická (exploatace). Analogicky *Boltzmannův turnaj*: horší jedinec vyhraje s pravděpodobností závislou exponenciálně na poměru rozdílu fitness a teploty. Přímá souvislost se simulovaným žíháním (kap. 11).

1.5.6 Environmentální selekce, elitismus, modely populace

- **Generační model** (*generational*): celá populace je nahrazena potomky, $|P'| = |P| = n$.
- **Model s ustáleným stavem** (*steady-state*): v každém kroku vznikne jen λ (často 1–2) potomků, kteří nahradí vybrané jedince (nejhoršího, náhodného, nejpodobnějšího — *crowding*). Parametr *generation gap* $G = \lambda/n$ udává podíl nahrazované populace.
- **Elitismus** (*elitism*): k nejlepších jedinců se bezpodmínečně kopíruje do další generace. Zaručuje monotonii nejlepší nalezené fitness a je nutnou podmínkou pro řadu konvergenčních vět; příliš silný elitismus však snižuje diverzitu.
- **Deterministická selekce ES**: $(\mu + \lambda)$ vybírá μ nejlepších z rodičů i potomků (implicitní elitismus), (μ, λ) jen z potomků (kap. 6).

Selekční tlak a doba převzetí

Selekční tlak (*selection pressure*) je míra zvýhodnění lepších jedinců. Kvantifikuje se *dobou převzetí* (*takeover time*) τ^* — počtem generací, za který jediná kopie nejlepšího jedince (bez variace) zaplní celou populaci. Pro turnaj velikosti k platí zhruba $\tau^* \approx \ln n / \ln k$, pro fitness-proporcionální selekci je doba převzetí delší a závisí na rozložení fitness.

1.6 Teoretické meze: No Free Lunch

Věta 1.1 (No Free Lunch, Wolpert & Macready 1995–1997). *Zprůměrováno přes všechny možné účelové funkce na konečné doméně mají libovolné dva deterministické neopakující se prohledávací algoritmy stejný očekávaný výkon. Neexistuje tedy univerzálně nejlepší optimalizační algoritmus.*

Praktický důsledek: vyplácí se vytvářet **doménově specifické varianty** EA — vhodnou reprezentaci, operátory zohledňující strukturu úlohy a hybridizaci s lokálními heuristikami. Věta neříká, že všechny algoritmy jsou stejně dobré na *reálných* úlohách; reálné funkce tvoří velmi malou a strukturovanou podmnožinu všech funkcí.

1.7 Historické větve evolučních algoritmů

	GA	ES	EP	GP
autor, rok	Holland, 1975	Rechenberg, Schwefel, 1964	L. Fogel, Owens, Walsh, 1965	Koza, 1989–92
reprezentace	binární řetězec	vektor $\mathbb{R}^n$ + strategické parametry	konečný automat (genotyp = fenotyp)	syntaktický strom (LISP)
hlavní operátor křížení	křížení	mutace	mutace	křížení podstromů
	1-bodové	rekombinace více rodičů (lokální/globální)	<i>není</i>	výměna podstromů
mutace	bit-flip p_M	gaussovská, samoadaptivní σ náhodná	„chytré“ mutace automatu	více typů
rodičovská selekce	ruleta		každý jednou	turnaj
env. selekce	generační náhrada	deterministická (μ, λ) , $(\mu + \lambda)$	turnaj (rodiče + potomci)	generační
teorie	schémata	konvergence, 1/5 pravidlo	—	bloat, schémata

Dnes se hranice mezi větvemi stírají (např. EP a ES prakticky splynuly) a mluvíme jednotně o evolučních algoritmech.

1.8 Shrnutí k zkoušce

- EA = populační stochastické prohledávání; cyklus *inicializace* → *ohodnocení* → *rodičovská selekce* → *rekombinace* → *mutace* → *ohodnocení* → *environmentální selekce*.
- Variace (křížení, mutace) tvoří diverzitu, selekce ji odebírá a směřuje hledání; návrh EA = hledání rovnováhy explorační/exploatační.
- Selekce: **ruleta** ($p_i = f_i / \sum f_j$, vysoký rozptyl, citlivá na škálování), **SUS** (stejně střední hodnoty, nízký rozptyl), **turnaj** (tlak řízen k , invariantní k transformacím fitness, nepotřebuje explicitní fitness), **pořadová** (konstantní tlak), **Boltzmannova** (tlak roste s časem).
- Elitismus zaručuje monotonii nejlepší fitness; generační vs. steady-state model se liší podílem nahrazované populace.
- No Free Lunch: neexistuje univerzálně nejlepší algoritmus ⇒ doménová specializace reprezentace a operátorů.
- Historické větve GA / ES / EP / GP a jejich charakteristické volby (viz srovnávací tabulka) je dobré umět vyjmenovat z paměti.

2 Genetické algoritmy

2.1 Jednoduchý genetický algoritmus (SGA)

Původní Hollandův návrh z roku 1975 se dnes označuje jako *simple genetic algorithm* (SGA). Používá minimální sadu operátorů, nejjednodušší kódování a byl navržen tak, aby umožňoval teoretickou analýzu (teorie schémat, kap. 4).

SGA

Populace n binárních řetězců délky m ; fitness-proporcionální (ruletová) selekce; jednobodové křížení s pravděpodobností p_C ; bitová mutace s pravděpodobností p_M na každý bit; generační náhrada populace ($P(t+1)$ zcela nahradí $P(t)$).

Typické hodnoty parametrů: $n \in [50, 500]$, $p_C \in [0,6; 0,9]$, $p_M \approx 1/m$ (tj. v průměru se změní jeden bit na jedince).

Algoritmus 2 Jednoduchý genetický algoritmus (SGA)

```
1:  $t \leftarrow 0$ ; vygeneruj náhodně  $P(0) = n$  řetězců délky  $m$ 
2: while není splněna ukončovací podmínka do
3:   spočítej  $f(x)$  pro každé  $x \in P(t)$ 
4:    $P(t+1) \leftarrow \emptyset$ 
5:   for  $n/2$  krát do
6:     vyber ruletou dvojici rodičů  $x, y$  z  $P(t)$ 
7:     s pravděpodobnostmi  $p_C$  zkříž  $x, y$  jednobodovým křížením
8:     každý bit  $x$  i  $y$  převrať s pravděpodobnostmi  $p_M$ 
9:     vlož  $x, y$  do  $P(t+1)$ 
10:  end for
11:   $t \leftarrow t+1$ 
12: end while
```

2.2 Binární reprezentace

Kódování celých a reálných čísel. Reálné číslo $x \in [a, b]$ kódované na k bitů jako celé číslo $z \in \{0, \dots, 2^k - 1\}$ dekódujeme

$$x = a + z \cdot \frac{b-a}{2^k-1}.$$

Přesnost je $(b-a)/(2^k-1)$. Výhoda: explicitní kontrola nad přesností a komprese prohledávaného prostoru. Nevýhoda: *Hammingovy útesy* (*Hamming cliffs*) — sousední hodnoty se mohou lišit ve všech bitech ($0111 \rightarrow 1000$), takže malá změna fenotypu vyžaduje velkou změnu genotypu.

Grayův kód. Řeší Hammingovy útesy: sousední celá čísla se v Grayově kódu liší právě v jednom bitu. Převod z binárního b na Grayův g : $g_1 = b_1$, $g_i = b_{i-1} \oplus b_i$. Zpět: $b_1 = g_1$, $b_i = b_{i-1} \oplus g_i$.

Hollandův argument pro binární abecedu. Binární řetězce délky 100 kódují zhruba stejnou informaci jako desítkové délky 30, ale obsahují více schémat (3^{100} oproti 11^{30}), takže GA „zpracovává“ více informací. Dnes víme, že schémata nejsou tak zásadní, jak se Holland domníval, a že rozhodující je **přirozenost kódování pro danou úlohu**. Pro reálnou optimalizaci se používají reálné vektory, pro TSP permutace atd.

2.3 Operátory křížení

Křížení (*crossover, recombination*)

Operátor, který ze dvou (nebo více) rodičů vytvoří potomka kombinací jejich genetického materiálu. V GA je hlavním operátorem; vyjadřuje naději, že rekombinací dobrých částí (*stavebních bloků*) vzniknou ještě lepší jedinci. Aplikuje se s pravděpodobností p_C , jinak potomci = kopie rodičů.

Jednobodové křížení. Zvol náhodně bod řezu $k \in \{1, \dots, m-1\}$ a vyměň koncové části.

Příklad. Rodiče $P_1 = 1011|0110$, $P_2 = 0010|1101$, bod řezu $k = 4$:

$$O_1 = \mathbf{1011} \ 1101, \quad O_2 = \mathbf{0010} \ 0110.$$

Dvoubodové křížení. Zvol dva body $k_1 < k_2$ a vyměň prostřední úsek. Výhoda oproti jednobodovému: řetězec se chová jako kruh, takže nejsou znevýhodněny geny na okrajích (u jednobodového křížení je pravděpodobnost rozbití úseku úměrná jeho *definující délce*, viz kap. 4).

Příklad. $P_1 = 10|1101|10$, $P_2 = 00|1011|01 \Rightarrow O_1 = 10 \ 1011 \ 10$, $O_2 = 00 \ 1101 \ 01$.

Uniformní křížení. Pro každou pozici j se hodí mince: s pravděpodobností 0,5 pochází gen od prvního rodiče, jinak od druhého (druhý potomek je komplementární). Nezvýhodňuje sousedící pozice, ale silně narušuje schémata s velkou definující délkou — má vysokou *disruptivitu*.

Příklad. $P_1 = 11111111$, $P_2 = 00000000$, maska 10110010 : $O_1 = 10110010$, $O_2 = 01001101$.

operátor	p-st rozbití schématu délky d	poznámka
jednobodové	$d/(m-1)$	poziční bias, chrání krátká schémata
dvoubodové	$\frac{2d(m-d)}{m(m-1)}$	řetězec jako kruh, odstraňuje bias konců
k -bodové	roste s k	
uniformní	$1 - (1/2)^{o(S)-1}$	distribuční bias, největší disruptivita

Poznámka k dvoubodovému křížení. Chápeme-li řetězec jako kruh, volíme dva ze m řezů; schéma je rozbito, padne-li *právě jeden* řez dovnitř jeho definujícího úseku o d mezerách, tj. s pravděpodobností $d(m-d)/\binom{m}{2} = 2d(m-d)/(m(m-1))$. Pro schéma sahající od prvního k poslednímu bitu ($d = m-1$) klesne rozbití z jistoty (jednobodové: $d/(m-1) = 1$) na pouhé $2/m$ — to je přesně ono „odstranění okrajového biasu“. Pro krátká schémata je naopak dvoubodové křížení asi dvakrát destruktivnější než jednobodové.

2.4 Mutace a inverze

Bitová mutace. Každý bit se s pravděpodobností p_M nezávisle překlápí. V SGA je mutace *sekundárním* operátorem — působí jako pojistka proti uvíznutí v lokálním extrému a proti trvalé ztrátě alely z populace. (Naopak v EP a raných ES je mutace jediným zdrojem variability.) Doporučení: $p_M \approx 1/m$.

Inverze. Původní Hollandův návrh obsahoval ještě operátor *inverze*: obrátí pořadí části řetězce, ale *zachová význam bitů* (gen si nese svůj identifikátor pozice). Cílem bylo umožnit evoluci samotného uspořádání genů tak, aby spolupracující geny ležely blízko sebe a nebyly rozbíjeny křížením. Technicky je to náročné a přínos se neprokázal; u permutační reprezentace se však inverze používá jako velmi účinná mutace (kap. 3).

2.5 Škálování fitness

Ruletová selekce je citlivá na absolutní hodnoty fitness. Na začátku běhu bývá rozptýl fitness velký (několik „supermanů“ ovládne populaci), na konci malý (selekce ztrácí tlak). Řešením je transformace $f \mapsto f'$:

- **Lineární škálování:** $f' = af + b$, kde a, b se volí tak, aby $\bar{f}' = \bar{f}$ a $f'_{\max} = C_{\text{mult}} \cdot \bar{f}$, typicky $C_{\text{mult}} \in [1, 2]$. Nutná pojistka proti záporným hodnotám u nejhorších jedinců.
- **Sigma truncation:** $f' = \max(0, f - (f - c\sigma_f))$, kde σ_f je směrodatná odchylka fitness a $c \in [1; 3]$. Odstraní odlehlé nízké hodnoty a automaticky se přizpůsobuje rozptylu.
- **Mocninné škálování:** $f' = f^k$, kde k se v průběhu běhu zvyšuje (tlak roste).
- **Pořadové škálování:** nahradí fitness pořadím (viz odst. 1.5) — nejrobustnější, dnes nejobvyklejší.
- **Boltzmannovo škálování:** $f' = e^{f/T}$ s klesající teplotou.

Příklad. Populace s fitness (169, 576, 64, 361), $\bar{f} = 292,5$, $f_{\max} = 576$. Lineární škálování s $C_{\text{mult}} = 1,5$: chceme $f'_{\max} = 438,75$ a $\bar{f}' = 292,5$. Z podmínek $af_{\max} + b = 438,75$ a $a\bar{f} + b = 292,5$ dostáváme $a = 146,25/283,5 = 0,516$, $b = 292,5 - 0,516 \cdot 292,5 = 141,6$. Nové fitness: (228,8; 438,8; 174,6; 327,9); podíl nejlepšího klesl z 49,2 % na 37,5 % — selekční tlak je zmírněn a nejhorší jedinec neztrácí šanci.

2.6 Shrnutí k zkoušce

- SGA = binární řetězce + ruleta + 1-bodové křížení + bit-flip mutace + generační náhrada. Parametry: n , $p_C \approx 0,7$, $p_M \approx 1/m$.
- Binární kódování reálných čísel: $x = a + z(b - a)/(2^k - 1)$; problém Hammingových útesů řeší Grayův kód.
- Křížení: 1-bodové (rozbití schématu s pravděpodobností $d(S)/(m-1)$), 2-bodové (bez okrajového biasu), uniformní (nejvíce disruptivní, bez pozičního biasu).
- Mutace je v GA sekundární — brání nevratné ztrátě alel; inverze je historický operátor pro evoluci uspořádání genů.
- Škálování fitness (lineární, sigma truncation, mocninné, pořadové, Boltzmannovo) opravuje slabiny fitness-proporcionální selekce: dominanci supermanů na začátku a ztrátu tlaku na konci.

3 Reprezentace jedince a genetické operátory

Reprezentace určuje, které operátory mají smysl, a tím i to, jak vypadá *fitness krajina* (*fitness landscape*) — tedy jak snadné je prohledávání. Toto je nejdůležitější praktické rozhodnutí při návrhu EA.

3.1 Celočíselná reprezentace

Jedinec je vektor $(z_1, \dots, z_m) \in \prod_j D_j$, kde D_j jsou konečné domény. Rozlišujeme:

- **Kardinální (ordinální) atributy** — na hodnotách existuje smysluplné uspořádání a metrika (např. počet strojů, věk).
- **Nominální atributy** — hodnoty jsou pouhé kódy (barva, typ komponenty). Zde nemá smysl mluvit o „blízkých“ hodnotách.

Mutace.

- *Nezaujatá* (*unbiased, random resetting*): nová hodnota je náhodná z celé domény. Jediná korektní volba pro nominální atributy.
- *Zaujatá* (*biased, creep mutation*): nová hodnota je původní posunutá o malý náhodný krok (např. z diskretizovaného normálního rozdělení). Vhodná jen pro ordinální atributy.

Křížení. Jednobodové, vícebodové, uniformní — stejně jako u binární reprezentace, jen se místo bitů kopírují celá čísla.

3.2 Reálná reprezentace

Jedinec je $\vec{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$. Historicky se reálná čísla kodovala do bitových řetězců; dnes se pracuje přímo s reálnými hodnotami a operátory se tomu přizpůsobují.

Mutace.

- **Gaussovská (normální) mutace:** $x'_i = x_i + \sigma_i \cdot N(0, 1)$. Parametr σ (šířka kroku) je klíčový; může být pevný, deterministicky klesající (např. $\sigma(t) = 1 - 0,9t/T$), adaptivní (pravidlo 1/5) nebo samoadaptivní (součást genomu — viz kap. 6).
- **Cauchyho mutace:** těžší chvosty $\Rightarrow$ častější velké skoky $\Rightarrow$ lepší únik z lokálních extrémů (používá se ve *fast EP*).
- **Uniformní mutace:** x'_i náhodně z $[a_i, b_i]$ — nezaujatá.
- Ošetření mezí: oříznutí (*clipping*), odraz (*reflection*), periodické zabalení (*wrapping*) nebo převzorkování.

Křížení. Rozlišujeme dva typy:

- **Strukturální (diskrétní):** 1-bodové, uniformní — hodnoty se jen přeskupují mezi rodiči; potomek je vrcholem hyperkvádrů daného rodiče.
- **Aritmetické:** hodnoty se kombinují.

Aritmetické křížení (*arithmetic crossover*)

Pro rodiče $\vec{x}, \vec{y}$ a $\alpha \in (0, 1)$:

$$\vec{z} = \alpha \vec{x} + (1 - \alpha) \vec{y} \quad (\text{konvexní kombinace}).$$

Varianty: *jednoduché* (kříží se všechny složky — nejběžnější), *jednoduché aritmetické* (jen jedna náhodně vybraná složka), *celé/single* (kombinace s jednobodovým křížením: za bodem řezu se aritmeticky kombinuje). Pro $\alpha = 1/2$ jde o prostý průměr rodičů.

Příklad. $\vec{x} = (2,0; -1,0; 4,0)$, $\vec{y} = (0,0; 3,0; 1,0)$, $\alpha = 0,25$:

$$\vec{z} = 0,25 \vec{x} + 0,75 \vec{y} = (0,5; 2,0; 1,75).$$

Nevýhoda čistě aritmetického křížení: potomci leží uvnitř konvexního obalu rodičů, takže rozptyl populace nutně klesá („smršťování“). Proto se používají rozšířené varianty:

- **BLX- α** (*blend crossover*): z_i se losuje uniformně z intervalu $[\min - \alpha I, \max + \alpha I]$, kde $I = |x_i - y_i|$; typicky $\alpha = 0,5$. Umožňuje potomkům ležet i vně rodičů.
- **SBX** (*simulated binary crossover*): napodobuje chování jednobodového binárního křížení v reálném prostoru; potomci jsou rozděleni symetricky kolem rodičů s hustotou řízenou parametrem η . Standardní operátor v NSGA-II.

3.3 Permutační reprezentace

Jedinec je permutace π na $\{1, \dots, n\}$. Typické úlohy: obchodní cestující (TSP), rozvrhování, přiřazovací úlohy, uspořádání operací. Klasické operátory zde selhávají — výsledek jednobodového křížení dvou permutací obecně není permutace. Je proto třeba operátory, které *zachovávají přípustnost* a zároveň dědí něco smysluplného. Které informace mají být zděděny, závisí na úloze:

operátor	co zachovává	typické použití
PMX	absolutní pozice měst	úlohy s významnou pozicí
OX	relativní pořadí	TSP (cyklické cesty)
CX	absolutní pozice (cykly)	úlohy s významnou pozicí
ER	hrany (sousednosti)	TSP — nejlepší kvalita

3.3.1 PMX — částečně mapované křížení

PMX (*partially mapped crossover*, Goldberg)

Dvoubodový operátor. (1) Vyber úsek mezi dvěma body řezu a vyměň jej mezi rodiči. (2) Z vyměněných úseků odvoď *mapování* odpovídajících si hodnot. (3) Zbylé pozice zkopíruj z původního rodiče; kde by vznikl konflikt (hodnota už je v potomkovi), použij mapování (případně opakovně).

Příklad. $P_1 = (123|4567|89)$, $P_2 = (452|1876|93)$.

1. Výměna úseků: $O_1 = (\dots|1876|\dots)$, $O_2 = (\dots|4567|\dots)$.
2. Mapování z úseků: $1 \leftrightarrow 4$, $8 \leftrightarrow 5$, $7 \leftrightarrow 6$, $6 \leftrightarrow 7$.
3. Doplnění bezkonfliktních pozic z P_1 do O_1 : $O_1 = (.23|1876|.9)$ (hodnoty 1 a 8 by kolidovaly).

4. Konflikty vyřešíme mapováním: $1 \rightarrow 4, 8 \rightarrow 5$, tedy $O_1 = (423|1876|59)$. Analogicky $O_2 = (182|4567|93)$.

3.3.2 OX — křížení zachovávající pořadí

OX (*order crossover*, Davis)

Dvoubodový operátor zachovávající *relativní pořadí*. (1) Zkopíruj úsek mezi body řezu z prvního rodiče. (2) Ze druhého rodiče čti hodnoty cyklicky počínaje pozicí za druhým bodem řezu a vynech ty, které už v potomkovi jsou. (3) Zbylé hodnoty vkládej do volných pozic potomka, opět cyklicky od pozice za druhým bodem řezu.

Příklad. $P_1 = (123|4567|89)$, $P_2 = (452|1876|93)$.

1. O_1 převezme úsek z P_1 : $O_1 = (\dots|4567|\dots)$.
2. Z P_2 čteme cyklicky od pozice 8: 9, 3, 4, 5, 2, 1, 8, 7, 6.
3. Vyškrtneme hodnoty už obsažené (4, 5, 6, 7): zbývá 9, 3, 2, 1, 8.
4. Doplňujeme od pozice 8 cyklicky (pozice 8, 9, 1, 2, 3): $O_1 = (218|4567|93)$.
5. Symetricky (úsek z P_2 , pořadí z P_1): $O_2 = (345|1876|92)$.

Princip: vnitřní úsek se převezme beze změny, zbytek se doplní v relativním pořadí druhého rodiče s vynecháním duplicit. Pro TSP je OX vhodnější než PMX, protože u okružní jízdy nezáleží na absolutní pozici města, ale na pořadí.

3.3.3 CX — cyklické křížení

CX (*cyclic crossover*, Oliver)

Zachovává *absolutní pozici*: každý gen potomka pochází ze stejné pozice u některého z rodičů. Sestrojí se cykly: začni na pozici 1, vezmi hodnotu z P_1 ; podívej se, jaká hodnota je na téže pozici v P_2 , najdi ji v P_1 a pokračuj, dokud se cyklus neuzavře. Sudé cykly ber z jednoho rodiče, liché z druhého.

Příklad. $P_1 = (123456789)$, $P_2 = (412876935)$. Začneme hodnotou 1 z P_1 na pozici 1. Pod ní je v P_2 hodnota 4, tu najdeme v P_1 na pozici 4 $\Rightarrow$ přidáme; pod ní je 8, ta je v P_1 na pozici 8 $\Rightarrow$ přidáme; pod ní je 3 (pozice 3), pod ní 2 (pozice 2), pod 2 je 1 — cyklus se uzavřel. Máme $O_1 = (1\ 2\ 3\ 4\ _\ _\ 8\ _)$; zbytek doplníme z P_2 : $O_1 = (123476985)$, analogicky $O_2 = (412856739)$.

3.3.4 ER — rekombinace hran

Pozorování: PMX, OX i CX zachovávají jen asi 60 % hran rodičů. Pro TSP je ale právě *hrana* nositelem kvality. *Edge recombination* (Whitley) proto:

1. Pro každé město sestaví *seznam sousedů* ze **obou** rodičů.
2. Začne v náhodném (nebo prvním) městě.
3. Dalším městem je ten dosud nenavštívený soused, který má *nejméně* zbývajících sousedů (heuristika: nenechávat města „osamocená“); při shodě rozhodne náhoda.
4. Pokud seznam sousedů dojde prázdný, pokračuje se náhodným nenavštíveným městem (nastává v 1–1,5 % případů).

ER2 navíc značí hrany, které se vyskytují v *obou* rodičích (symbolem #), a upřednostňuje je při výběru.

Příklad. $P_1 = (123456789)$, $P_2 = (412876935)$ (cyklické cesty). Seznamy sousedů: 1: 9, 2, 4; 2: 1, 3, 8; 3: 2, 4, 9, 5; 4: 3, 5, 1; 5: 4, 6, 3; 6: 5, 7, 9; 7: 6, 8; 8: 7, 9, 2; 9: 8, 1, 6, 3. Start v 1: kandidáti 9 (4 sousedi), 2 (3), 4 (3) — 9 vypadává, mezi 2 a 4 losujeme, vyjde 4. Sousedé 4 jsou 3 a 5; 3 má ještě 3 volné sousedy, 5 jen 2, bereme tedy 5, atd. — výsledek např. (145678239).

Technická poznámka: korektně se navštívené město nejprve odstraní ze *všech* seznamů a teprve pak se počítají zbývající sousedé (výše jsou u startu uvedeny počty ještě *před* odebráním jedničky: po odebrání by bylo 9:3, 2:2, 4:2). Na pořadí kandidátů to zde nic nemění, ale při vlastním počítání na papíře je třeba postupovat takto, jinak vycházejí jiné remízy.

3.3.5 Mutace permutací

- **Swap:** prohození dvou náhodných měst.
- **Insert:** vyjmutí města a vložení jinam (posun zbytku).
- **Inverze (2-opt krok):** obrácení souvislého úseku — pro TSP nejúčinnější, protože mění jen dvě hrany.
- **Scramble:** náhodné přeházení souvislého úseku.
- **Výměna podcest** a heuristické mutace (2-opt, 3-opt, Lin-Kernighan).

3.4 Další reprezentace

Stromy (kap. 8), grafy a DAG (Cartesian GP, neuroevoluce), matice (rozvrhy, TSP jako matice předchůdců), konečné automaty (EP), seznamy pravidel (LCS, kap. 12), agenti umělého života.

3.5 Shrnutí k zkoušce

- Reprezentace + operátory = fitness krajina. Pro nominální atributy jen nezaujaté mutace, pro ordinální i zaujaté (creep).
- Reálná reprezentace: gaussianská mutace $x' = x + \sigma N(0, 1)$; aritmetické křížení $\alpha \vec{x} + (1 - \alpha) \vec{y}$ (zmenšuje rozptyl), proto BLX- α a SBX.
- Permutace: PMX (pozice + mapování), OX (relativní pořadí), CX (absolutní pozice pomocí cyklů), ER (hrany, nejlepší pro TSP). Umět předvést PMX a OX na konkrétním osmiprvkovém příkladu!
- Mutace permutací: swap, insert, inverze (2-opt), scramble.

4 Teorie schémat

Teorie schémat je první pokus formálně vysvětlit, *proč* genetické algoritmy fungují. Pochází od Hollanda (1975) a popularizoval ji Goldberg (1989). Dnes je považována za historicky významnou, ale nedostatečnou; přesto je standardní součástí zkoušky a její kritika je stejně důležitá jako věta sama.

4.1 Schéma, jeho řád a definující délka

Schéma (*schema*)

Nechť jedinec je slovo délky m nad abecedou $\{0, 1\}$. **Schéma** S je slovo délky m nad abecedou $\{0, 1, *\}$, kde $*$ znamená „na hodnotě nezáleží“ (*don't care*). Schéma reprezentuje množinu všech jedinců, kteří se s ním shodují na všech pevných pozicích; schéma s r hvězdičkami reprezentuje 2^r jedinců.

Příklad: schéma $1*0*$ reprezentuje $\{1000, 1001, 1100, 1101\}$; schéma $**\dots*$ reprezentuje celý prostor.

Řád, definující délka, fitness schématu

Pro schéma S délky m definujeme:

- **Řád** $o(S)$ = počet pevných pozic (počet nul a jedniček).
- **Definující délka** $d(S)$ = vzdálenost mezi první a poslední pevnou pozicí ($d(S) = 0$ pro schéma s jedinou pevnou pozicí).
- **Fitness schématu** $F(S, t)$ = průměrná fitness jedinců v populaci $P(t)$, kteří schéma S reprezentují. *Pozor:* tato hodnota závisí na aktuální populaci, nikoli jen na S a f .

Příklad. Pro $S = 1**0*1$ ($m = 6$) je $o(S) = 3$ (pozice 1, 4, 6) a $d(S) = 6 - 1 = 5$.

Kombinatorika schémat.

- Schémat délky m je celkem 3^m .
- Jeden jedinec délky m je reprezentantem 2^m schémat (na každé pozici buď jeho bit, nebo *).
- Populace n jedinců reprezentuje mezi 2^m a $n \cdot 2^m$ schémat (podle míry jejich podobnosti).

4.2 Hollandova věta o schématech

Věta 4.1 (Věta o schématech (TST), Holland 1975). *V jednoduchém genetickém algoritmu s fitness-proporcionální selekcí, jednobodovým křížením s pravděpodobností p_C a bitovou mutací s pravděpodobností p_M platí pro očekávaný počet reprezentantů schématu S v následující generaci:*

$$\mathbb{E}[C(S, t+1)] \geq C(S, t) \frac{F(S, t)}{\bar{f}(t)} \left[1 - p_C \frac{d(S)}{m-1} - p_M o(S) \right].$$

*Slovně: **krátká** (malá definující délka), **nízkého řádu** a **nadprůměrná** schémata se v po sobě jdoucích generacích množí exponenciálně.*

4.2.1 Odvození

Označme $C(S, t)$ počet jedinců v $P(t)$ reprezentujících S , $n = |P(t)|$, $\bar{f}(t)$ průměrnou fitness populace. Postupujeme ve třech krocích podle operátorů.

Krok 1 — selekce. Pravděpodobnost výběru jedince v ruletou je $p_s(v) = f(v)/F(t)$, kde $F(t) = \sum_{u \in P(t)} f(u)$. Pravděpodobnost, že vybraný jedinec reprezentuje S , je

$$p_s(S) = \sum_{v \in S \cap P(t)} \frac{f(v)}{F(t)} = \frac{C(S, t) F(S, t)}{F(t)}.$$

Při n nezávislých tazích tedy

$$\mathbb{E}[C(S, t+1)] = n p_s(S) = C(S, t) \frac{F(S, t)}{\bar{f}(t)}, \quad \bar{f}(t) = F(t)/n.$$

Je-li schéma nadprůměrné o ε , tj. $F(S, t) = \bar{f}(t) (1 + \varepsilon)$ a tento poměr se udržuje v čase, dostáváme

$$C(S, t+1) = C(S, t)(1 + \varepsilon) \implies C(S, t) = C(S, 0) (1 + \varepsilon)^t,$$

tedy **exponenciální růst** (resp. pokles pro $\varepsilon < 0$).

Krok 2 — křížení. Jednobodové křížení volí bod řezu z $m - 1$ možností. Schéma je *jistě* rozbito, padne-li řez mezi jeho první a poslední pevnou pozici, tj. s pravděpodobností nejvýše $d(S)/(m - 1)$ (nejvýše proto, že i při řezu uvnitř může být schéma obnoveno, pokud druhý rodič obsahuje tytéž hodnoty). Křížení se provádí s pravděpodobností p_C , takže

$$p_{\text{surv}}^{\text{cross}}(S) \geq 1 - p_C \frac{d(S)}{m-1}.$$

Krok 3 — mutace. Schéma přežije, pokud se nezmění žádná z jeho $o(S)$ pevných pozic:

$$p_{\text{surv}}^{\text{mut}}(S) = (1 - p_M)^{o(S)} \approx 1 - p_M o(S) \quad \text{pro } p_M \ll 1.$$

Složení. Za předpokladu (přibližné) nezávislosti obou destruktivních jevů a zanedbáním jejich součinu:

$$\mathbb{E}[C(S, t+1)] \geq C(S, t) \frac{F(S, t)}{\bar{f}(t)} \left[1 - p_C \frac{d(S)}{m-1} - p_M o(S) \right]. \quad \blacksquare$$

4.2.2 Číselný příklad

Vezměme Goldbergovu populaci ze str. 4 ($m = 5, n = 4$): 01101 (169), 11000 (576), 01000 (64), 10011 (361); $\bar{f} = 292,5$. Parametry $p_C = 0,7, p_M = 0,001$.

schéma S	reprezentanti	$F(S)$	$o(S), d(S)$	přežití	$\mathbb{E}[C(S, t+1)]$
1****	11000, 10011	468,5	1; 0	$1 - 0 - 0,001 = 0,999$	$2 \cdot 1,602 \cdot 0,999 = 3,2$
1***1	10011	361	2; 4	$1 - 0,7 - 0,002 = 0,298$	$1 \cdot 1,234 \cdot 0,298 = 0,37$
0****	01101, 01000	116,5	1; 0	0,999	$2 \cdot 0,398 \cdot 0,999 = 0,80$

Interpretace: krátké nadprůměrné schéma 1**** se rozšíří z 2 na cca 3,2 zástupce. Schéma 1***1 je sice nadprůměrné ($F/\bar{f} = 1,23$), ale jeho velká definující délka je fatální — křížení ho rozbije a očekáváme jeho vymizení. Podprůměrné 0**** klesá. To je přesně obsah věty.

4.3 Hypotéza stavebních bloků

Hypotéza stavebních bloků (*building blocks hypothesis*)

Genetický algoritmus hledá dobré řešení tak, že **rekombinuje krátká, nízkého řádu a nadprůměrná schémata** — tzv. *stavební bloky* — do stále lepších řešení. Goldbergova metafora: „tak jako dítě staví hrad z jednoduchých dřevěných kostek, tak GA hledá téměř optimální řešení skládáním stavebních bloků“.

Praktické důsledky:

- **Na kódování záleží.** Geny, které spolu interagují, mají ležet blízko sebe (*linkage*), jinak je křížení rozbíjí. Odtud snahy o inverzi, *messy GA*, *linkage learning* a algoritmy odhadu rozdělení (EDA).
- **Na velikosti populace záleží.** Populace musí obsahovat dostatek vzorků od každého stavebního bloku (teorie *population sizing*).
- **Předčasná konvergence škodí** — ztráta diverzity znemožní objevení chybějících bloků.

4.4 Implicitní paralelismus a víceruký bandita

Implicitní paralelismus (*implicit parallelism*)

GA pracuje explicitně s n jedinci, ale implicitně současně „zpracovává“ mnohem více schémat — mezi 2^m a $n \cdot 2^m$. Holland odhadl, že počet schémat, která jsou zpracovávána *užitečně* (rostou exponenciálně a nejsou příliš rozbíjena), je řádu n^3 .

Bývá to komentováno jako jediný případ, kdy kombinatorická exploze pracuje v náš prospěch.

Souvislost s úlohou o banditovi. Hollandova původní motivace byla „adaptivní plán“ hledající rovnováhu mezi exploraací a exploatací. Formálně:

- **Dvouruký bandita.** Máme N mincí a automat se dvěma pákami s neznámými středními výplatami μ_1, μ_2 a rozptyly. Alokujeme n mincí horší a $N - n$ lepší páce. Analytické řešení říká, že optimální strategie alokuje *exponenciálně* více pokusů momentálně vítězné páce: $N - n^* = O(e^{cn^*})$.

- **GA jako bandita.** Věta o schématech říká, že GA alokuje exponenciálně více „slotů v populaci“ úspěšnějším schématům — tedy že řeší dilemma explorační/exploatační (téměř) optimálně.
- Původně se soudilo, že SGA hraje jednu hru s 3^m pákami. Ve skutečnosti hraje mnoho her paralelně: schémata soutěží o konkrétní pevné pozice, a schémata řádu k hrají hru s 2^k pákami.

4.5 Kritika teorie schémat

4.5.1 Kdy GA selhává — deceptivní úlohy

Uvažujme funkci, kde jsou schémata (111*****) a (*****11) nadprůměrná, ale

$$f(111*****11) \ll f(000*****00).$$

GA je selekcí veden k jedničkovým blokům, přestože optimum leží jinde. Takové úlohy se nazývají **deceptivní** (*deceptive*, Goldberg 1987): dílčí stavební bloky nevedou po složení ke globálnímu optimu. Deception bývá navrhována jako míra obtížnosti úlohy pro EA (ne všichni s tím souhlasí).

Opačným testem jsou **Royal Road funkce** (Mitchell, Forrest, Holland): fitness je součtem příspěvků za kompletní „bloky“ jedniček (např. osm bloků po osmi bitech), krajina je tedy ušitá přesně na míru hypotéze stavebních bloků a GA by na ní měl excelovat. Experiment dopadl opačně: jednoduchý GA prohrál s náhodně mutujícím horolezeckým algoritmem (RMHC). Příčinou je *hitchhiking* („stopování“): spolu s nalezeným dobrým blokem se selekcí šíří i nahodilý „parazitní“ bity na ostatních pozicích a ničí diverzitu potřebnou k nalezení zbývajících bloků. Royal Road funkce jsou proto standardním empirickým protipříkladem k naivnímu čtení BBH.

4.5.2 Statické průměrné fitness — statická BBH

Grefenstette (1991): lidé si BBH vykládají tak, že GA konverguje k řešení s dobrou *skutečnou* *statickou* průměrnou fitness schématu (přes celý prostor). GA však odhaduje fitness schématu jen ze vzorků v populaci, a ten odhad je vychýlený ze dvou důvodů:

- **Kolaterální konvergence** (*collateral convergence*). Jakmile GA někam konverguje, přestává vzorkovat schémata rovnoměrně. Je-li např. schéma $111* \dots *$ dobré, po několika generacích má skoro každý jedinec tento prefix. Pak je ale téměř každý vzorek schématu $***000 \dots$ zároveň vzorkem $111000 \dots$ a fitness prvního schématu je odhadnuta zcela zkresleně.
- **Velký rozptyl fitness schématu.** Uvažujme $f(x) = 2$ pro $x \in 111* \dots *$, $f(x) = 1$ pro $x \in 0* \dots *$ a 0 jinak. Statická průměrná fitness schématu $1* \dots *$ je $\approx 1/2$, zatímco $F(0* \dots *) = 1$. GA však odhadne $F(1* \dots *) \approx 2$, protože v populaci převládají jedinci s prefixem 111. Vysoký rozptyl vede k systematickému zkreslení a k tomu, že GA nevzorkuje schémata nezávisle.

4.5.3 Souhrn slabin

- Věta je **nerovnost** a pouze **jednokrokový** výsledek. Její iterace na více generací vyžaduje předpoklad, že $F(S, t)/\bar{f}(t)$ zůstává konstantní — což typicky neplatí (obojí se dynamicky mění).
- Populace jsou **konečné a malé** $\Rightarrow$ výběrové chyby; exponenciální růst je jen střední hodnota.
- Věta zohledňuje jen **destruktivní** účinek operátorů, nikoli jejich *konstruktivní* roli (křížení může schéma i vytvořit).
- Soutěž schémat je **silně závislá**, nikoli nezávislá jako páky bandity.
- Analýza je „on-line“: SGA maximalizuje průběžný výkon, hodí se tedy spíše pro adaptivní úlohy, zatímco nejběžnější použití je nalezení jednoho nejlepšího řešení (off-line kritérium).

Přesto zůstává TST užitečnou intuicí: *co je krátké, jednoduché a dobré, to se šíří.*

4.6 Shrnutí k zkoušce

- Schéma = slovo nad $\{0, 1, *\}$; $o(S)$ = počet pevných pozic; $d(S)$ = vzdálenost první a poslední pevné pozice; $F(S)$ = průměrná fitness reprezentantů v dané populaci. Celkem 3^m schémat, jedinec je reprezentantem 2^m z nich.

- TST: $\mathbb{E}[C(S, t+1)] \geq C(S, t) \frac{F(S)}{f} [1 - p_C \frac{d(S)}{m-1} - p_M o(S)]$; odvození ve třech krocích (selekce $\rightarrow$ exponenciální růst, křížení $\rightarrow d(S)/(m-1)$, mutace $\rightarrow (1 - p_M)^{o(S)}$).
- BBH: GA skládá krátká, nízkého řádu, nadprůměrná schémata (stavební bloky). Důsledky: záleží na kódování, na velikosti populace, škodí předčasná konvergence.
- Implicitní paralelismus ($\sim n^3$ užitečně zpracovaných schémat), analogie s k -rukým banditou (exponenciální alokace pokusů vítězi).
- Kritika: nerovnost pro jeden krok, konečné populace, deceptivní úlohy, kolaterální konvergence, velký rozptyl fitness, ignorování konstruktivních efektů, závislost schémat.

5 Pravděpodobnostní modely jednoduchého GA

V 90. letech (Vose, Liepins, Nix, Whitley) vznikly *exaktní* modely SGA, které nahradily přibližnou teorii schémat. Cílem bylo: popsat přesně, jak vypadá populace, popsat zobrazení přechodu mezi populacemi, zkoumat jeho vlastnosti a asymptotické chování SGA. Rozlišují se dva přístupy: **neko-****nečné populace** (deterministický dynamický systém) a **konečné populace** (Markovův řetězec).

5.1 Zjednodušený SGA

Pro analýzu se SGA ještě zjednoduší: v každém kroku se vyberou dva rodiče, s pravděpodobností p_C se zkříží, **jeden z potomků se zahodí**, zbylý zmutuje a vloží se do nové populace. Tím je tvorba každého jedince nové populace nezávislá.

5.2 Formalizace populace

Každý binární řetězec délky l ztotožníme s číslem $i \in \{0, 1, \dots, 2^l - 1\}$ (např. 00000111 $\equiv$ 7). Populaci v čase t popisují dva vektory délky 2^l :

Vektor populace a vektor selekce

- $p(t) = (p_0(t), \dots, p_{2^l-1}(t))$, kde $p_i(t)$ je *relativní četnost* řetězce i v populaci. Platí $p_i \geq 0$, $\sum_i p_i = 1$, tedy $p(t)$ leží v simplexu Λ .
- $s(t) = (s_0(t), \dots, s_{2^l-1}(t))$, kde $s_i(t)$ je pravděpodobnost, že selekce vybere řetězec i .

Vektor p popisuje *složení* populace, vektor s *selekční pravděpodobnosti*.

5.3 Operátor $\mathcal{G}$

Cílem je najít operátor $\mathcal{G} : \Lambda \rightarrow \Lambda$ takový, že

$$p(t+1) = \mathcal{G}(p(t)),$$

a rozložit ho na dvě části: $\mathcal{G} = \mathcal{M} \circ \mathcal{F}$, kde $\mathcal{F}$ odpovídá selekci (fitness) a $\mathcal{M}$ tzv. míchání (*mixing*) — křížení a mutaci dohromady. Pak stačí operátor iterovat z počáteční populace a známe úplné chování SGA.

Část $\mathcal{F}$ — selekce. Definujme diagonální matici $\mathbf{F}$ s $\mathbf{F}_{ii} = f(i)$ a $\mathbf{F}_{ij} = 0$ pro $i \neq j$. Fitness-proporcionální selekce je pak přesně

$$s(t) = \frac{\mathbf{F} p(t)}{\sum_j \mathbf{F}_{jj} p_j(t)}.$$

Čitatel $(\mathbf{F}p)_i = f(i)p_i$ je „váha“ řetězce i v populaci, jmenovatel je normalizační konstanta (průměrná fitness populace). Vypustíme-li nyní křížení a mutaci, je nová populace prostě n nezávislých tahů podle rozdělení $s(t)$, takže $\mathbb{E}[p(t+1)] = s(t)$. Dosazením do definice selekce dostáváme $\mathbb{E}[s(t+1)] \propto \mathbf{F} s(t)$, tedy iterování $\mathcal{F}$ je (až na normalizaci) opakované násobení maticí $\mathbf{F}$ — *mocninná metoda*. Ta konverguje k vlastnímu vektoru příslušnému největší diagonální hodnotě, tj. k populaci

složené výhradně z kopií nejlepšího jedince *přítomného v počáteční populaci*. U konečných populací způsobují statistické chyby odchylky od střední hodnoty; čím větší populace, tím menší odchylka. Nekonečné populace jsou exaktní, byť nepraktické.

Mini-příklad. Pro $l = 2$ máme čtyři řetězce 00, 01, 10, 11, tedy indexy 0, 1, 2, 3. Necht' $f = (1, 2, 3, 4)$ a populace osmi jedinců obsahuje 2×00 , 4×01 , 1×10 , 1×11 , tj. $p = (0,25; 0,5; 0,125; 0,125)$. Pak $Fp = (0,25; 1,0; 0,375; 0,5)$, součet je 2,125 (= průměrná fitness populace) a

$$s = (0,118; 0,471; 0,176; 0,235).$$

Podíl nejlepšího řetězce 11 vzrostl z 12,5 % na 23,5 %, podíl nejhoršího 00 klesl z 25 % na 11,8 % — přesně o faktor $f(i)/\bar{f}$, jak víme z věty o schématech.

Část $\mathcal{M}$ — míchání. Zavedeme funkci

$$r(i, j, k) = \Pr[\text{potomek } k \text{ vznikne z rodičů } i, j],$$

která v sobě zahrnuje jak křížení (přes všechny body řezu), tak mutaci. Pak

$$\mathbb{E}[p_k(t+1)] = \sum_i \sum_j s_i(t) s_j(t) r(i, j, k).$$

Jde tedy o *kvadratický* operátor na simplexu. Odvození $r(i, j, k)$ je technicky náročné, ale možné. Klíčové zjednodušení plyne z toho, že jak jednobodové křížení, tak bitová mutace „nevidí“ absolutní hodnoty bitů, jen jejich shody a neshody. Formálně: pro libovolné k platí

$$r(i, j, k) = r(i \oplus k, j \oplus k, 0),$$

kde $\oplus$ je bitový XOR. Stačí tedy znát jedinou *míchací matici* $\mathbf{M}$ s prvky $\mathbf{M}_{ij} = r(i, j, 0)$ (rozměru $2^l \times 2^l$) a zbytek se dopočte permutacemi $\sigma_k : x \mapsto x \oplus k$: $\mathcal{M}(s)_k = (\sigma_k s)^\top \mathbf{M}(\sigma_k s)$. To je hlavní technický přínos Voseho formalismu — z 2^{3l} čísel se úloha smrskne na 2^{2l} .

5.4 Výsledky pro nekonečné populace

- SGA je **diskrétní dynamický systém** na simplexu; posloupnost $p(0), p(1), \dots$ je trajektorie tohoto systému.
- **Pevné body $\mathcal{F}$:** populace tvořené pouze kopiemi nejlepšího jedince z počáteční populace (selekce „zaostřuje“).
- **Pevný bod $\mathcal{M}$:** jediný — rovnoměrné rozdělení (míchání „rozostřuje“, maximalizuje entropii).
- Pevné body složeného $\mathcal{G}$ nejsou obecně známy; jejich hledání je hlavní otevřený problém. Chování je kombinací zaostřování a rozostřování, které se projevuje jako **přerušované rovnováhy** (*punctuated equilibria*) — dlouhá období metastability střídaná rychlými přechody. Tento jev je znám i z biologie.

5.5 Konečné populace: Markovův řetězec

Markovův řetězec

Stochastický proces v diskrétním čase se stavy, kde pravděpodobnost přechodu do dalšího stavu závisí *jen* na aktuálním stavu (nikoli na historii). Úplným popisem je matice přechodových pravděpodobností.

Modelujeme SGA s populací velikosti n nad řetězcí délky l :

- **Stavy** = jednotlivé možné populace (multimnožiny n řetězců). Jejich počet je

$$N = \binom{n + 2^l - 1}{2^l - 1},$$

což je počet multimnožin velikosti n z 2^l typů.

- **Matice $\mathbf{Z}$:** sloupce odpovídají populacím, $\mathbf{Z}(y, i)$ = počet výskytů jedince y v populaci i . Sloupce matice $\mathbf{Z}$ jsou tedy stavy řetězce.
- **Matice přechodů $\mathbf{Q}$** rozměru $N \times N$; lze ji explicitně odvodit z $\mathcal{G}$ (přechod je multinomické losování n jedinců podle rozdělení $\mathcal{G}(p)$):

$$\mathbf{Q}_{i,j} = n! \prod_{y=0}^{2^l-1} \frac{(\mathcal{G}(p_i)_y)^{\mathbf{Z}(y,j)}}{\mathbf{Z}(y,j)!}.$$

Problém velikosti. Už pro $n = l = 8$ má $\mathbf{Q}$ řádivě 10^{29} prvků. Model je tedy exaktní, ale prakticky nepoužitelný pro výpočet; jeho hodnota je teoretická.

Kvalitativní důsledky.

- Je-li $p_M > 0$, je řetězec **ireducibilní a aperiodický** (z libovolné populace lze mutacemi dosáhnout libovolné jiné) $\Rightarrow$ existuje jediné stacionární rozdělení a SGA navštíví každou populaci (tedy i tu s globálním optím) nekonečně mnohokrát s pravděpodobností 1.
- Bez elitismu ale **nekonverguje** k optimu — optimum se objeví a zase zmizí. *Elitistický* GA (uchovávající nejlepší nalezené řešení) konverguje k globálnímu optimu s pravděpodobností 1 při $t \rightarrow \infty$. To je standardní konvergenční výsledek; neříká však nic o *rychlosti*.
- Existuje korespondence mezi modelem nekonečné a konečné populace: pro $n \rightarrow \infty$ se chování konečného modelu blíží deterministické trajektorii $\mathcal{G}$.

5.6 Algoritmy odhadu rozdělení (EDA)

Voseho model ukazuje, že SGA je ve skutečnosti stroj na transformaci pravděpodobnostního rozdělení nad prostorem řetězců. *Algoritmy odhadu rozdělení* (*estimation of distribution algorithms*, EDA) berou toto pozorování doslova: zahodí křížení i mutaci a pracují s modelem přímo.

EDA — obecné schéma

Namísto operátorů se udržuje **explicitní pravděpodobnostní model** $p_\theta(x)$ nad prostorem řešení. Iterace: (1) navzorkuj z p_θ populaci, (2) ohodnoť ji a vyber lepší část, (3) *přeúč* model θ na vybraných jedincích, (4) opakuj. Model θ je jediná informace přenášená mezi generacemi.

Algoritmy se liší tím, jak bohaté závislosti mezi geny model umí zachytit:

model	algoritmus	poznámka
bez závislostí	PBIL, UMDA, cGA	vektor $p = (p_1, \dots, p_m)$ pravděpodobností jedničky na každé pozici
párové závislosti	MIMIC, BMMA	řetěz či strom závislostí mezi geny
obecné závislosti	BOA, hBOA, EBNA	Bayesovská síť učená z vybraných jedinců
spojité domény	EMNA, CMA-ES	gaussovské rozdělení, resp. jeho kovarianční matice

UMDA (*univariate marginal distribution algorithm*) prostě spočítá na vybraných jedincích relativní četnost jedničky na každé pozici a z těchto marginál navzorkuje celou novou populaci.

PBIL (*population-based incremental learning*) tentýž vektor neuloží znovu, ale posune ho k nejlepšímu jedinci s učící rychlostí α :

$$p_i \leftarrow (1 - \alpha)p_i + \alpha x_i^{\text{best}}, \quad \alpha \approx 0,1,$$

plus drobná náhodná porucha jako obdoba mutace. Populaci lze pak chápat jen jako pomocný vzorek — „genofond“ je vektor p .

Příklad. Nechť $p = (0,5; 0,5; 0,5)$ a nejlepší jedinec je $x^{\text{best}} = (1, 0, 1)$, $\alpha = 0,2$. Pak $p \leftarrow (0,8 \cdot 0,5 + 0,2; 0,8 \cdot 0,5; 0,8 \cdot 0,5 + 0,2) = (0,6; 0,4; 0,6)$ — model se naklonil ke vzoru 101.

Proč to funguje (a kdy ne). Nezávislé marginály odpovídají přesně tomu, co dělá uniformní křížení na již konvergované populaci, ale bez disruptivity. Jakmile však mezi geny existuje *epistáze* (interakce), univariátní model ji nezachytí a algoritmus selže — proto BOA, který se strukturu závislostí (*linkage*) učí. Tím EDA elegantně řeší problém, kvůli němuž Holland kdysi navrhoval operátor inverze: kde mají spolupracující geny ležet, se algoritmus naučí sám.

5.7 Další teoretické nástroje (přehled)

- **Analýza doby běhu** (*runtime analysis*): asymptotický počet vyhodnocení fitness pro jednoduché algoritmy a funkce. Klasicky (1+1)-EA na funkci OneMax potřebuje $\Theta(m \log m)$ vyhodnocení, na funkci LeadingOnes $\Theta(m^2)$, na „jehle v kupce sena“ exponenciálně. Nástroje: driftová analýza, metoda fitness úrovní.
- **Analýza fitness krajiny**: korelace fitness sousedů, počet lokálních optim, epistáze (interakce genů), NK-krajiny.
- **Exaktní věty o schématech** (Stephens & Waters): rovnosti (nikoli nerovnosti) popisující i *konstruktivní* efekt křížení, tedy přesně to, co Hollandově větě chybí.

5.8 Shrnutí k zkoušce

- Voseho model: řetězce ztotožníme s čísly $0..2^l - 1$; populaci popisuje vektor relativních četností $p(t)$ v simplexu a vektor selekčních pravděpodobností $s(t)$.
- Hledáme $\mathcal{G} = \mathcal{M} \circ \mathcal{F}$ s $p(t+1) = \mathcal{G}(p(t))$. $\mathcal{F}$ (selekce) je dána diagonální maticí fitness: $s = \mathbf{F}p / \sum_j \mathbf{F}_{jj}p_j$; $\mathcal{M}$ (křížení + mutace) je kvadratický operátor $\mathbb{E}[p'_k] = \sum_{i,j} s_i s_j r(i, j, k)$. Díky symetrii $r(i, j, k) = r(i \oplus k, j \oplus k, 0)$ stačí jediná míchací matice.
- Pevné body: $\mathcal{F}$ — populace kopií nejlepšího jedince; $\mathcal{M}$ — rovnoměrné rozdělení. Kombinace vede k přerušovaným rovnováhám. Pevné body $\mathcal{G}$ nejsou známy.
- Konečné populace: Markovův řetězec se stavy = populace, $N = \binom{n+2^l-1}{2^l-1}$ stavů, matice $\mathbf{Q}$ je exaktní, ale astronomicky velká. Ireducibilita při $p_M > 0$; elitistický GA konverguje k optimu s pravděpodobností 1.
- Nekonečné populace = deterministický model (exaktní, nepraktický), konečné = stochastický (realistický, nezvládnutelný). Limitní přechod je spojuje.
- **EDA** berou pohled „GA = transformace rozdělení“ doslova: místo operátorů se učí model p_θ (UMDA/PBIL — nezávislé marginály, MIMIC/BMDA — párové závislosti, BOA — Bayesovská síť, CMA-ES — gaussovka). Cyklus: vzorkuj $\rightarrow$ vyber lepší $\rightarrow$ přeuč model. PBIL: $p_i \leftarrow (1 - \alpha)p_i + \alpha x_i^{\text{best}}$.

6 Evoluční strategie

6.1 Motivace a základní rysy

Evoluční strategie (*evolution strategies*, ES) navrhli Rechenberg a Schwefel v Berlíně v 60. letech pro optimalizaci reálných parametrů v obtížných technických úlohách (tvar trysky, ohyb potrubí). Charakteristiky:

- Jedinec je vektor reálných čísel; **mutace je hlavní operátor**.
- Jedinec obsahuje kromě *genetických parametrů* (vlastní řešení) i *strategické, endogenní parametry* (*endogenous parameters*), které řídí samotnou evoluci — typicky směrodatné odchylky mutací. Odtud slogan „evoluce evoluce“.
- Rodičovská selekce je **náhodná** (nezávislá na fitness); environmentální selekce je **deterministická** — vybírá se μ nejlepších.
- Nejmodernějším a nejúspěšnějším zástupcem je CMA-ES.

Notace ES

μ — velikost populace (počet rodičů), λ — počet vytvořených potomků, ρ — počet rodičů podílejících se na jednom potomkovi.

- $(\mu + \lambda)$ -ES: nová populace se vybírá z μ rodičů i λ potomků (elitistická).
- (μ, λ) -ES: nová populace se vybírá jen z λ potomků; rodiče vždy vymřou (neelitistická), nutně $\lambda > \mu$.
- $(\mu/\rho + \lambda)$ -ES: explicitní zápis s rekombinací ρ rodičů.

	(μ, λ)	$(\mu + \lambda)$
pool pro selekci	jen potomci	rodiče + potomci
elitismus	ne (fitness může klesnout)	ano
únik z lokálních optim	lepší	horší
konvergence	pomalejší	rychlejší
doporučení	spojité domény, zašuměná či dynamická fitness	diskrétní domény, drahá fitness
samoadaptace σ	funguje dobře (špatné σ vymře)	může uváznout (starý rodič s malým σ přežívá)
poměr	typicky $\lambda/\mu \approx 7$	i $\mu \geq \lambda$; $\lambda = 1$ = steady-state ES

6.2 $(1 + 1)$ -ES a pravidlo $1/5$

Nejjednodušší ES: populace je jediný jedinec, vytváří se jediný potomek gaussovskou mutací a přijímá se, jen pokud je lepší.

Algoritmus 3 $(1 + 1)$ -ES s pravidlem $1/5$ (minimalizace $f : \mathbb{R}^n \rightarrow \mathbb{R}$)

```

1: inicializuj  $\vec{x}(0)$  náhodně,  $\sigma > 0$ ,  $t \leftarrow 0$ 
2: repeat
3:    $\vec{y}(t) \leftarrow \vec{x}(t) + \sigma \cdot N(\vec{0}, \mathbf{I})$ 
4:   if  $f(\vec{y}(t)) < f(\vec{x}(t))$  then  $\vec{x}(t+1) \leftarrow \vec{y}(t)$ 
5:   else  $\vec{x}(t+1) \leftarrow \vec{x}(t)$ 
6:   end if
7:   sleduj  $p$  = podíl úspěšných mutací za posledních  $k$  iterací
8:   každých  $k$  iterací uprav  $\sigma$ :
9:     if  $p > 1/5$  then  $\sigma \leftarrow \sigma/c$                                 ▷ daří se: dělej větší kroky, více exploração
10:    if  $p < 1/5$  then  $\sigma \leftarrow \sigma \cdot c$                             ▷ nedaří se: zmenšuj kroky, více exploatace
11:    if  $p = 1/5$  then  $\sigma$  beze změny                                    ▷ typicky  $0,8 \leq c \leq 1$ 
12:     $t \leftarrow t + 1$ 
13: until  $\vec{x}(t)$  je dost dobré

```

Pravidlo $1/5$ úspěchu ($1/5$ success rule)

Heuristika odvozená Rechenbergem z analýzy dvou modelových funkcí (kulová a koridorová): optimální poměr úspěšných mutací je přibližně $1/5$. Je-li podíl úspěchů vyšší, je krok příliš malý (jsme příliš opatrní) a σ se zvětší; je-li nižší, je krok příliš velký a σ se zmenší. Jde o adaptivní (nikoli samoadaptivní) řízení parametru — používá zpětnou vazbu z běhu, ale parametr není součástí genomu.

6.3 Samoadaptace strategických parametrů

Samoadaptace (*self-adaptation*)

Strategické parametry (odchylky σ , případně úhly natočení) jsou **součástí genomu** a podléhají mutaci a selekci spolu s řešením. Selektce je nepřímá: jedinci s vhodným σ produkují lepší potomky, a tím se jejich σ šíří populací. Pořadí je zásadní: **nejprve se mutují strategické parametry, teprve pak se s nimi mutují genetické parametry** — jinak by se hodnotilo staré σ .

Jedinec má tvar $C = [\vec{x}, \vec{\sigma}]$, kde $\vec{x} \in \mathbb{R}^n$ a $\vec{\sigma}$ má k složek:

k	název	geometrie mutace
1	jedno globální σ	izotropní — vrstevnice hustoty jsou koule
n	nekorelované mutace	elipsoid s osami rovnoběžnými se souřadnicemi
$2n$	částečně korelované	n odchylek + n hlavních úhlů natočení
$n(n+1)/2$	korelované mutace	obecný elipsoid (n odchylek + $n(n-1)/2$ úhlů natočení α_{ij}), odpovídá plné kovarianční matici

Kovarianční matice $\mathbf{C}$ je součinem rotační matice: $c_{ii} = \sigma_i^2$, $c_{ij} = \frac{1}{2}(\sigma_i^2 - \sigma_j^2) \tan(2\alpha_{ij})$.

Standardní pravidla mutace (Schwefel).

$$\sigma'_i = \sigma_i \cdot \exp(\tau' N(0, 1) + \tau N_i(0, 1)), \quad x'_i = x_i + \sigma'_i \cdot N_i(0, 1),$$

kde $\tau' \propto 1/\sqrt{2n}$ (společná porucha pro celý jedinec) a $\tau \propto 1/\sqrt{2\sqrt{n}}$ (individuální porucha složky). Lognormální tvar zajišťuje kladnost σ a symetrii zvětšení/zmenšení. Zavádí se dolní mez $\sigma_{\min}$, aby σ nezdegenerovalo na nulu. Úhly se mutují aditivně: $\alpha'_{ij} = \alpha_{ij} + \beta \cdot N(0, 1)$.

6.4 Rekombinace v ES

Z ρ rodičů vzniká *jeden* potomek. Podle počtu rodičů rozlišujeme **lokální** ($\rho = 2$) a **globální** ($\rho = \mu$) rekombinaci; podle způsobu kombinace:

- **Diskrétní (dominantní):** hodnota na dané pozici se náhodně vybere od jednoho z rodičů.
- **Intermediární (aritmetická):** hodnota je průměrem hodnot rodičů na dané pozici.

Obvyklá kombinace: diskrétní rekombinace pro genetické parametry (zachovává rozmanitost) a intermediární pro strategické parametry (stabilizuje adaptaci σ).

6.5 CMA-ES

Covariance Matrix Adaptation ES (Hansen, Ostermeier) je dnes referenční algoritmus pro spojitou black-box optimalizaci. Místo samoadaptace jednotlivých σ v genomu udržuje **jediné rozdělení pro celou populaci** a učí se jeho kovarianční matici z historie úspěšných kroků.

Princip CMA-ES

Stav algoritmu je $\theta = (\vec{m}, \sigma, \mathbf{C}, \vec{p}_\sigma, \vec{p}_c)$ a vzorkovací rozdělení

$$p_\theta(\vec{x}) = \mathcal{N}(\vec{x} | \vec{m}, \sigma^2 \mathbf{C}),$$

kde $\vec{m}$ je střed (těžiště nejlepších jedinců), σ globální délka kroku, $\mathbf{C}$ kovarianční matice určující tvar a orientaci mraku vzorků, a $\vec{p}_\sigma, \vec{p}_c$ jsou *evoluční cesty* — exponenciálně vyhlazené záznamy o tom, kudy se střed v posledních iteracích pohyboval.

Iterace:

Algoritmus 4 Obecný cyklus ES

```
1:  $t \leftarrow 0$ ; inicializuj náhodně populaci  $P_t$  o  $\mu$  jedincích  $[\vec{x}, \vec{\sigma}]$ 
2: ohodnoť jedince v  $P_t$ 
3: while není splněna ukončovací podmínka do
4:    $C_{t+1} \leftarrow \emptyset$ 
5:   for  $\lambda$  krát do
6:     vyber náhodně  $\rho$  rodičů z  $P_t$ 
7:     rekombinuj je do jednoho potomka  $[\vec{x}, \vec{\sigma}]$ 
8:     mutuj strategické parametry  $\vec{\sigma}$ 
9:     mutuj genetické parametry  $\vec{x}$  pomocí nových  $\vec{\sigma}$ 
10:    ohodnoť potomka a vlož do  $C_{t+1}$ 
11:   end for
12:    $(\mu, \lambda)$ :  $P_{t+1} \leftarrow \mu$  nejlepších z  $C_{t+1}$ 
13:    $(\mu + \lambda)$ :  $P_{t+1} \leftarrow \mu$  nejlepších z  $P_t \cup C_{t+1}$ 
14:    $t \leftarrow t + 1$ 
15: end while
```

1. Navzorkuj λ bodů $\vec{x}_i = \vec{m} + \sigma \mathcal{N}(\vec{0}, \mathbf{C})$.
2. Ohodnoť je a vyber μ nejlepších (*rank-based* — CMA-ES používá jen pořadí, je tedy invariantní vůči monotónním transformacím fitness).
3. Nový střed $\vec{m}' = \sum_{i=1}^{\mu} w_i \vec{x}_{i:\lambda}$ (vážený průměr nejlepších).
4. Aktualizuj evoluční cesty úměrně posunu $\vec{m}' - \vec{m}$.
5. **Řízení délky kroku:** je-li $\|\vec{p}_\sigma\|$ větší, než by odpovídalo náhodné procházce, kroky jdou souhlasným směrem $\Rightarrow$ zvětši σ ; jsou-li kroky protichůdné, zmenši σ .
6. **Adaptace $\mathbf{C}$:** *rank-1 update* podle $\vec{p}_c \vec{p}_c^\top$ (podporuje směr dosavadního postupu) a *rank- μ update* podle empirické kovariance vybraných bodů.

Výsledkem je algoritmus, který se automaticky přizpůsobí anizotropii a korelacím úlohy — v podstatě se učí aproximaci inverzní Hessovy matice bez gradientu (*invariance vůči lineárním transformacím prostoru*). Má velmi málo laditelných parametrů (defaulty závisí jen na n).

Zařazení: obecný princip stochastického prohledávání. CMA-ES je speciálním případem obecné šablony: algoritmus udržuje rozdělení $p_\theta(\vec{x})$, v každé iteraci z něj navzorkuje n bodů, ohodnotí je a podle dat aktualizuje θ . Parametr θ je jediná informace přenášená mezi iteracemi. Podle volby θ dostáváme: EA (θ = rodičovská populace), ES (θ = střed a rozptyl gaussovky), EDA (θ = parametry modelu, např. Bayesovské sítě), simulované žíhání (θ = aktuální bod a teplota).

6.6 Shrnutí k zkoušce

- ES: reálné vektory, mutace hlavní operátor, náhodná rodičovská selekce, deterministická environmentální selekce; jedinec = $[\vec{x}, \vec{\sigma}]$.
- (μ, λ) vs. $(\mu + \lambda)$: umět vyjmenovat rozdíly (elitismus, konvergence, lokální optima, samoadaptace).
- $(1 + 1)$ -ES a pravidlo $1/5$: $p > 1/5 \Rightarrow$ zvětši σ , $p < 1/5 \Rightarrow$ zmenši; jde o adaptivní, nikoli samoadaptivní řízení.
- Samoadaptace: $\sigma' = \sigma \exp(\tau' N(0, 1) + \tau N_i(0, 1))$, poté $x' = x + \sigma' N(0, 1)$; **nejdřív σ , pak x** . Počet strategických parametrů: 1 (izotropní), n (nekorelované), $n(n + 1)/2$ (korelované, s úhly natočení).
- Rekombinace: lokální ($\rho = 2$) / globální ($\rho = \mu$), diskrétní (dominantní) / intermediární (aritmetická).
- CMA-ES: $\theta = (\vec{m}, \sigma, \mathbf{C}, \vec{p}_\sigma, \vec{p}_c)$, vzorkování $\mathcal{N}(\vec{m}, \sigma^2 \mathbf{C})$, evoluční cesty, rank-1 a rank- μ update, řízení kroku podle délky $\vec{p}_\sigma$; invariantní vůči monotónním transformacím fitness a lineárním transformacím prostoru.

7 Diferenciální evoluce

7.1 Motivace

Diferenciální evoluce (*differential evolution*, DE; Storn a Price, 1997) je jednoduchý a překvapivě silný algoritmus pro black-box optimalizaci funkcí $f : \mathbb{R}^D \rightarrow \mathbb{R}$. Klíčová myšlenka je **geometrická**: velikost i směr mutačního kroku se neodhaduje z parametru σ , ale přebírá se z *rozdílů dvojic jedinců v populaci*. Populace tak sama kóduje měřítko a orientaci úlohy: na začátku je rozptýlená (velké kroky), v konvergujícím stavu jsou rozdíly malé (jemné doladění). Jde o jakousi implicitní samoadaptaci zdarma.

Aplikace sahají od strojového učení po plánování optimálních trajektorií kosmických sond (Evropská kosmická agentura).

7.2 Algoritmus DE/rand/1/bin

Populace je N vektorů $\vec{x}_1, \dots, \vec{x}_N \in \mathbb{R}^D$. Pro **každého** jedince $\vec{x}_i$ (rodičovská selekce je tedy triviální — rodičem je postupně každý) se provede:

1. **Mutate.** Vyber tři navzájem různé jedince $\vec{x}_a, \vec{x}_b, \vec{x}_c$ různé od $\vec{x}_i$ a vytvoř *dárce* (*donor vector*)

$$\vec{v}_i = \vec{x}_a + F(\vec{x}_b - \vec{x}_c), \quad F \in [0, 2], \text{ typicky } F \approx 0,8.$$

2. **Křížení (binomické, uniformní „s pojistkou“).** Vytvoř *zkušební vektor* (*trial vector*) $\vec{u}_i$:

$$u_{j,i} = \begin{cases} v_{j,i} & \text{pokud } \text{rand}_j \leq C \text{ nebo } j = I_{\text{rand}}, \\ x_{j,i} & \text{jinak,} \end{cases}$$

kde $\text{rand}_j \sim U(0, 1)$, $C \in [0, 1]$ je práh křížení a I_{rand} je náhodný index z $\{1, \dots, D\}$. Podmínka $j = I_{\text{rand}}$ je „pojistka“: zaručuje, že alespoň jedna složka pochází od dárce, takže potomek není nikdy identickou kopií rodiče.

3. **Environmentální selekce (souboj 1:1).**

$$\vec{x}_i(t+1) = \begin{cases} \vec{u}_i & \text{pokud } f(\vec{u}_i) \leq f(\vec{x}_i(t)), \\ \vec{x}_i(t) & \text{jinak.} \end{cases}$$

Selekce je tedy elitistická na úrovni jednotlivce — populace se nikdy nezhoršuje.

Algoritmus 5 DE/rand/1/bin

Vstup: velikost populace N , měřítko F , práh křížení C

```
1: inicializuj populaci  $\{\vec{x}_i\}_{i=1}^N$  náhodně v mezích úlohy
2: while není splněna ukončovací podmínka do
3:   for  $i \leftarrow 1$  to  $N$  do
4:     vyber náhodně různé indexy  $a, b, c \neq i$ 
5:      $\vec{v} \leftarrow \vec{x}_a + F(\vec{x}_b - \vec{x}_c)$ 
6:      $I_{\text{rand}} \leftarrow$  náhodný index z  $\{1, \dots, D\}$ 
7:     for  $j \leftarrow 1$  to  $D$  do
8:        $u_j \leftarrow v_j$  pokud  $\text{rand}() \leq C$  nebo  $j = I_{\text{rand}}$ , jinak  $u_j \leftarrow x_{j,i}$ 
9:     end for
10:    if  $f(\vec{u}) \leq f(\vec{x}_i)$  then  $\vec{x}'_i \leftarrow \vec{u}$ 
11:    else  $\vec{x}'_i \leftarrow \vec{x}_i$ 
12:    end if
13:  end for
14:   $\{\vec{x}_i\} \leftarrow \{\vec{x}'_i\}$ 
15: end while
```

7.3 Číselný příklad jednoho kroku

Nechť $D = 3$, minimalizujeme $f(\vec{x}) = \sum_j x_j^2$, parametry $F = 0,8$, $C = 0,9$. Aktuální jedinec $\vec{x}_i = (4; -3; 2)$, $f(\vec{x}_i) = 16 + 9 + 4 = 29$. Náhodně vybraní jedinci:

$$\vec{x}_a = (2; -1; 0,5), \quad \vec{x}_b = (1; 0; -1), \quad \vec{x}_c = (-1; 2; 1).$$

Mutace:

$$\vec{x}_b - \vec{x}_c = (2; -2; -2), \quad \vec{v} = (2; -1; 0,5) + 0,8(2; -2; -2) = (3,6; -2,6; -1,1).$$

Křížení: nechť $I_{\text{rand}} = 1$ a náhodná čísla $\text{rand} = (0,40; 0,95; 0,20)$. Pak

- $j = 1: 0,40 \leq 0,9$ (a navíc $j = I_{\text{rand}} \Rightarrow u_1 = v_1 = 3,6$;
- $j = 2: 0,95 > 0,9$ a $j \neq I_{\text{rand}} \Rightarrow u_2 = x_{2,i} = -3$;
- $j = 3: 0,20 \leq 0,9 \Rightarrow u_3 = v_3 = -1,1$.

Zkušební vektor je $\vec{u} = (3,6; -3; -1,1)$.

Selekce: $f(\vec{u}) = 12,96 + 9 + 1,21 = 23,17 < 29 = f(\vec{x}_i)$, takže $\vec{u}$ nahrazuje $\vec{x}_i$ v nové populaci.

7.4 Varianty mutace

Zápis DE/ $x/y/z$: x = jak se volí základní vektor, y = počet rozdílových vektorů, z = typ křížení (**bin** = binomické uniformní, **exp** = exponenciální, souvislý úsek).

varianta	dárce $\vec{v}$
DE/rand/1	$\vec{x}_a + F(\vec{x}_b - \vec{x}_c)$
DE/rand/2	$\vec{x}_a + F(\vec{x}_b - \vec{x}_c + \vec{x}_d - \vec{x}_e)$
DE/best/1	$\vec{x}_{\text{best}} + F(\vec{x}_b - \vec{x}_c)$
DE/best/2	$\vec{x}_{\text{best}} + F(\vec{x}_b - \vec{x}_c + \vec{x}_d - \vec{x}_e)$
DE/current-to-best/1	$\vec{x}_i + F(\vec{x}_{\text{best}} - \vec{x}_i) + F(\vec{x}_b - \vec{x}_c)$

Varianty s **best** konvergují rychleji, ale snáze uvíznou v lokálním optimu; **rand** je robustnější. Moderní varianty (jDE, SaDE, JADE, SHADE, L-SHADE) adaptují F a C za běhu a patří ke špičce v soutěžích CEC.

7.5 Srovnání DE s ES a GA

	GA (reálný)	ES	DE
zdroj kroku mutace	pevné σ	evolvované σ	rozdíly jedinců
rodičovská selekce	podle fitness	náhodná	každý jedinec jednou
env. selekce	generační / elitismus	μ nejlepších	souboj rodič vs. potomek
hlavní operátor	křížení	mutace	diferenční mutace
adaptace měřítka	žádná	explicitní (v genomu)	implicitní (z populace)

7.6 Shrnutí k zkoušce

- DE/rand/1/bin: dárce $\vec{v} = \vec{x}_a + F(\vec{x}_b - \vec{x}_c)$; binomické křížení s prahem C a pojistkou $j = I_{\text{rand}}$; souboj potomka s vlastním rodičem (nahrazení jen při zlepšení).
- Parametry: $F \in [0, 2]$ (typ. 0,5–0,9), $C \in [0, 1]$, $N \approx 10D$.
- Geometrická podstata: měřítka a orientace kroků se přebírají z aktuálního rozptylu populace $\Rightarrow$ automatická adaptace bez strategických parametrů.
- Varianty: rand/1, rand/2, best/1, best/2, current-to-best; **bin** vs. **exp** křížení.
- Umět předvést jeden krok s konkrétními čísly (mutace $\rightarrow$ křížení $\rightarrow$ selekce).

8 Genetické a evoluční programování, gramatická evoluce

8.1 Historie

Myšlenku evoluce programů zmínil už Alan Turing v 50. letech. Forsyth (1980, systém BEAGLE) ji aplikoval na rozpoznávání vzorů, Michael Cramer (1985) poprvé popsal stromové jedince a John Koza (1989–1992) formuloval stromové genetické programování v dnešní podobě (včetně patentu).

Genetické programování (*genetic programming*, GP)

Evoluční algoritmus, jehož jedinci jsou **spustitelné struktury** — programy, výrazy, obvody, modely. Fitness se určuje *spuštěním* jedince na testovacích datech. Velikost i tvar jedince nejsou pevné, mění se v průběhu evoluce.

Algoritmus 6 Obecné schéma GP

- 1: vygeneruj počáteční populaci náhodných programů
 - 2: **while** není nalezeno dost dobré řešení **do**
 - 3: ohodnoť programy jejich **spuštěním** na trénovacích datech
 - 4: selekce podle fitness (typicky turnaj)
 - 5: křížení dvou programů, mutace programů
 - 6: vytvoř novou populaci
 - 7: **end while**
-

8.2 Stromové GP

Reprezentace. Program je *syntaktický strom*. Definujeme dvě množiny:

- **Množina terminálů** T — listy: proměnné, konstanty, funkce bez argumentů (např. čtení senzoru).
- **Množina neterminálů (funkcí)** F — vnitřní uzly s danou *aritou*: aritmetické operace, logické spojky, podmínky, cykly.

Požaduje se *uzavřenost* (*closure*) — každá funkce musí umět zpracovat libovolný výstup libovolné jiné (odtud „chráněné dělení“ %, které pro nulového dělitele vrací 1) — a *dostatečnost* (*sufficiency*) — z $T \cup F$ musí být řešení vůbec vyjádřitelné.

Příklad: strom $(+ (* x x) (\sin y))$ reprezentuje výraz $x^2 + \sin y$.

Inicializace.

- **Grow:** uzly se losují z $T \cup F$, dokud se nedosáhne limitu hloubky/počtu uzlů $\Rightarrow$ nepravidelné stromy různé velikosti.
- **Full:** do dané hloubky se losuje jen z F , na poslední úrovni jen z $T \Rightarrow$ plné, pravidelné stromy.
- **Ramped half-and-half** (Koza): polovina populace metodou grow, polovina metodou full, pro rovnoměrně rozložené hloubky (např. 2–6). Standardní volba; vytváří spíše „košaté“ pravidelné tvary, což vyhovuje úlohám se symetriemi, kde jsou všechny vstupy podobně důležité.
- **Uniformní vzorkování** všech možných stromů dává nepravidelnější tvary (některé terminály blíže ke kořeni).
- **Seeding:** vložení částečných či expertních řešení do počáteční populace — zrychluje, ale hrozí předčasná konvergence.

Křížení.

- **Výměna podstromů** (standardní Kozovo křížení): v každém rodiči se zvolí náhodný uzel a podstromy pod ním se vymění. Neterminály mívají vyšší pravděpodobnost výběru (typicky 90 % neterminál, 10 % terminál) — jinak by se často vyměňovaly jen listy.

- **Homologní křížení** se snaží zachovat pozici genetického materiálu. Nejprve se najde *společná oblast* C (procházením od kořene, dokud se shoduje arita uzlů):
 - *Jednobodové*: bod křížení se volí ze společné oblasti.
 - *Uniformní* (GP_{UX}, Poli a Langdon): každý uzel společné oblasti se s konstantní pravděpodobností vymění; uzly *uvnitř* C se vymění bez dopadu na jejich podstromy, uzly na *hranici* C se vymění i s podstromy. V raných fázích se tak vyměňují velké podstromy, s konvergencí populace se operátor stává stále lokálnější.
- **Křížení zachovávající kontext** (*context preserving*): uzel je identifikován *souřadnicemi* — posloupností odboček na cestě od kořene (např. (2, 1, 3, 1)). *Silná* varianta povoluje křížení jen mezi uzly se *stejnými* souřadnicemi, *slabá* je volnější.

Mutace. V GP je nutné (nikoli jen užitečné) používat *více typů* mutací:

- **Podstromová mutace** — nahradí náhodný podstrom novým náhodným.
- **Size-fair** podstromová mutace — velikost nového podstromu se losuje tak, aby odpovídala odstraněnému.
- **Mutace uzlu** (*node replacement*) — záměna uzlu za jiný *stejně arity*.
- **Hoist** — jedinec se nahradí svým podstromem (zmenšuje).
- **Shrink** — podstrom se nahradí terminálem (zmenšuje).
- **Permutační mutace** — prohození podstromů jednoho neterminálu.
- **Mutace konstant** — GP tradičně špatně doladuje číselné hodnoty. Pomáhá aritmetická mutace konstant nebo dokonce lokální optimalizace *všech* konstant ve stromu (hill climbing, gradientní metoda) — memetický přístup.

Fitness a selekce. Fitness = chyba na trénovacích datech (symbolická regrese), počet správných odpovědí, kvalita chování v simulaci. Selektce bývá turnajová. Riziko přeučení je stejné jako u strojového učení (řeší se validační množinou, penalizací složitosti).

8.3 Bloat

Bloat

Bloat je tendence programů v GP růst do velikosti bez odpovídajícího zlepšení fitness. Typicky jde o hromadění *intronů* — částí kódu, které nemají vliv na výstup (např. `(+ x 0)`, `(if false ...)`).

Vysvětlují se tři konkurenční (a vzájemně slučitelné) teorie:

- **Replication accuracy** — introny chrání jedince před destruktivním křížením (větší podíl neutrálních bodů řezu), takže větší jedinci předávají svou fitness spolehlivěji; introny tedy „svezou“ (*hitchhiking*) s dobrým kódem.
- **Removal bias** — odstraněný podstrom musí být malý, aby se jedinci fitness nezhoršila, ale vložený podstrom je libovolně velký; střední velikost proto systematicky roste.
- **Crossover bias** — náhodné křížení podstromů generuje limitní rozdělení velikostí s těžkým chvostem a s mnoha *velmi malými* jedinci; ti mají špatnou fitness, selekce je vyřadí a průměrná velikost populace tím vzroste.

Obecněji: v prostoru programů je mnohem víc velkých programů se stejným chováním než malých, takže neutrální drift sám o sobě tlačí k růstu. V přírodě růst naráží na fyzikální a energetické limity, v GP je nutné bojovat s bloatem explicitně:

- **Tvrdé limity** na hloubku či počet uzlů (potomek překračující limit se zahodí nebo nahradí rodičem).
- **Parsimony pressure** — penalizační člen ve fitness $f' = f - \alpha \cdot \text{velikost}$; případně *lexikografická* varianta: při shodné fitness vyhrává menší jedinec.
- **Anti-bloat operátory** — size-fair křížení a mutace, hoist, shrink.

- **Vícekritériální přístup** — velikost jako druhé kritérium (kap. 15).

8.4 ADF a typované GP

ADF (*automatically defined functions*)

Podprogramy evolvované společně s hlavním programem. Jedinec se skládá z větve **main** a jedné či více větví ADF; volání ADF se v hlavním programu chová jako nový neterminál dané arity. Genetické operátory pracují na větvích **odděleně** (main se kříží s mainem, ADF_1 s ADF_1). ADF přináší **modularitu** a schopnost zachytit symetrie a znovupoužitelné podproblémy; bez nich GP obtížně škáluje.

Rozšíření: automaticky definované smyčky, iterace, rekurse, paměť. Alternativou je *koevoluce* dvou populací — hlavních programů a podprogramů (kap. 9).

Vynucení struktury.

- **Silně typované GP** (*strongly typed GP*): každý uzel má typ vstupů a výstupu; operátory smějí vytvářet jen typově korektní stromy. Redukuje prohledávaný prostor a umožňuje smíšené domény (čísla, booleovské hodnoty, vektory). Existují i typové hierarchie a polymorfismus.
- **Gramatické GP**: přípustné tvary stromů popíšeme bezkontextovou gramatikou, která se použije při inicializaci i jako vodítko pro operátory.

8.5 Lineární a grafové GP

Lineární GP. Program je posloupnost instrukcí (často strojový nebo byte kód) pracující nad registry. Výhody: jednoduché operátory (vektorové křížení a mutace), rychlá interpretace, přirozená blízkost skutečnému hardwaru. Nevýhoda: vysoké riziko vzniku nesmyslných programů. Oblíbené v umělém životě a evoluci herních botů. Slavný příklad: **Tierra** (T. Ray) — simulace umělého ekosystému, kde jedinci jsou sebe-kopírující programy ve strojovém kódu soutěžící o paměť a procesorový čas; emergentně vznikli paraziti, hyperparaziti a obrana proti nim. Následovníci: Avida, Avida-ED.

Grafové GP. Program je obecný (často acyklický) graf. Původně rozšíření stromového GP na paralelní programy; ukázalo se, že grafy dobře popisují elektrické obvody, konečné automaty, neuronové sítě, plány. Problémem jsou složité operátory — jak křížit obecné grafy.

Kartézské GP (CGP). Elegantní obchvat: DAG se reprezentuje svým rovinným obrazem v 2D mřížce se souřadnicemi uzlů (fenotyp), zatímco genotyp je **lineární** posloupnost celých čísel popisujících pro každý uzel jeho funkci (index do tabulky funkcí), jeho vstupy a nakonec výstupní uzly grafu. Vlastnosti:

- Mutace je bodová mutace lineárního genomu; musí se hlídat, aby vznikaly jen platné funkce a platná propojení (omezení hloubky spojů).
- Malá změna genotypu může mít velký dopad na fenotyp; naopak řada mutací je *neutrální* (uzly nepřipojené k výstupu tvoří přirozené introny, což CGP prospívá).
- Křížení je málo důležité; naivní jednobodové je příliš destruktivní, používá se aritmetická varianta a řezy na hranicích uzlů.
- Selektce: obvykle $(1 + \lambda)$ -ES s $\lambda = 4$.
- Genom má konstantní délku (bloat je omezen mřížkou).

Vývojové (developmental) GP. Místo přímé evoluce grafu se evoluuje *strom-program*, který graf postupně staví. Začíná se „embryem“ a aplikují se operace pro strukturu (sériové/paralelní zapojení, třicestné dělení) a pro elementy (vložit rezistor, kondenzátor). Použití: Koza — evoluce elektrických

obvodů (znovuobjevení patentovaných zapojení), Gruau — *cellular encoding* pro neuronové sítě, AutoML — evoluce modelů scikit-learn.

8.6 Gramatická evoluce

Gramatická evoluce (*grammatical evolution*, GE)

Genotyp je **lineární seznam celých čísel** (proměnné délky), fenotyp je program odvozený z bezkontextové gramatiky. Číslo z genomu určuje, které pravidlo se použije pro expanzi nejlevějšího neterminálu:

$$\text{pravidlo} = (\text{gen} \bmod \text{počet pravidel pro tento neterminál}).$$

Postup: genotyp $\rightarrow$ posloupnost aplikovaných pravidel $\rightarrow$ derivační strom $\rightarrow$ (redukci) syntaktický strom = fenotyp.

Vlastnosti:

- Použití operace modulo činí kódování **robustní** — každý genom je interpretovatelný.
- Genom je lineární $\Rightarrow$ lze používat **standardní operátory GA**, což je hlavní praktická výhoda.
- Gramatika zaručuje syntaktickou správnost a umožňuje snadno vnutit doménové znalosti či typy.
- Technické problémy: derivace nemusí skončit. Řeší se *wrapping* (po vyčerpání genomu se čte znovu od začátku, s limitem počtu obalení), nízkou fitness pro nepřipustné jedince nebo opravou (po překročení limitu se z gramatiky odeberou pravidla rozšiřující neterminály).
- Nevýhoda: operátory nejsou „lokální“ — malá změna genotypu (zejména na začátku) může zcela změnit fenotyp (*ripple effect*).
- *Grammatical swarm*: místo GA se pro prohledávání použije PSO.

8.7 Evoluční programování

Evoluční programování (*evolutionary programming*, EP)

Větev EA (L. Fogel, 1965 — starší než GA), jejímž cílem bylo evolvoval „umělou inteligenci“: agenta, který predikuje stav prostředí a volí vhodnou akci. Agent je reprezentován **konečným automatem**; platí **genotyp = fenotyp** (žádné zvláštní kódování). **Křížení se nepoužívá**, veškerá variabilita pochází z mutací.

EP nad automaty. Prostředí je posloupnost symbolů konečné abecedy. Automat dostává symboly na vstup, jeho výstup se porovnává s následujícím symbolem posloupnosti; fitness = úspěšnost predikce (absolutní chyba, střední kvadratická chyba), často s penalizací za velikost automatu. Mutace: změna výstupního symbolu, změna cílového stavu přechodu, přidání stavu, odebrání stavu, změna počátečního stavu. Polovina populace se nahrazuje novými automaty.

Slavný příklad — predikce prvočísel. Cílem je predikovat posloupnost

$$111010100010100010100010000010\dots,$$

kde 1 znamená „toto číslo je prvočíslo“ (tj. naučit se Eratosthenovo síto). Postupuje se iterativně: nejprve se učí krátká posloupnost, pak se prodlužuje o jeden symbol. Fitness: bod za každý uhodnutý symbol minus $0,01 \cdot N$ za počet stavů. Výsledek je poučný: automaty se zjednodušovaly, až nakonec vznikl jednostavový automat generující stále 0 — prvočísla jsou totiž řídká, takže konstantní odpověď „není prvočíslo“ má vysokou úspěšnost. Klasická ukázka toho, jak evoluce zneužije špatně navrženou fitness.

EP dnes.

- Kódování má být *přirozené* pro úlohu (obvykle genotyp = fenotyp).
- Sada „chytrých“, doménově specifických mutací; žádné křížení.
- Rodičovská selekce: každý jedinec je právě jednou rodičem.
- Environmentální selekce: turnaj mezi rodiči a potomky.
- EP nad reálnými vektory obsahuje — stejně jako ES — rozptyly mutací v genomu ($\sigma'_j = \sigma_j(1 + \alpha N(0, 1))$), poté $x'_j = x_j + \sigma'_j N(0, 1)$. EP a ES tak prakticky splynuly.

8.8 Je křížení k něčemu? Experiment s bezhlavým kuřetem

Tradiční obhajoba křížení = výměna stavebních bloků. Kritika (typická pro EP): křížení je jen podivná velká náhodná mutace, která navíc nerespektuje kódování.

Headless chicken test (Jones, 1995)

Porovnáme standardní křížení s *náhodným* křížením, kde se jedinec kříží s **náhodně vygenerovaným** řetězcem. Je-li výsledek stejný nebo lepší, pak zkoumaný operátor ve skutečnosti nefunguje jako křížení, ale jako makromutace — „nenazývejme bezhlavé kuře kuřetem, i když má mnoho kuřecích rysů“.

Interpretace: existují-li jasné stavební bloky, je právě křížení lepší. Vyhraje-li náhodné křížení, znamená to, že v dané úloze/kódování stavební bloky nejsou — špatné kódování, špatný operátor, nebo úloha pro křížení nevhodná.

8.9 Shrnutí k zkoušce

- Stromové GP (Koza): terminály + neterminály, uzavřenost a dostatečnost; inicializace grow / full / ramped half-and-half; křížení = výměna podstromů (neterminály s vyšší pravděpodobností), více typů mutací včetně mutace konstant; fitness = spuštění programu.
- Bloat = růst velikosti bez zlepšení fitness (introny); tři vysvětlení: replication accuracy, removal bias, crossover bias. Obrana: limity velikosti, penalizace (parsimony pressure), anti-bloat operátory (hoist, shrink, size-fair), vícekriteriální přístup.
- ADF = evolvované podprogramy (modularita); operátory pracují na větvích odděleně. Typované GP a gramatické GP omezují prostor na smysluplné programy.
- Varianty: lineární GP (byte kód, Tierra), grafové GP, Cartesian GP (lineární genotyp → 2D DAG, bodová mutace, (1 + 4)-ES), vývojové GP (strom staví graf — obvody, sítě).
- Gramatická evoluce: lineární genom celých čísel, výběr pravidla gramatiky pomocí modula, derivací strom → fenotyp; robustní kódování, standardní operátory GA, ale nelokální mapování.
- EP: konečné automaty, genotyp = fenotyp, jen mutace, turnajová selekce z rodičů i potomků; příklad predikce prvočísel (degenerace fitness).
- Headless chicken experiment: test, zda křížení skutečně kombinuje stavební bloky, nebo je jen makromutací.

9 Koevoluce a otevřená evoluce

9.1 Kontextová fitness

Koevoluce (*coevolution*)

Evoluční algoritmus, v němž fitness jedince **není absolutní**, ale závisí na ostatních jedincích — na jeho vlastní populaci nebo na jiné, souběžně se vyvíjející populaci. Fitness je tedy *kontextová* a fitness krajina je **dynamická**: mění se tím, jak se mění protivníci či partneři.

Motivace: pro mnoho úloh neumíme napsat absolutní účelovou funkci („jak dobrá je strategie pro dámu?“), ale umíme dva jedince *porovnat* (zahrát partii). Kromě toho koevoluce slibuje *evoluční*

závody ve zbrojení — protivníci se navzájem tlačí ke stále lepším řešením. Typy koevoluce:

	kompetitivní	kooperativní
vztah	fitness jednoho roste na úkor druhého	jedinci se doplňují do společného řešení
příklad	predátor–kořist, host–parazit, hráč–protihráč, řadicí sítě vs. posloupnosti čísel	dekompozice problému na podproblémy, moduly a plány (CoDeep-NEAT), hlavní programy a ADF
fitness	výsledek souboje (turnaje)	kvalita složeného řešení, na němž se jedinec podílel
riziko	cyklení, odpojení, přespecializace	kredit assignment, líní partneři

9.2 Kompetitivní koevoluce

Klasický příklad: Hillisovy řadicí sítě (1990). Populace řadicích sítí koevolvuje proti populaci testovacích posloupností. Fitness sítě = podíl správně seřazených testů; fitness testu = kolik sítí na něm selhalo. Parazitické testy se specializují na slabiny sítí, sítě se musí zlepšovat. Hillis takto našel síť pro 16 prvků s **61** komparátory. *Pozor na častý omyl*: nejlepší známý *lidský* návrh (Green, 1969) má 60 komparátorů, koevoluce ho tedy nepřekonala. Podstatné je srovnání uvnitř experimentu — týž genetický algoritmus *bez* koevolvujících parazitů dosáhl jen 65 komparátorů. Koevoluce zde tedy prokazatelně zlepšila optimalizační schopnost EA a přiblížila se lidskému výsledku na jediný komparátor.

Predátor–kořist v evoluční robotice: roboti-lovci a roboti-uprchlíci se navzájem zdokonalují. **Host–parazit**: modely v umělém životě (Tierra), testování software.

Turnajové hodnocení. Fitness se získává soubojem: každý s každým (drahé), náhodný vzorek protivníků (levné, zašuměné), *hall of fame* (souboj s archivem historicky nejlepších — brání zapomínání), nebo *turnaj se sdílenou fitness* (*competitive fitness sharing*: za porážení protivníka, kterého poráží málokdo, je vyšší odměna).

9.3 Patologie koevoluce

- **Cyklení** (*cycling*, intransitivita): populace obíhají v kruhu jako v kámen–nůžky–papír; zdánlivý pokrok není skutečný. Dynamika koevoluce může být až deterministicky chaotická.
- **Odpojení** (*disengagement*): jedna populace zcela přeroste druhou, všechny souboje končí stejně, fitness ztrácí rozlišovací schopnost a selekce přestane fungovat (gradient zmizí).
- **Efekt Červené královny** (*Red Queen effect*): všichni se zlepšují, ale relativní fitness zůstává konstantní — z naměřených hodnot nepoznáme, zda je pokrok skutečný. Řešení: měření proti pevné externí sadě protivníků (*master tournament*).
- **Průměrné stabilní stavy** (*mediocre stable states*): populace se ustálí ve vzájemné rovnováze na nízké úrovni.
- **Přespecializace** (*overspecialization*, *focusing*): jedinec se optimalizuje na aktuálního protivníka a ztratí obecnost.
- **Zapomínání** (*forgetting*): schopnost porazit staré protivníky se ztrácí. Řešení: archivy (*hall of fame*, Pareto-koevoluční archivy, *Nash memory*).

9.4 Kooperativní koevoluce

CCGA (*cooperative coevolutionary GA*, Potter & De Jong)

Problém se rozdělí na k komponent (např. souřadnice, moduly, pravidla). Pro každou komponentu se udržuje **samostatná populace**. Fitness jedince z populace i se určí tak, že se *spojí* se zástupci ostatních populací (typicky s jejich aktuálně nejlepšími jedinci, případně i s náhodnými) a ohodnotí se výsledné složené řešení.

Výhody: rozklad velkého prostoru na menší, přirozená paralelizace, vhodné pro modulární úlohy. Problémy: *přiřazení zásluh* (*credit assignment*) — jak rozdělit kvalitu složeného řešení mezi komponenty; *provázanost* (silně interagující komponenty nelze dobře oddělit); *relativní přespecializace* (populace se přizpůsobí konkrétním partnerům).

Příklady v praxi: koevoluce hlavních programů a ADF v GP, CoDeepNEAT (populace modulů a populace „blueprintů“, kap. 13), meta-lamarckovské memetické algoritmy (populace memů a populace řešení, kap. 11).

9.5 Evoluce kooperace a věžňovo dilema

Koevoluce úzce souvisí se základní otázkou evoluční biologie: *jak může vzniknout altruismus*, když z definice snižuje fitness jedince? Darwinismus je ve své podstatě soutěživý („přežití nejzdatnějšího“), přesto je v přírodě i ve společnosti mnoho spolupráce. Teorie:

- **Skupinová selekce** — evoluce působí i na skupiny (Darwin); problém: jak vysvětlit podvodníky uvnitř skupiny.
- **Příbuzenská selekce** (*kin selection*) — zachování téměř identických genů u příbuzných; problém: altruismus vůči cizím a jiným druhům.
- **Sobecký gen** (Dawkins) — jednotkou evoluce je gen, ne jedinec. (Wilson: „organismus je jen způsob, jak DNA vyrábí další DNA“.)
- **Reciproční altruismus** (Trivers, 1971) — vzájemný prospěch, *stín budoucnosti*; formální model = iterované věžňovo dilema.

Věžňovo dilema (*prisoner's dilemma*)

Hra dvou hráčů s akcemi C (kooperace) a D (zrada) a výplatami

	C	D
C	R/R	S/T
D	T/S	P/P

$T > R > P > S$ (a pro iterovanou verzi $2R > T + S$).

Typicky $R = -1$, $T = 0$, $P = -2$, $S = -3$. Protože $T > R$ a $P > S$, je zrada **dominantní strategií** obou hráčů; DD je **Nashovo equilibrium**, ale jako jediný výsledek **není Pareto-optimální** — CC maximalizuje společný zisk.

Racionální hráči tedy dospějí k výsledku, který je pro oba horší než dosažitelná spolupráce. Tentýž mechanismus v n -hráčské podobě se nazývá *tragédie obecní pastviny* (*tragedy of the commons*): každému se individuálně vyplatí čerpat sdílený zdroj o něco víc, a zdroj proto zanikne. Otázka „co je vlastně racionální (a chovají se tak lidé?)“ je jádrem celé diskuse kolem věžňova dilematu.

Dominantní strategie, Nashovo equilibrium, Pareto optimalita

Strategie s je *dominantní* pro hráče i , dává-li stejný nebo lepší výsledek než kterákoli jiná strategie i proti *všem* strategiím protihráče. Dvojice (s_i, s_j) je v *Nashově equilibriu*, jsou-li vzájemně nejlepšími odpověďmi (žádný hráč nemá důvod jednostranně změnit strategii). Řešení je *Pareto-optimální*, pokud nelze zlepšit výsledek jednoho hráče, aniž by se zhoršil výsledek jiného. Nashova věta: každá hra s konečně mnoha strategiemi má equilibrium ve *smíšených* strategiích (náhodná volba mezi čistými) — např. kámen–nůžky–papír.

Iterovaná verze a Axelrodovy turnaje. Hraje-li se opakovaně a hráči si pamatují historii, může se vyplatit spolupráce (*stín budoucnosti*). Zná-li se předem přesný počet kol N , indukci od konce plyne, že optimální je stále zrazovat — v praxi ale hráči konec neznají. Axelrod (1980) uspořádal turnaje strategií:

- 1. turnaj: 14 strategií + RANDOM, 200 her, každý s každým (i sám se sebou), 5 opakování. Vyhrála **TFT** (*tit for tat*): začni kooperací, pak opakuj poslední tah soupeře.
- 2. turnaj: 62 strategií, všichni znali výsledky prvního — TFT vyhrála znovu.
- 3. „ekologický“ turnaj: simulace jako v GA — počet jedinců strategie v další generaci je úměrný počtu vítězství, 1000 generací. TFT opět zvítězila.

Čtyři vlastnosti úspěšných strategií: *slušnost* (niceness — nezrazuj první), *dráždivost* (provocability — zradu potrestej okamžitě), *odpouštění* (forgiveness — ale rychle se uklidni), *srozumitelnost* (clarity — buď jednoduchý, aby ti ostatní rozuměli). Žádná strategie nevyhraje proti všem; je nutné být úspěšný proti velmi rozmanitým soupeřům (ALL-D, TFTT, RANDOM, TRIGGER) a umět hrát dobře sám se sebou.

Poučení a pozdější vývoj. V prostředích, kde výplaty přejí spolupráci a kde je vysoká pravděpodobnost opakování, se spolupráce obvykle vyvine — a není k tomu potřeba racionalita ani vědomí, stačí vyšší fitness. V zašuměném prostředí (chybné provedení tahu) je úspěšnější strategie **Pavlov** (*win-stay, lose-shift*): dostal-li jsem R nebo P , zopakuj tah (C); dostal-li jsem T nebo S , změň jej (D). Turnaj opakovaný po 20 letech vyhrál *tým* strategií z University of Southampton: prvních zhruba 10 tahů sloužilo k rozpoznání spoluhráče, poté všichni členové týmu obětavě pomáhali jedné „otcovské“ strategii získat vysoké skóre.

9.6 Diverzita, druhy a niky

Malý selekční tlak stačí k tomu, aby diverzita populace vymizela. (Podobnou úvahu rozvíjí Flegrova *zamrzlá evoluce*: geneticky proměnlivý druh se vůči selekci chová pružně jen krátce, poté „zamrzne“ a na změny prostředí přestane reagovat.) Mechanismy pro udržení diverzity (užitečné jak pro koevoluci, tak pro dynamickou fitness a vícekritériální optimalizaci):

- **Fitness sharing:** fitness se dělí počtem podobných jedinců $f'(i) = f(i) / \sum_j sh(d(i, j))$, kde $sh(d) = 1 - (d/\sigma_{share})^\alpha$ pro $d < \sigma_{share}$, jinak 0. Vytváří tlak na obsazení různých nik.
- **Crowding:** nový jedinec nahrazuje *nejpodobnějšího* jedince (deterministic crowding, RTS).
- **Nenáhodné páření** (*non-random mating*): křížení jen mezi podobnými jedinci (*speciace*), nebo podle topologie — jedinci žijí na 2D/3D mřížce a kříží se jen se sousedy.
- **Ostrovní model** (*island model*): několik subpopulací s občasnou migrací (podrobněji níže).
- **Explicitní druhy** pomocí značkovacích bitů (*tag bits*), nebo podle strukturální podobnosti genomů (NEAT, kap. 13); páření je pak povoleno jen uvnitř druhu.

Problémem je vždy definice podobnosti (Hammingova vzdálenost, vzdálenost v prostoru fenotypů, počet shodných inovačních čísel) a volba prahu niky.

Paralelní modely EA. Struktura populace úzce souvisí s paralelizací; rozlišují se tři modely:

- **Master–slave** (globální paralelizace): jedna populace, mezi uzly se rozdělí jen *vyhodnocení fitness*. Algoritmus je matematicky totožný se sekvenčním; vyplatí se při drahé fitness.
- **Ostrovní / hrubozrný model** (*coarse-grained*): několik nezávislých subpopulací, mezi nimiž občas *migruje* několik jedinců. Parametry: topologie ostrovů, *migrační interval* a *migrační míra*, výběr migrantů a nahrazovaných. Vzácná migrace udržuje diverzitu (ostrovy konvergují jinak), příliš častá model degeneruje na jedinou populaci.
- **Buněčný / jemnozrný model** (*fine-grained*, difuzní): jedinci sedí na 2D/3D mřížce a kříží se jen se sousedy. Dobré řešení se šíří populací jako vlna, což silně zpomaluje předčasnou konvergenci; jde o implicitní formu nenáhodného páření.

Ostrovní i buněčný model tedy nejsou jen implementační trik — mění samotnou dynamiku algoritmu a patří mezi standardní nástroje udržení diverzity.

9.7 Dynamické fitness krajiny

Fitness krajina (*fitness landscape*)

Zavedl Sewall Wright (1932). Grafické znázornění závislosti fitness na genotypu (nebo na frekvenci alel či na rysu fenotypu). Teoretický nástroj pro uvažování o EA; ve vyšších dimenzích ovšem není intuitivní.

Fitness se často mění v čase: robot v místnosti, kde někdo rozsvítí; začne pršet; krize na burze; turnaje agentů (koevoluce). Klasické GA si vedou špatně, protože jsou „přeučené“ na statické úlohy — rychle konvergují a ztratí diverzitu, kterou by pro reakci na změnu potřebovaly. De Jongovo srovnání: (1 + 10)-ES reaguje na náhlou změnu pomalu, protože samoadaptací zmenšila krok; generační GA s ruletovou selekcí a bez elitismu si udrží více diverzity a zotaví se rychleji.

Úpravy EA pro dynamické krajiny:

- **Diploidní reprezentace** (Goldberg, Smith) — dvě sady chromozomů s dominancí fungují jako dlouhodobá paměť při oscilující fitness.
- **Hypermutable** (Cobb, Grefenstette) — klesne-li průměrná fitness populace (pravděpodobně se změnilo prostředí), výrazně se zvýší pravděpodobnost mutace.
- **Náhodná migrace** — do populace se vloží dávka nových náhodných jedinců.
- **Udržování diverzity** — nižší selekční tlak, crowding, niching, subpopulace, čárková ES (μ, λ) (rodiče nepřezívají, populace se snáz odpoutá od zastaralého optima).

Typy změn: malé plynulé (opotřebení hardwaru), morfologické (vznikají a zanikají kopce), cyklické (roční období), katastrofické nespojitě. Mění-li se fitness příliš rychle, žádné řešení není trvale dobré (srov. No Free Lunch).

9.8 Otevřená evoluce

Otevřená evoluce (*open-ended evolution*)

Evoluční proces, který **nemá pevný cíl** a který neustále produkuje novou složitost a nové „druhy“ chování, aniž by konvergoval. Vzorem je pozemská biosféra: nemá účelovou funkci, přesto trvale generuje novinky. V umělém životě se sleduje, zda systém vykazuje trvalý růst složitosti, vznik nových tříd entit (paraziti, hyperparaziti), ekologické vztahy a nezastavující se dynamiku.

Znaky otevřené evoluce. Aby bylo možné o systému vůbec rozhodnout, formulují se *znaky* (*hallmarks*) otevřenosti:

- **trvalá produkce novosti** — nové chování se objevuje i po libovolně dlouhém běhu, systém se neustálí;
- **neomezený růst složitosti** jedinců i jejich vzájemných vztahů;

- **vznik nových tříd entit** a nových úrovní organizace (v biologii *major transitions*: replikátor → buňka → mnohobuněčnost → společenství);
- **evoluce evolvability** — mění se i samotný způsob, jak systém generuje novinky (reprezentace, operátory, genotyp–fenotyp mapování).

Systémy a proč se zasekávají. *Tierra* (Ray) a *Avida* — sebekopírující programy ve strojovém kódu, kde emergentně vznikli paraziti a hyperparaziti; *Polyworld* (Yaeger) — agenti s neuronovými sítěmi ve 2D světě; *ECHO* (Holland) — agenti s „tagy“ a zdroji modelující ekologické a ekonomické vztahy. Prakticky každý z nich po čase dospěje do stavu, kdy už se nic kvalitativně nového neobjevuje: prostředí je konečné, prostor možných „vynálezů“ se vyčerpá a dynamika zamrzne. Hlavní hypotézy proč: chybí dostatečně bohaté a samo se rozšiřující prostředí, chybí skutečně otevřená reprezentace a chybí mechanismus, který by průběžně zvyšoval nároky na jedince. Dosažení skutečně otevřené evoluce zůstává otevřeným výzkumným problémem.

Generativní (morfo genetické) reprezentace. S otevřenou evolucí těsně souvisí otázka, jak vůbec zakódovat neomezeně rostoucí složitost. Odpovědi jsou *nepřímá* kódování, kde genotyp není popis výsledku, ale **návod, jak výsledek vypěstovat**:

- **L-systémy** (Lindenmayer) — přepisovací gramatiky generující větvící se struktury (rostliny, cévní systémy, morfologie robotů);
- **celulární automaty** — lokální pravidla generující globální vzory; klasický model emergence;
- **buněčné kódování** (Gruau) a **CPPN/HyperNEAT** (kap. 13) — tytéž principy aplikované na neuronové sítě;
- **vývojové GP** (kap. 8) — strom-program, který staví obvod či graf.

Společný přínos: malý genom, velký fenotyp, přirozené vyjádření symetrií a opakujících se motivů — tedy právě to, co příroda dělá a co přímé kódování neumí.

Novelty search. Radikální myšlenka (Lehman & Stanley, 2011: *Abandoning Objectives*): u klamavých (deceptivních) úloh je cílová funkce zavádějící — vede do lokálních optim a neexistuje spojitá trajektorie ke globálnímu optimu. Proto **zahodme cíl a odměňujeme pouze novost**.

Řídkost (*sparseness*) a novelty search

Definujeme *prostor chování* (*behavior space*) — prostor fenotypických projevů (např. koncová pozice robota v bludišti). Novost jedince x se měří řídkostí okolí:

$$\rho(x) = \frac{1}{k} \sum_{i=1}^k \text{dist}(x, \mu_i),$$

kde μ_i jsou k nejbližších sousedů x v aktuální populaci **a v permanentním archivu**. Jedinci s ρ nad prahem se do archivu přidávají. Fitness = ρ ; původní účelová funkce se *nepoužívá vůbec*.

Novelty search bývá na úlohách typu „robot v bludišti“ úspěšnější než cílená optimalizace, protože systematicky prochází dosažitelné chování. Kombinuje se s NEAT. Navazující směr **quality-diversity** (*novelty search with local competition*, MAP-Elites) hledá archiv řešení, která jsou zároveň rozmanitá a v rámci své niky kvalitní; MAP-Elites udržuje mřížku buněk indexovaných deskriptory chování a v každé buňce nejlepší nalezené řešení. Dalším směrem jsou algoritmy generující vlastní úlohy (*minimal criterion coevolution*, POET).

9.9 Interaktivní evoluce

Interaktivní evoluce (*interactive evolutionary computation, IEC*)

Fitness funkci nahradí **člověk**: uživateli se předloží populace kandidátů a on vybere ty, které se mu líbí či které jsou podle něj slibné. Používá se všude tam, kde kritérium kvality neumíme zapsat — estetika, design, hudba, chuť, ergonomie, ale i identikit svědka.

Klasickými ukázkami jsou Dawkinsovy *Biomorphs*, Simsovy evolvované obrazy a zejména **Picbreeder** a **EndlessForms** (evoluce obrázků, resp. 3D tvarů pomocí CPPN, kap. 13), kde navíc uživatelé navazují na cizí rozpracované linie — jde tedy o kolektivní, otevřenou a necílenou evoluci. Právě zkušenost z Picbreederu (nejzajímavější objekty nikdy nevznikly tak, že by je někdo cíleně hledal) byla přímou motivací pro novelty search.

Omezení: člověk je nesrovnatelně pomalejší a dražší než výpočet $\Rightarrow$ nutné malé populace (typicky 9–20 jedinců na obrazovku), málo generací a únava uživatele vede k nekonzistentnímu hodnocení. Řešení: hodnotit jen pořadí místo čísel, prokládat lidské hodnocení naučeným *surrogate* modelem, nebo použít IEC jen k občasnému nasměrování jinak automatické evoluce.

9.10 Shrnutí k zkoušce

- Koevoluce = kontextová fitness + dynamická krajina. *Kompetitivní*: predátor–kořist, host–parazit, Hillisovy řadičí sítě. *Kooperativní*: dekompozice na komponenty, CCGA, moduly + blueprinty.
- Patologie: cyklení (intransitivita), odpojení, efekt Červené královny, průměrné stabilní stavy, přespecializace, zapomínání. Léky: archivy (hall of fame), sdílená fitness, měření proti pevné sadě.
- Vězňovo dilema: $T > R > P > S$, $2R > T + S$; D dominantní, DD Nashovo equilibrium a jediné Pareto-neoptimální; iterovaná verze $\Rightarrow$ TFT; čtyři vlastnosti úspěšné strategie (slušnost, dráždivost, odpouštění, srozumitelnost); Pavlov v zašuměném prostředí.
- Diverzita: fitness sharing, crowding, speciace, ostrovní model, nenáhodné páření.
- Dynamická fitness: diploidie, hypermutace, náhodná migrace, udržování diverzity, (μ, λ) -ES.
- Otevřená evoluce: bez pevného cíle; znaky = trvalá novost, neomezený růst složitosti, nové třídy entit, evoluce evolvability. Systémy Tierra, Avida, Polyworld, ECHO — všechny se dříve či později zaseknou. Generativní (nepřímé) reprezentace: L-systémy, celulární automaty, buněčné kódování, CPPN.
- Novelty search měří řidkost $\rho(x)$ v prostoru chování vůči populaci a permanentnímu archivu, účelovou funkci vůbec nepoužívá; navazuje quality-diversity a MAP-Elites (mřížka deskriptorů chování).
- Interaktivní evoluce: fitness = člověk (Biomorphs, Picbreeder, EndlessForms); omezením je malá populace a únava uživatele.

10 Rojové optimalizační algoritmy

Rojová inteligence (*swarm intelligence*)

Přístup, kde globálně inteligentní chování **emerguje** z jednoduchých lokálních pravidel mnoha jedinců bez centrálního řízení. Rysy: decentralizace, jednodušší jedinci, nepřímá komunikace přes prostředí (*stigmergie*), robustnost vůči výpadku jedince. Na rozdíl od EA zde obvykle není křížení, mutace ani „smrt“ jedinců — jedinci se pohybují a pamatují si.

10.1 Optimalizace rojem částic (PSO)

Particle swarm optimization (Eberhart a Kennedy, 1995) je inspirována hejny ptáků a ryb. Jedinec je *částice* — bod v $\mathbb{R}^n$ s rychlostí. Nepoužívá se křížení ani mutace; místo toho má algoritmus **paměť**:

- $\vec{p}_i$ ($pBest$) — nejlepší pozice, kterou částice i kdy navštívila (kognitivní, individuální paměť),

- $\vec{g}$ ($gBest$) — nejlepší pozice nalezená celým rojem (sociální, kolektivní paměť).

Pohybové rovnice PSO

$$\vec{v}_i \leftarrow \underbrace{w \vec{v}_i}_{\text{setrvačnost}} + \underbrace{c_1 r_1 (\vec{p}_i - \vec{x}_i)}_{\text{kognitivní složka}} + \underbrace{c_2 r_2 (\vec{g} - \vec{x}_i)}_{\text{sociální složka}}, \quad \vec{x}_i \leftarrow \vec{x}_i + \vec{v}_i,$$

kde $r_1, r_2 \sim U(0, 1)$ (losují se pro každou složku zvlášť), c_1, c_2 jsou koeficienty učení (v původní verzi $c_1 = c_2 = 2$) a w je *setrvačná váha* (*inertia weight*). V původní verzi z roku 1995 je $w = 1$ (chybí); Shi a Eberhart ji doplnili a doporučují lineární pokles např. z 0,9 na 0,4: velké w podporuje exploraci, malé exploataci.

Algoritmus 7 PSO

```

1: inicializuj pozice  $\vec{x}_i$  a rychlosti  $\vec{v}_i$  náhodně;  $\vec{p}_i \leftarrow \vec{x}_i$ 
2: repeat
3:   for každou částici  $i$  do
4:     vyhodnoť  $f(\vec{x}_i)$ 
5:     if  $f(\vec{x}_i)$  je lepší než  $f(\vec{p}_i)$  then  $\vec{p}_i \leftarrow \vec{x}_i$ 
6:   end if
7: end for
8:  $\vec{g} \leftarrow$  nejlepší z  $\{\vec{p}_i\}$ 
9: for každou částici  $i$  do
10:   aktualizuj rychlost a pozici dle pohybových rovnic
11: end for
12: until je dosaženo maxima iterací nebo požadované přesnosti

```

Číselný příklad. Částice v $\mathbb{R}^2$: $\vec{x} = (1; 2)$, $\vec{v} = (0,5; -0,5)$, $\vec{p} = (0; 3)$, $\vec{g} = (-1; 1)$, $w = 0,7$, $c_1 = c_2 = 1,5$, $r_1 = 0,4$, $r_2 = 0,8$.

$$\begin{aligned}
\vec{v}' &= 0,7(0,5; -0,5) + 1,5 \cdot 0,4((0; 3) - (1; 2)) + 1,5 \cdot 0,8((-1; 1) - (1; 2)) \\
&= (0,35; -0,35) + 0,6(-1; 1) + 1,2(-2; -1) = (-2,65; -0,95), \\
\vec{x}' &= (1; 2) + (-2,65; -0,95) = (-1,65; 1,05).
\end{aligned}$$

Částice tedy přeletěla přes $\vec{g}$ — typické chování PSO, které je zdrojem explorace. Proto se rychlost omezuje ($|v_j| \leq v_{\max}$) nebo se používá *constriction faktor* (Clerc a Kennedy): $\vec{v} \leftarrow \chi[\vec{v} + c_1 r_1 (\vec{p} - \vec{x}) + c_2 r_2 (\vec{g} - \vec{x})]$, kde $\chi = 2/|2 - \varphi - \sqrt{\varphi^2 - 4\varphi}|$ pro $\varphi = c_1 + c_2 > 4$; pro $c_1 = c_2 = 2,05$ vychází $\chi \approx 0,729$. Tato volba zaručuje konvergenci roje.

Topologie sousedství. Místo globálního $\vec{g}$ (topologie *gbest*, tj. úplný graf) lze použít *lbest* — nejlepší jedinec v lokálním okolí:

- **Kruh** (každá částice má k sousedů): informace se šíří pomalu $\Rightarrow$ pomalejší konvergence, ale lepší explorace a menší riziko uvíznutí.
- **Von Neumannova mřížka, hvězda, náhodné grafy.**
- Empiricky: *gbest* je rychlejší na jednoduchých úlohách, *lbest* robustnější na multimodálních.

Varianty a vztah k EA. Diskrétní/binární PSO (rychlost interpretována jako pravděpodobnost překlopení bitu přes sigmoidu), PSO s více roji, hybridizace s lokálním prohledáváním, *grammatical swarm*.

	genetický algoritmus	PSO
společné	náhodná inicializace, fitness, stochastické aktualizace	totéž
jedinci	vznikají a zanikají	trvale existují, pohybují se
paměť	jen v populaci	explicitní ($\vec{p}_i, \vec{g}$)
operátory	křížení, mutace	žádné, jen pohyb
tok informace	obousměrný mezi rodiči	jednosměrný: od lepších k ostatním

10.2 Optimalizace mravenčí kolonií (ACO)

Ant colony optimization (M. Dorigo, 1991) napodobuje hledání cesty u mravenců: mravenec pokládá *feromonovou stopu*, po kratší cestě se vrátí dřív, feromon se tam hromadí rychleji, což přitahuje další mravence — kladná zpětná vazba. Feromon se navíc **odpařuje**, takže systém zapomíná zastaralou informaci a neuvízne.

Stigmergie

Nepřímá komunikace prostřednictvím změn v prostředí. Mravenci spolu nekomunikují přímo; jediným kanálem je feromon na hranách. Obecný princip reaktivních multiagentních architektur.

Ilustrace — Steelsův průzkumník Marsu. Roj robotů má sbírat vzorky hornin; základna vysílá navigační signál (stačí sledovat gradient). Robot má „drobky“ (radioaktivní značky) pro nepřímou komunikaci. Pravidla s prioritou: R1: vidíš-li překážku, změň směr; R2: neseš-li vzorek a jsi na základně, polož ho; R3: neseš-li vzorek, jdi po gradientu nahoru; R4: vidíš-li vzorek, seber ho; R5: jinak se pohybuj náhodně. Druhá iterace přidá stigmergii: R7: neseš-li vzorek a nejsi na základně, polož 2 drobky a jdi po gradientu nahoru; R8: cítíš-li drobek, seber 1 drobek a jdi po gradientu dolů. Vznikne tak efektivní kolektivní sběr z bohatých nalezišť, aniž by roboti měli mapu či komunikaci.

Obecné schéma ACO. Úlohu převedeme na hledání cesty v ohodnoceném grafu; mravenci jsou agenti konstruující řešení po hranách.

1. Každý mravenec stochasticky zkonstruuje řešení (posloupnost hran).
2. Volitelně *akce démona* (*daemon actions*) — centralizovaný krok, typicky lokální prohledávání zlepšující nalezené cesty.
3. Porovnají se kvality řešení.
4. Aktualizují se feromony na hranách (odpaření + uložení).

Ant System (AS) — pravidlo přechodu

Mravenec k v městě i zvolí následující město j z množiny dosud nenavštívených $\mathcal{N}_i^k$ s pravděpodobností

$$p_{ij}^k = \frac{[\tau_{ij}]^\alpha [\eta_{ij}]^\beta}{\sum_{l \in \mathcal{N}_i^k} [\tau_{il}]^\alpha [\eta_{il}]^\beta}, \quad \eta_{ij} = \frac{1}{d_{ij}},$$

kde τ_{ij} je množství feromonu na hraně, η_{ij} heuristická „viditelnost“ (převrácená vzdálenost) a α, β váhy obou vlivů (typicky $\alpha = 1, \beta \in [2, 5]$).

$$\tau_{ij} \leftarrow (1 - \rho) \tau_{ij} + \sum_{k=1}^m \Delta \tau_{ij}^k, \quad \Delta \tau_{ij}^k = \begin{cases} Q/L_k & \text{prošel-li mravenec } k \text{ hranou } (i, j), \\ 0 & \text{jinak,} \end{cases}$$

kde $\rho \in (0, 1)$ je míra odpařování a L_k délka trasy mravence k . Kratší trasa $\Rightarrow$ více feromonu.

Číselný příklad (pravidlo přechodu). Mravenec stojí ve městě i a může jít do měst A, B, C se vzdálenostmi $d = (5; 10; 2)$ a feromony $\tau = (2; 4; 1)$; $\alpha = 1$, $\beta = 2$. Viditelnosti: $\eta = (0,2; 0,1; 0,5)$. Váhy:

$$w_A = 2 \cdot 0,2^2 = 0,08, \quad w_B = 4 \cdot 0,1^2 = 0,04, \quad w_C = 1 \cdot 0,5^2 = 0,25, \quad \sum = 0,37.$$

Pravděpodobnosti: $p_A = 0,216$, $p_B = 0,108$, $p_C = 0,676$. Přestože hrana do B má nejvíce feromonu, krátká vzdálenost do C (umocněná $\beta = 2$) rozhoduje.

Číselný příklad (aktualizace). Nechť $\tau_{ij} = 2$, $\rho = 0,5$, $Q = 1$ a hranou prošli dva mravenci s délkami tras $L_1 = 20$, $L_2 = 25$:

$$\tau_{ij} \leftarrow 0,5 \cdot 2 + \left(\frac{1}{20} + \frac{1}{25}\right) = 1 + 0,09 = 1,09.$$

ACS (Ant Colony System). Vylepšení konkurenceschopné se specializovanými řešiči TSP; inspirováno Q-learningem:

- **Globální aktualizace jen nejlepším řešením** (elitismus): feromon přidává pouze globálně (nebo iteračně) nejlepší mravenec.
- **Lokální aktualizace:** kdykoli mravenec projde hranou, feromon na ní se mírně sníží ($\tau \leftarrow (1 - \xi)\tau + \xi\tau_0$) — tím se hrana stává méně atraktivní pro ostatní a roste rozmanitost tras.
- **Pseudonáhodné pravidlo výběru:** s pravděpodobností q_0 se zvolí deterministicky nejlepší hrana podle $\tau_{il}\eta_{il}^\beta$ (exploatace), jinak se použije standardní pravidlo AS (explorace).

Další varianty. *Elitist AS* (nejlepší trasa přispívá navíc), *Rank-based AS* (přispívá N nejlepších, váha podle pořadí), **MAX-MIN AS** (feromon je držen v intervalu $[\tau_{\min}, \tau_{\max}]$, aktualizuje jen nejlepší, inicializace na $\tau_{\max}$ — brání předčasně stagnaci), ACO pro spojitou optimalizaci (diskretizace prostoru na podintervaly, feromon vede hledání do lepších podprostorů).

Vlastnosti a aplikace. Decentralizace, emergence, nepřímá komunikace; blízko reaktivním architekturám agentů (Brooksova subsumpční architektura). Velkou předností je schopnost **optimalizovat v dynamickém prostředí** — při změně grafu se feromony přizpůsobí. Použití: TSP, směřování vozidel, směřování v sítích, sekvenování DNA, skládání proteinů, detekce hran v obraze, rozvrhování.

10.3 Další rojové algoritmy

- **Umělá včelí kolonie** (*artificial bee colony*, ABC; Karaboga 2005). Populace zdrojů potravy = řešení. Tři role: *dělnice* (prohledávají okolí svého zdroje), *pozorovatelky* (vybírají zdroj ruletou podle kvality a prohledávají jeho okolí), *průzkumnice* (zdroj, který se *limit* pokusů nezlepšil, se opustí a nahradí náhodným — vestavěný restart).
- **Firefly algorithm** (Yang, 2008). Jedinci = světlušky; jasnost je úměrná fitness a s vzdáleností klesá jako $I(r) = I_0 e^{-\gamma r^2}$. Méně jasná světluška se posouvá směrem k jasnější: $\vec{x}_i \leftarrow \vec{x}_i + \beta_0 e^{-\gamma r_{ij}^2} (\vec{x}_j - \vec{x}_i) + \alpha \vec{\epsilon}$. Díky omezenému dosahu přitažlivosti se roj přirozeně dělí na podskupiny $\Rightarrow$ vhodné pro multimodální úlohy.
- **Bat algorithm, cuckoo search, grey wolf optimizer** a desítky dalších „metaforických“ algoritmů. Kritika: mnohé jsou jen přejmenované varianty PSO či ES a přinášejí spíše terminologický zmatek než nové principy.

10.4 Shrnutí k zkoušce

- PSO: rovnice $\vec{v} \leftarrow w\vec{v} + c_1r_1(\vec{p} - \vec{x}) + c_2r_2(\vec{g} - \vec{x})$, $\vec{x} \leftarrow \vec{x} + \vec{v}$; setrvačnost w (klesá $0,9 \rightarrow 0,4$), kognitivní a sociální složka, $c_1 = c_2 = 2$; omezení $v_{\max}$ nebo constriction $\chi \approx 0,729$; topologie gbest vs. lbest (kruh, mřížka).
- Rozdíl PSO vs. GA: žádné operátory, částice mají paměť ($\vec{p}_i, \vec{g}$), informace teče jednosměrně od lepších.
- ACO: pravděpodobnost přechodu $p_{ij} \propto \tau_{ij}^\alpha \eta_{ij}^\beta$ s $\eta = 1/d$; aktualizace $\tau \leftarrow (1 - \rho)\tau + \sum_k Q/L_k$ (odpařování + uložení úměrné kvalitě).
- AS (původní, všichni aktualizují) vs. ACS (globální update jen nejlepším, lokální update při průchodu, pseudonáhodné pravidlo q_0) vs. MAX-MIN (meze feromonu). Akce démona = lokální prohledávání.
- ABC (dělnice / pozorovatelky / průzkumnice s limitem = restart), firefly (přitažlivost klesá exponenciálně se vzdáleností).
- Klíčová slova: stigmergie, emergence, decentralizace, kladná zpětná vazba + odpařování, vhodnost pro dynamické prostředí.

11 Lokální prohledávání a memetické algoritmy

11.1 Lokální prohledávání a horolezecký algoritmus

Okolí a lokální prohledávání

Pro prostor řešení X definujeme *okolí* $N(x) \subseteq X$ — množinu řešení dosažitelných z x jednou elementární změnou (překlopení bitu, prohození dvou měst, 2-opt krok). *Lokální prohledávání* iterativně přechází do souseda podle daného pravidla. *Lokální optimum* vzhledem k N je x takové, že žádný soused není lepší — závisí tedy na definici okolí!

Horolezecký algoritmus (*hill climbing*)

- 1. Steepest ascent (nejstrmější stoupání):** projdi *celé* okolí a přejdi do nejlepšího souseda, je-li lepší než x ; jinak skonči.
- 2. First-improvement (první zlepšení):** procházej okolí a přejdi do prvního nalezeného lepšího souseda (levnější, často stejně dobré).
- 3. Stochastic hill climbing:** vyber náhodného souseda; přijmi jej, je-li lepší (případně s pravděpodobností úměrnou zlepšení).
- 4. Random-restart hill climbing:** po uvíznutí začni z nové náhodné pozice; pamatuj si nejlepší nalezené řešení. S rostoucím počtem restartů konverguje pravděpodobnost nalezení globálního optima k 1.

Slabiny: uvíznutí v lokálním optimu, na *plošinách* (plateau, kde jsou všichni sousedé stejní — pomáhají *sideways moves*) a na hřebenech. Výhody: jednoduchost, malá paměť, rychlost — proto je ideální jako vylepšovací procedura uvnitř EA.

Nelder–Mead (*downhill simplex*). Pro spojitě domény se jako lokální metoda bez gradientu často používá simplexová metoda: udržuje se $n + 1$ bodů v $\mathbb{R}^n$ (simplex) a nejhorší z nich se nahrazuje pomocí operací *reflexe*, *expanze*, *kontrakce* a *smrštění* simplexu. Je to nejběžnější „mem“ v memetických algoritmech pro reálné parametry a spolu s hill climbingem a simulovaným žíháním patří do standardní výbavy black-box optimalizace.

11.2 Simulované žhání

Simulované žhání (*simulated annealing*, SA)

Kirkpatrick, Gelatt, Vecchi (1983); fyzikální analogie s pomalým chladnutím kovu, při němž atomy zaujmou uspořádání s minimální energií. Algoritmus je horolezec, který **připouští i zhoršující kroky** s pravděpodobností klesající s velikostí zhoršení a s teplotou.

Metropolisovo kritérium

Pro minimalizaci, kandidáta $y \in N(x)$ a $\Delta = f(y) - f(x)$:

$$P(\text{přijmout } y) = \begin{cases} 1, & \Delta \leq 0 \quad (\text{zlepšení}), \\ e^{-\Delta/T}, & \Delta > 0 \quad (\text{zhoršení}), \end{cases}$$

kde $T > 0$ je teplota.

Číselný příklad. Zhoršení $\Delta = 2$. Při $T = 1$ je $P = e^{-2} = 0,135$; při $T = 0,5$ je $P = e^{-4} = 0,018$; při $T = 5$ je $P = e^{-0,4} = 0,67$. Na začátku (horko) se tedy algoritmus chová téměř jako náhodná procházka, na konci (chladno) jako čistý horolezec.

Algoritmus 8 Simulované žhání

```
1:  $x \leftarrow$  počáteční řešení;  $T \leftarrow T_0$ ;  $x^* \leftarrow x$ 
2: while  $T > T_{\min}$  a není splněna ukončovací podmínka do
3:   for  $k$  krát (délka Markovova řetězce při dané teplotě) do
4:     vyber náhodně  $y \in N(x)$ ;  $\Delta \leftarrow f(y) - f(x)$ 
5:     if  $\Delta \leq 0$  nebo  $\text{rand}() < e^{-\Delta/T}$  then  $x \leftarrow y$ 
6:   end if
7:   if  $f(x) < f(x^*)$  then  $x^* \leftarrow x$ 
8:   end if
9: end for
10:   $T \leftarrow \text{cool}(T)$ 
11: end while
```

Výstup: x^*

Rozvrh chlazení (*cooling schedule*).

- *Geometrický*: $T_{k+1} = \alpha T_k$, $\alpha \in [0,8; 0,99]$ — v praxi nejběžnější.
- *Lineární*: $T_k = T_0 - \beta k$.
- *Logaritmický*: $T_k = c / \log(k + 1)$ — jediný, pro který je dokázána konvergence.
- Adaptivní (reheating: při stagnaci se teplota zvýší).

Věta 11.1 (Konvergence SA). *Při logaritmickém rozvrhu $T_k \geq c / \log(k + 1)$ s dostatečně velkou konstantou c (souvisí s výškou nejhlubší „bariéry“ v krajině) konverguje simulované žhání k množině globálních optim s pravděpodobností 1.*

Prakticky je tento rozvrh nepoužitelně pomalý; SA je tedy teoreticky uspokojivá, ale v praxi se používají rychlejší heuristické rozvrhy. Formálně je SA nehomogenní Markovův řetězec; při pevné T konverguje k Boltzmannovu rozdělení $\pi(x) \propto e^{-f(x)/T}$, které se pro $T \rightarrow 0$ koncentruje na globálních optimech. Souvislost s EA: *Boltzmannovský turnaj* (kap. 1.5) přenáší tutéž myšlenku do selekce.

11.3 Tabu prohledávání

Tabu search (Glover, 1986)

Lokální prohledávání s *krátkodobou pamětí*: uchovává se *tabu list* nedávno provedených tahů (efektivnější než ukládat celá řešení), které jsou po dobu *tabu tenure* zakázány. V každém kroku se přejde do **nejlepšího nezakázaného souseda**, a to i tehdy, je-li horší než aktuální řešení — tím se algoritmus dostane z lokálního optima a necyklí.

- **Tabu tenure T** : statická (pevný počet iterací) nebo dynamická (náhodná z rozsahu).
- **Aspirační kritérium**: tabu se poruší, vede-li tah k řešení lepšímu než dosud nejlepší nalezené (jinak by tabu mohlo blokovat postup).
- **Střednědobá paměť**: pravidla směřující hledání do slibných oblastí (intenzifikace).
- **Dlouhodobá paměť / frekvenční paměť**: počítadlo, jak často byl který prvek řešení použit; často navštěvované konfigurace se penalizují (diverzifikace), případně se restartuje z málo navštívené oblasti.

Aplikace: TSP (výměny hran, *ejection chains*), rozvozní úlohy, sekvenování DNA, rozvrhování. Pro: únik z lokálních optim, žádné cyklení, funguje v diskrétních i spojitých doménách. Proti: mnoho parametrů, vyšší výpočetní náročnost (procházení celého okolí).

11.4 Scatter search

Populační metaheuristika zaměřená na **diverzitu**. Udržuje malou *referenční množinu R* (řádově desítky jedinců), do níž se vybírají jedinci podle kombinace kvality a vzájemné vzdálenosti (maximalizuje se minimální vzdálenost nově přidávaného jedince od zbytku R). Cyklus: (1) *diverzifikace* — chytrá inicializace, (2) *generování podmnožin* — typicky všechny dvojice z R , (3) *kombinace* — aritmetické (či permutační) křížení, (4) *vylepšení* — lokální prohledávání, mutace, tabu search, (5) *aktualizace R* . Referenční množinu lze aktualizovat *inkrementálně* (přidá se jeden či několik jedinců za generaci), *úplnou obnovou* každou generaci, nebo ji tvořit už ve vnitřním cyklu — noví jedinci se do R zařazují okamžitě a rekombinuje se rovnou s nimi. Vhodné pro úlohy s velmi drahou fitness (např. black-box optimalizátor OptQuest).

11.5 Memetické algoritmy

Mem a memetický algoritmus

Mem (Dawkins, 1976) je jednotka kulturní informace šířící se napodobováním — „gen kultury“. *Memetický algoritmus* (Moscato, 1989) je hybridní metoda: **evoluční algoritmus s vnořeným lokálním prohledáváním**, které vylepšuje jednotlivé jedince („individuální učení“). Synonyma: hybridní EA, genetické lokální prohledávání, lamarckovské či baldwinovské EA, kulturní algoritmy.

Algoritmus 9 Memetický algoritmus

- 1: inicializuj populaci
 - 2: **while** není splněna ukončovací podmínka **do**
 - 3: ohodnoť jedince v populaci
 - 4: vytvoř novou populaci stochastickými operátory (selekce, křížení, mutace)
 - 5: vyber podmnožinu O jedinců, kteří projdou vylepšením
 - 6: **for** každého $x \in O$ **do**
 - 7: proved' individuální učení x pomocí memu (lokálního prohledávače), s pravděpodobností p po dobu t
 - 8: výsledek zpracuj **lamarckovsky** nebo **baldwinovsky**
 - 9: **end for**
 - 10: **end while**
-

Podstata: lokální prohledávání vnořené do evolučního cyklu. Genetické operátory přenášejí informaci *mezi* jednotlivými běhy lokálního prohledávače, takže celkové hledání je mnohem efektivnější než opakovaný restart lokální metody (Krasnogor & Smith). Jako mem lze použít hill climbing, simulované žíhání, tabu search, kombinatorickou heuristiku (2-opt) nebo gradientní metodu (zpětné šíření chyby při učení neuronové sítě).

Lamarckismus vs. baldwinismus

Co udělat s vylepšeným jedincem $x' = LS(x)$?

- **Lamarckovský:** do populace se vloží x' i s jeho fitness — *genotyp se změní podle fenotypu*. Z darwinistického hlediska „nekošer“, ale prakticky rychlejší.
- **Baldwinovský:** jedinci x se přiřadí *fitness* vylepšeného x' , ale genotyp x zůstane *nezměněn*. Darwinisticky korektní; fitness se interpretuje jako „potenciál“ jedince (*Baldwinův efekt*: schopnost učit se zvýhodňuje jedince a evoluce se posléze „doučí“ sama — genetická asimilace).

Lamarckismus konverguje rychleji, ale více snižuje diverzitu a může zkreslit krajinu; baldwinismus zachovává diverzitu a vyhlazuje fitness krajinu, je však pomalejší.

Návrhové otázky. Které jedince vylepšovat (všechny / jen elitu / náhodný podíl p)? Jak dlouho (t kroků)? Jak silně (hloubka lokálního prohledávání)? Přílišné lokální prohledávání znamená zbytečné náklady a ztrátu diverzity; příliš malé nepřináší užitek.

Memetické počítání (*memetic computing*). Zobecnění: memy nejsou známy předem, ale jsou samy předmětem hledání.

- **Multi-mem MA:** mem je zakódován *jako součást genotypu* jedince; dědí se, kříží se (potomek zdědí mem lepšího rodiče, nebo náhodně) a používá se pro vylepšení tohoto jedince.
- **Meta-lamarckovský MA:** memy mají *vlastní populaci* a s jedinci koevolví; při vylepšení se jedinci přiřadí mem, úspěch jedince je odměnou pro mem, nad memy běží druhý evoluční algoritmus.
- **Memetický prostor** jako protějšek prostoru fenotypů: memy lze vybírat heuristikami, nechat je soutěžit podle úspěšnosti, dělit do subpopulací nebo řídit meta-memetickým algoritmem.

11.6 Ladění a řízení parametrů

Úzce souvisí: EA má mnoho parametrů (velikost populace, p_C , p_M , velikost turnaje, míra elitismu). Ty nejsou nezávislé a vyčerpávající prohledání není možné.

- **Ladění (*tuning*)** — parametry se určí *před* během (pokus-omyl, grid search, meta-EA, iterované závodění — irace).
- **Řízení (*control*)** — parametry se mění *během* běhu:
 - *Pevná strategie* (deterministická): závislost jen na čase (např. $\sigma(t) = 1 - 0,9t/T$, pokles p_M o 0,1 % za generaci); stochastická varianta = simulované žíhání.
 - *Adaptivní:* podle zpětné vazby z běhu (kvalita řešení, rozptyl fitness, diverzita) — např. pravidlo 1/5, hypermutace při poklesu fitness, adaptivní zjemňování reprezentace (zvyšování počtu bitů na reálné číslo, klesne-li diverzita měřená Hammingovou vzdáleností nejlepšího a nejhoršího jedince).
 - *Samoadaptivní:* parametry jsou součástí genomu a evolvují (ES, EP, multi-mem MA).
- Řídit lze i *velikost populace* nepřímo: každému novému jedinci se přidělí „délka života“ úměrná jeho fitness; po jejím vypršení zemře. Lepší jedinci žijí déle, velikost populace je odvozená dynamická veličina.

11.7 Shrnutí k zkoušce

- Hill climbing: steepest ascent / first improvement / stochastic / s náhodnými restarty; problémy: lokální optima, plošiny, hřebeny. Pro spojitě domény navíc Nelder–Mead (reflexe, expanze, kontrakce, smrštění simplexu).
- SA: Metropolisovo kritérium $P = \min(1, e^{-\Delta/T})$; rozvrh chlazení (geometrický v praxi, logaritmický pro důkaz konvergence); pro pevné T konverguje k Boltzmannovu rozdělení $\propto e^{-f/T}$.
- Tabu search: tabu list tahů, tenure, aspirační kritérium, střednědobá a dlouhodobá (frekvenční) paměť; vždy se jde do nejlepšího nezakázaného souseda, i když je horší.
- Scatter search: referenční množina kombinující kvalitu a diverzitu, aritmetické kombinace + vylepšení.
- Memetický algoritmus = EA + lokální prohledávání nad jedinci; **lamarckismus** (mění genotyp, rychlejší) vs. **baldwinismus** (mění jen fitness, zachovává diverzitu). Memetic computing: memy v genomu nebo v koevolvující populaci.
- Ladění vs. řízení parametrů; pevné / adaptivní / samoadaptivní strategie.

12 Evoluce pravidlových a expertních systémů

12.1 Michigan vs. Pittsburgh

Cílem je naučit se **sadu pravidel** tvaru IF podmínka THEN akce z trénovacích dat nebo z interakce s prostředím (dobývání znalostí, expertní systémy, řízení robotů, zpětnovazební učení). Existují dva základní přístupy:

	Michigan (Holland)	Pittsburgh (Smith)
jedinec	jedno pravidlo	celá sada pravidel
řešení	celá populace dohromady	jeden jedinec
fitness	síla/přesnost pravidla (nutné rozdělení odměny)	kvalita celého systému (přímochař)
operátory	standardní, evoluce probíhá jen občas a na části populace	složitě, desítky operátorů nad sadami i jednotlivými pravidly
výhody	on-line učení, malý prostor, přirozené pro RL	jednoznačná fitness, žádný credit assignment
nevýhody	přirazení zásluh, pravidla musí spolupracovat, ale zároveň soutěží	velcí jedinci, drahé vyhodnocení, riziko bloatu

12.2 Learning classifier systems (Michigan)

LCS (*learning classifier system*)

Systém, kde populace jednoduchých pravidel (*klasifikátorů*) tvoří dohromady expertní či řídicí systém. Klasifikátor má levou stranu (podmínku) jako řetězec nad $\{0, 1, *\}$ porovnávaný se vstupem, pravou stranu (akci či klasifikační třídu) a **váhu** / **sílu** odrážející jeho úspěšnost, která slouží jako fitness. Evoluce neprobíhá generačně, ale průběžně a jen na části populace.

Architektura LCS. Klíčové je uvědomit si, že *expertním systémem je celá populace*, nikoli jedinec. Systém tvoří smyčka:

1. **Detektory** převedou stav prostředí na vstupní zprávu.
2. Zpráva se uloží do **bufferu zpráv** a porovná se s levými stranami všech pravidel $\Rightarrow$ *match set*.
3. Mezi vyhovujícími pravidly proběhne **aukce/soutěž** o právo jednat (podle síly pravidla).
4. Vítězné pravidlo zapíše svou pravou stranu buď do bufferu (vnitřní zpráva), nebo přes **efektory** do prostředí jako akci.

5. Prostředí vrátí **odměnu**; ta se rozdělí mechanismem *bucket brigade* mezi pravidla, která k ní vedla.
6. Občas a jen na části populace běží **GA**, který pravidla obměňuje.

Problém reaktivity. Čistě reaktivní systém nemá vnitřní paměť. Holland proto zavedl *zprávy* (*messages*): pravá strana pravidla může kromě akce zapsat vnitřní zprávu do bufferu, levá strana má *receptory* pro její zachycení. Vzniká tak řetězení pravidel — systém získá vnitřní stav (a tím i výpočetní sílu), ale zároveň s ním přichází problém, jak rozdělit odměnu mezi celý řetězec.

Bucket brigade (algoritmus „požárního řetězu“)

Mechanismus rozdělení odměny v LCS. Pravidlo, které chce být aplikováno, musí „zaplatit“ část své síly pravidlu, jež ho aktivovalo (jako by kupovalo právo jednat). Odměna z prostředí připadne poslednímu článku řetězu a postupně „proteče“ zpět k pravidlům, která k ní vedla. Prakticky je obtížné vyvážit tuto ekonomiku pravidel, dnes se proto téměř nepoužívá.

ZCS (Wilson, 1994) — „zero-level“ LCS. Zjednodušení: žádné vnitřní zprávy, žádná složitá redistribuce.

- P — populace pravidel; vstup x z detektorů.
- M — *match set*: pravidla, jejichž podmínka odpovídá x .
- Akce a se z M vybere ruletou podle síly pravidel; $A \subseteq M$ — *action set*: pravidla prosazující zvolenou akci.
- **Posílení**: z pravidel v A se odebere podíl β jejich síly do „kbelíku“ B ; po obdržení odměny r se každému pravidlu v A přičte $\beta r/|A|$ a pravidlům z předchozího action setu A_{-1} se přičte $\gamma B/|A_{-1}|$. Pravidla z $M \setminus A$ (odpovídala, ale prosazovala jinou akci) jsou zdaněna.
- **Cover operátor**: neexistuje-li pro aktuální vstup žádné pravidlo, vygeneruje se ad hoc (do podmínky se náhodně přidají hvězdičky, akce se zvolí náhodně).

XCS (Wilson, 1995) — fitness založená na přesnosti. Slabiny ZCS: neevoluje *úplný* systém pravidel pokrývající všechny situace; pravidla na začátku řetězců jsou zřídka odměňována a vymírají; pravidla vedoucí k malým, ale správně předpovězeným odměnám také vymírají. Příčinou je, že síla sloužila zároveň jako fitness pro GA i jako kritérium výběru akce. Wilsonova myšlenka: *predikce* má odhadovat odměnu, ale *evoluce* má preferovat **spolehlivé (přesné)** klasifikátory.

XCS — klasifikátor jako pětice

(c, a, p, ϵ, F) : podmínka, akce, predikce odměny p , chyba predikce ϵ , fitness F . Přesnost se počítá jako klesající funkce chyby, $\kappa = \alpha(\epsilon/\epsilon_0)^{-\nu}$ (pro $\epsilon > \epsilon_0$, jinak $\kappa = 1$; α, ϵ_0, ν jsou parametry); fitness je *relativní* přesnost v rámci action setu. Predikce a chyba se aktualizují pravidly Q-learningu.

Běh XCS: podle vstupu se sestaví match set M , rozdělí se na action sety A_i podle akcí; pro každou akci se predikuje výplata jako průměr predikcí vážený fitness; akce se zvolí buď maximem (exploatace), nebo ruletou (explorace); provede se; odměna se rozdělí do action setu předchozího kroku (aktualizace p pomocí $r + \gamma \max_a p_a$, pak ϵ , pak F); GA běží jako **nikový** — pouze uvnitř action setu, nikoli nad celou populací.

Výsledek: XCS propojuje LCS se zpětnovazebním učením (Q-learning jako mechanismus přiřazení zásluh), evoluje **úplnou a minimální** mapu vstup–výstup (nejen optimální politiku, ale i kompaktní srozumitelný popis) a má řadu praktických aplikací (dobývání znalostí, řízení, bioinformatika).

Dnešní rodina LCS. Podle úlohy se rozlišují: *XCS* (zpětnovazební učení, predikce odměny), *UCS* (*sUpervised Classifier System*) — varianta pro učení s učitelem, kde je správná třída známa, takže se místo predikce odměny rovnou měří přesnost klasifikace, a *XCSF* pro aproximaci spojitých

funkcí (pravá strana je lineární model, nikoli konstanta). Podmínky se dnes běžně zapisují nejen nad $\{0, 1, *\}$, ale i intervaly pro reálné atributy, což z LCS dělá plnohodnotný nástroj dobývání znalostí — s tou předností, že výsledkem je **čitelná sada pravidel**.

12.3 Pittsburghský přístup a systém GIL

Jedinec = celá sada pravidel, tedy kompletní klasifikační systém. Hodnocení je složitější (priority pravidel, konflikty, falešně pozitivní a negativní odpovědi) a operátorů je celá řada, na několika úrovních. Důraz se klade na bohatou doménovou reprezentaci (množiny, výčty, intervaly).

GIL je klasický představitel: pro binární klasifikaci, jedinec klasifikuje implicitně do jedné třídy (pravidla nemají pravou stranu). Jedinec je *disjunkce komplexů*, komplex je *konjunkce selektorů* (po jednom pro každou proměnnou) a selektor je *disjunkce hodnot* z domény dané proměnné, zakódovaná bitovou maskou. Například

$$((X=A_1) \wedge (Z=C_3)) \vee ((X=A_2) \wedge (Y=B_2)) \equiv [001|11|0011 \vee 010|10|1111].$$

Operátory na třech úrovních:

- *jedinec*: výměna pravidel, kopie pravidla, generalizace pravidla, smazání pravidla, specializace pravidla, zahrnutí pozitivního příkladu;
- *komplex*: rozdělení komplexu na jednom selektoru, generalizace selektoru (nahrazení $11 \dots 1$), specializace, zahrnutí negativního příkladu;
- *selektor*: mutace $0 \leftrightarrow 1$, rozšíření $0 \rightarrow 1$, zúžení $1 \rightarrow 0$.

Tyto operátory přímo odpovídají operacím induktivního učení (generalizace/specializace), takže GIL je vlastně evolučně řízený induktivní algoritmus.

12.4 Připomenutí: metriky klasifikace

Z matice záměn (TP, FP, FN, TN):

$$\begin{aligned} \text{přesnost (accuracy)} &= \frac{TP + TN}{TP + TN + FP + FN}, \\ \text{senzitivita} &= \frac{TP}{TP + FN}, \quad \text{specifita} = \frac{TN}{TN + FP}. \end{aligned}$$

Fitness pravidlového systému bývá kombinací přesnosti, pokrytí a jednoduchosti (počet pravidel) — tedy přirozeně vícekritériální úloha.

12.5 Shrnutí k zkoušce

- **Michigan**: jedinec = jedno pravidlo, řešením je celá populace, nutné rozdělení odměny (credit assignment). **Pittsburgh**: jedinec = celá sada pravidel, fitness přímočará, operátory složité (GIL).
- Hollandův LCS: podmínky nad $\{0, 1, *\}$, vnitřní zprávy a receptory, bucket brigade (pravidlo platí předchůdci za právo jednat).
- ZCS: bez zpráv; match set $M \rightarrow$ ruletový výběr akce $\rightarrow$ action set A ; posílení A a A_{-1} , daň pro $M \setminus A$; cover operátor.
- XCS: klasifikátor (c, a, p, ϵ, F) , **fitness podle přesnosti predikce** $\kappa = \alpha(\epsilon/\epsilon_0)^{-\nu}$, aktualizace Q-learningem, nikový GA v action setu; evoluje úplnou a minimální mapu vstup-výstup. Varianty: UCS (učení s učitelem), XCSF (aproximace funkcí).
- Umět vysvětlit, proč přesnost jako fitness řeší nedostatky „strength-based“ systémů.

13 Neuroevoluce

Neuroevoluce (*neuroevolution*)

„Neuroevoluce je metoda pro úpravu vah, topologií nebo ansámbků neuronových sítí za účelem naučení konkrétní úlohy. Evoluční výpočty se používají k hledání parametrů sítě maximalizujících fitness, která měří výkon v úloze. Ve srovnání s jinými metodami učení je neuroevoluce velmi obecná — umožňuje učení bez explicitních cílových hodnot, s nediferencovatelnými aktivačními funkcemi a s rekurentními sítěmi.“ (Miikkulainen, 2017)

Co lze evolvovat: váhy, další parametry (aktivační funkce, konstanty učení), architekturu (topologii, propojení), architekturu i váhy zároveň, případně parametry samotného učicího algoritmu. Hlavní doménou je **zpětnovazební učení** a robotika, kde nejsou k dispozici cílové výstupy pro učení s učitelem.

13.1 Evoluce vah

Přímočaré: váhy se zakódují do vektoru pevné délky a použije se reálný GA, ES nebo DE se standardními operátory. Vlastnosti:

- Pomalejší než specializované gradientní metody (zpětné šíření chyby) pro učení s učitelem.
- + snadná paralelizace, nepotřebuje gradient (funguje pro nediferencovatelné a rekurentní sítě), použitelné pro RL.
- **Problém konkurujících konvencí** (*competing conventions*, permutační problém): tatáž funkce sítě odpovídá mnoha různým genotypům (permutace skrytých neuronů), takže křížení dvou funkčně shodných rodičů často vytvoří nefunkčního potomka.
- Novodobý návrat zájmu: evoluce vah je konkurenceschopná i pro velké sítě, používá-li se hodnocení na minibatchích.

13.2 Evoluce architektury

Fitness architektury = odhad výkonu sítě: síť je nutné postavit, inicializovat a naučit (zpětným šířením či jiným EA), nejlépe vícekrát — proto je vyhodnocení fitness drahé a zašuměné.

- **Přímé kódování** (*direct encoding*): struktura propojení jako binární matice; jedinec je buď linearizovaná matice (standardní operátory), nebo matice sama (speciální 2D operátory). Jednoduché, ale neškáluje (velikost genomu roste kvadraticky).
- **Gramatické kódování** (Kitano, 1990): matici propojení generuje jednoduchá 2D formální gramatika a evolvují se *pravidla gramatiky*. Kompaktní (logaritmické), umožňuje opakující se motivy, ale je nepřímé a hůře laditelné.
- **Buněčné kódování** (*cellular encoding*, Gruau): jedincem je **program v GP**, který popisuje, jak síť *vyrůst* z jediné buňky pomocí operací: přidej neuron, rozděl neuron sériově, rozděl paralelně, přepoj synapsi, změň váhu. Vývojový (developmental) přístup.
- **Simulovaný růst** spojů ve 2D rovině — rané pokusy v evoluční robotice, neškalovalo.

GNARL (Angeline a kol., 1993). Evoluční programování nad grafy: vstupní a výstupní neurony jsou pevné, skryté neurony a spoje se evolvují; architektura i váhy *současně*. Dva druhy mutací: *strukturální* (přidej neuron bez spojů, přidej spoj s nulovou váhou, smaž neuron, smaž spoj) a *parametrická* (zaujatá mutace váhy). Síla mutace je řízena *teplotou* (přístup simulovaného žíhání); strukturální změny mají být malé a nepříliš destruktivní. Selektce: rodiči je lepší polovina populace. Křížení se nepoužívá (EP).

13.3 NEAT

NEAT (*NeuroEvolution of Augmenting Topologies*, K. Stanley, 2002)

Síť je reprezentována jako **seznam hran**; každý gen-hrana obsahuje: vstupní a výstupní neuron, váhu, příznak *enabled/disabled* a **globální inovační číslo** (*innovation number*), které se přiděluje při prvním vzniku dané strukturální novinky.

Tři klíčové myšlenky:

1. **Inovační čísla řeší problém konkurujících konvencí.** Kříží se pouze geny se *shodným inovačním číslem* (mají tedy společný evoluční původ); geny přebývající u jednoho rodiče (*disjoint* a *excess*) se přebírají od zdatnějšího rodiče. Křížení tak nerozbíjí strukturu — žádné „bezhlavé kuře“.
2. **Speciace (niching) chrání inovace.** Na vektorech inovačních čísel se definuje míra podobnosti

$$\delta = \frac{c_1 E}{N} + \frac{c_2 D}{N} + c_3 \overline{\Delta W},$$

kde E (*excess*) je počet genů, jejichž inovační číslo leží za nejvyšším inovačním číslem druhého genomu, D (*disjoint*) počet genů chybějících u druhého rodiče *uvnitř* společného rozsahu inovačních čísel, $\overline{\Delta W}$ průměrný rozdíl vah genů se *shodným* inovačním číslem a N velikost většího genomu (normalizace). Síť s δ pod prahem tvoří jeden *druh*; fitness se počítá jako sdílená (relativní v rámci druhu). Nová topologie má tak čas „doladit váhy“ dřív, než ji vytlačí zavedená řešení — jinak by strukturální novinka, která zpočátku fitness zhoršuje, okamžitě vymřela.

3. **Postupné narůstání složitosti** (*complexification*): populace startuje z minimálních sítí (jen vstupy a výstupy) a teprve mutacemi *přidej spoj* a *přidej neuron* (rozdělí existující spoj: původní se zakáže, vzniknou dva nové) roste. Prohledávání tedy začíná v prostoru nejnižší dimenze a zvyšuje ji jen tehdy, když se to vyplatí.

13.4 HyperNEAT a CPPN

HyperNEAT

Rozšíření NEAT založené na dvou myšlenkách: (1) **Substrát** — váhy sítě se nekódují jako vektor čísel, ale *generuje je funkce* S , která dostane souřadnice dvou neuronů v rovině a vrátí váhu jejich spojení: $S : \mathbb{R}^4 \rightarrow \mathbb{R}$ (odtud „hyper“ — zobrazuje hyperkrychli do $\mathbb{R}$; ve 3D uspořádání jde o $S : \mathbb{R}^6 \rightarrow \mathbb{R}$). (2) **CPPN** (*compositional pattern-producing network*) — funkce S je reprezentována sítí ve tvaru DAG, jejíž uzly počítají různé funkce ze slovníku (sigmoida, gaussovka, sinus, kosinus, polynomy, absolutní hodnota). Tuto CPPN evoluuje samotný NEAT.

Přínos: reprezentace vah funkcí umožňuje vyjádřit **symetrie, opakování a regularity** úlohy a generovat síť s milióny spojů z malého genomu (geometrie úlohy je zakódovaná v souřadnicích substrátu). Varianty: evoluce hlubokých struktur, evoluce i pozic neuronů (ES-HyperNEAT), 3D substráty. Ukázky estetického potenciálu CPPN: Picbreeder, EndlessForms.

13.5 Neuroevoluce v éře hlubokých sítí

NAS (*neural architecture search*). Hledání architektury hlubokých sítí se popisuje třemi osami:

- **Prohledávaný prostor:** lineární (posloupnost vrstev — typ vrstvy a její hyperparametry: počet jednotek, velikost jádra, počet filtrů, krok, typ poolingů; chromozom je proměnné délky a podmíněný), DAG modulů (moderní architektury mají *skip* spoje; hledají se *buňky* — „normal“ zachovávající rozměr a „reduction“ zmenšující — které se pak stohují), nebo podarchitektura jedné velké „super sítě“.
- **Prohledávací strategie:** evoluční algoritmy, náhodné prohledávání (překvapivě silná baseline), bayesovská optimalizace, zpětnovazební učení, gradientní metody (DARTS).

- **Odhad výkonu:** zkrácené učení, nižší rozlišení, menší sítě, sdílení vah, prediktory učicích křivek; jedno- nebo vícekritériální (přesnost vs. latence, počet parametrů).

Konkrétní systémy.

- **ENAS** (efficient NAS): prostor je jediný velký DAG, hledání znamená výběr podgrafu; *sdílení vah* mezi kandidáty odstraňuje nutnost učit každý model od nuly (řádkové zrychlení).
- **DeepNEAT**: uzel chromozomu není neuron, ale celá vrstva DNN; nutné opravy, aby na sebe rozměry vstupů a výstupů navazovaly.
- **CoDeepNEAT**: koevoluce dvou populací — *modulů* (malé DNN) a *blueprintů* (grafů, jejichž uzly odkazují na moduly). Aplikováno i na LSTM se speciálními mutacemi (změna vnitřní struktury buňky, hledání uspořádání jednotek ve vrstvách).
- **Hill-climbing NAS** (Elsken, Metzen, Hutter 2018): začni malou předtrénovanou sítí a v horolezecké smyčce aplikuj *morfismy* sítě (prohloub — přidej ConvBlock; rozšiř — zvýš počet kanálů; přidej skip spoje), potomka krátce douč. Ukázka memetického přístupu v NAS.
- **Deep neuroevolution / ES pro RL** (OpenAI 2017; Salimans a kol., Miikkulainen a Clune): jednoduchá ES *pouze s mutacemi* nad milióny vah hlubokých sítí pro úlohy MuJoCo a Atari. ES lze chápat jako odhad gradientu vyhlazené fitness; algoritmus je triviálně paralelizovatelný (uzly si vyměňují jen semínka generátoru náhodných čísel) a překonal A3C zhruba $10\times$ v reálném čase. Pro učení s učitelem (klasifikace obrázků) zůstává zpětné šíření chyby výrazně rychlejší.

13.6 Shrnutí k zkoušce

- Co se evoluuje: váhy / topologie / obojí / hyperparametry učení. Výhody EA: bez gradientu, rekurentní sítě, RL, paralelizace.
- Problém konkurujících konvencí (permutační problém) — křížení sítí je bez opatření destruktivní.
- Kódování architektury: přímé (matice), gramatické (Kitano), buněčné (Gruau, GP program pro růst sítě); GNARL = EP nad grafy, strukturální i parametrické mutace řízené teplotou.
- NEAT: seznam hran s **inovačními čísly** (křížení jen shodných genů), **speciace** podle podobnosti genomů se sdílenou fitness (ochrana nových topologií), **complexification** od minimální sítě.
- HyperNEAT: substrát $S : \mathbb{R}^4 \rightarrow \mathbb{R}$ generující váhy z souřadnic neuronů, reprezentovaný CPPN (DAG s různými aktivačními funkcemi) evolvovaným pomocí NEAT; zachycuje symetrie a regularity.
- NAS: prostor (lineární / DAG buněk / super síť), strategie (EA, RL, bayesovská, gradientní), odhad výkonu (sdílení vah — ENAS); DeepNEAT a CoDeepNEAT (moduly + blueprints); ES pro deep RL.

14 Kombinatorické úlohy a práce s omezeními

Evoluční algoritmy jsou oblíbeným nástrojem pro NP-těžké kombinatorické úlohy: nezaručí optimum, ale poskytnou dobré řešení v rozumném čase a snadno se přizpůsobí dodatečným (třeba i ošklivým) omezením reálné úlohy.

14.1 Problém batohu

Zadání. Batoh o kapacitě $C_{\max}$, N předmětů s cenami v_i a objemy c_i . Maximalizuj $\sum_i x_i v_i$ za podmínky $\sum_i x_i c_i \leq C_{\max}$, $x_i \in \{0, 1\}$.

Kódování je triviální — bitová mapa, např. 0110010 znamená „vezmi předměty 2, 3 a 6“. Standardní operátory (křížení, bit-flip mutace) fungují. **Problém je fitness:** jedinci mohou porušovat kapacitu. Máme dvě protichůdná kritéria: maximalizovat $\sum v_i$ a zároveň splnit $\sum c_i \leq C_{\max}$.

14.2 Tři obecné strategie pro omezení

Práce s omezeními v EA

- 1. Penalizace** (*penalty*): nepřipustná řešení zůstávají v populaci, ale jejich fitness se sníží, např. $f'(x) = \sum_i x_i v_i - \alpha \max(0, \sum_i x_i c_i - C_{\max})$. Volba α je zásadní: příliš malá $\Rightarrow$ populace se plní nepřipustnými řešeními, příliš velká $\Rightarrow$ ztrácí se možnost projít nepřipustnou oblastí ke slibnému řešení. Používá se i dynamická (α roste v čase) či adaptivní penalizace.
- 2. Oprava** (*repair*): nepřipustné řešení se opraví (u batohu: vyhazuj předměty s nejhorším poměrem v_i/c_i , dokud se vejde). Opravu lze provést jen pro výpočet fitness (baldwinovsky) nebo trvale zapsat do genotypu (lamarckovsky).
- 3. Dekodér** (*decoder*) / uzavřené operátory: kódování se navrhne tak, že *každý* genotyp reprezentuje přípustné řešení. U batohu: bit 1 znamená „přidej předmět, pokud se ještě vejde“, jinak jej přeskoč. Elegantní a účinné; nevýhodou je nejednoznačnost mapování (více genotypů dává tentýž fenotyp) a jeho horší lokalita.

Čtvrtá možnost: chápat porušení omezení jako **druhé kritérium** a použít vícekritériální EA (kap. 15) — výsledkem je pak celá Pareto fronta kompromisů mezi kvalitou řešení a mírou porušení omezení.

14.3 Problém obchodního cestujícího

TSP: N měst, najdi nejkratší okružní jízdu. Fitness je triviální (délka cesty), **problémem je kódování a zejména křížení**.

Reprezentace.

- Sousednostní** (*adjacency*): na pozici i je město j právě tehdy, vede-li hrana $i \rightarrow j$. Např. (248397156) odpovídá cestě 1–2–4–3–8–5–9–6–7. Každá cesta má jedinou reprezentaci, ale některé seznamy nejsou platné cesty. Klasické křížení nefunguje; zajímavé je, že schémata zde mají smysl (např. $(*3 * \dots) =$ všechny cesty s hranou 2–3). *Nepoužívat*.
- Ordinální** (*buffer*): udržuje se seznam měst, gen i je *pozice* města v aktuálním seznamu a po použití se město ze seznamu vyjme. Pro seznam (123456789) je cesta 1–2–4–3–8–5–9–6–7 zakódována jako (112141311). Motivací bylo umožnit standardní jednobodové křížení — to sice produkuje platné cesty, ale výsledek nemá s rodiči nic společného. *Také nepoužívat*.
- Cestová (permutační)**: cesta 5–1–7–8–9–4–6–2–3 je (517894623). Nejpřirozenější; vyžaduje speciální operátory — PMX, OX, CX, ER (kap. 3).
- Maticová**: binární matice, kde 1 na pozici (i, j) znamená buď hranu $i \rightarrow j$, nebo (častěji) že i předchází j . Speciální operátory: *konjunkce* (bitové AND obou rodičů a náhodné doplnění chybějících hran), *disjunkce* (rozdělení na kvadranty, dva se zahodí, odstraní se spory a zbytek se doplní náhodně).

Inicializace. Náhodné permutace, nebo konstrukční heuristiky: *nejbližší soused* (začni náhodným městem, vždy jdi do nejbližšího nenavštíveného) a *vkládání hran* (k částečné cestě T vyber nejbližší město c mimo T , najdi hranu $k-j$ v T minimalizující $d(k, c) + d(c, j) - d(k, j)$, hranu nahraď dvojicí $k-c, c-j$).

Mutace. Inverze úseku (nejdůležitější — odpovídá 2-opt kroku), vložení města jinam, posun podcesty, prohození dvou měst, prohození podcest, heuristické mutace 2-opt a 3-opt (vezmi dvě hrany a přepoj čtyři města jinak). Kombinace ES či GA s „chytrými mutacemi“ typu 2-opt je v praxi velmi účinná — de facto memetický algoritmus.

14.4 SAT

Zadání. Formule $f : \mathbb{B}^n \rightarrow \mathbb{B}$ v CNF (konjunkce klauzulí, klauzule = disjunkce literálů); najdi ohodnocení x s $f(x) = 1$. 2-SAT je v P, 3-SAT a výše je NP-úplný. Paradigmatická NP-úplná úloha.

Reprezentace.

- **Bitový řetězec** — jedinec je přímo ohodnocení proměnných.
- **Reálná** — proměnná x_j se nahradí reálným y_j , pravda odpovídá $y = 1$, nepravda $y = -1$, literály se přepíše jako $x \mapsto (1 - y)^2$ a $\neg x \mapsto (1 + y)^2$ a celý výraz se *minimalizuje*; výsledek se nakonec zaokrouhlí (záporné na -1 , kladné na 1). Hodnota literálu je tedy *penalta*: splněný literál dává 0, nesplněný 4. Proto se **disjunkce** (klauzule) přepíše jako **součin** a **konjunkce** (celá formule) jako **součet**: klauzule je splněna, stačí-li jeden literál — a součin je nulový, je-li nulový jeden činitel; formule je splněna, jsou-li splněny všechny klauzule — a součet je nulový, jsou-li nulové všechny sčítance. (Toto je nejčastěji prohazovaná dvojice; ověřte si ji vždy na $(x_1 \vee x_2)$ s $y_1 = 1, y_2 = -1$: součin dává $0 \cdot 4 = 0$ „splněno“, kdežto součet $0 + 4 = 4$ by klauzuli nesprávně prohlásil za nesplněnou.)
- **Klauzulová** — pro každou klauzuli se najdou přípustná ohodnocení jejích proměnných; jedinec je vektor ohodnocení pro všechny klauzule (délka $m \cdot k$ pro k -SAT s m klauzulemi). Nutná speciální fitness řešící globální nekonzistence.
- **Cestová** — procházejí se klauzule a v každé se zvolí alespoň jedno konzistentní ohodnocení; jedinec neurčuje všechny proměnné, reprezentuje tedy množinu řešení. Opět potřebuje speciální fitness.

Fitness. Samotné f je nepoužitelné (nabývá jen 0/1, krajina je „jehla v kupce sena“). Standardem je **počet splněných klauzulí**; zlepšení: vážení problematických klauzulí (váhy se aktualizují po několika iteracích) a *zjemňující funkce* (*refining function*) — k fitness se přidá regularizační člen $\alpha r(x)$, který rozliší jedince se stejným počtem splněných klauzulí (a pomůže tak z plošin). Váhy v_j zachycují tendenci proměnné x_j nabývat 1 či 0. Klasickou lokální heuristikou pro SAT je **WalkSAT/WSAT** — hodnotí řešení počtem splněných klauzulí a chytře volí směr lokálního kroku (kombinace hladového a náhodného výběru); kombinace EA + WSAT je typickým memetickým řešením.

14.5 Rozvrhování a plánování výroby

Školní rozvrh. Jedinec je rozvrh jako matice (řádky = učitelé, sloupce = třídy, hodnoty = kódy předmětů). Mutace míchá předměty, křížení vyměňuje mezi jedinci lepší řádky. Fitness kombinuje kvalitu jednotlivých řádků (spokojenost učitele) a další *měkká* kritéria (okna, rovnoměrnost). **Tvrdá omezení** (učitel nemůže učit dvě třídy naráz, dostupnost místností) je nutné respektovat *přímo v operátorech* — jinak vzniká příliš mnoho nepřipustných jedinců.

Job shop scheduling. Výrobky $o_1 \dots o_N$ se skládají z dílů $p_1 \dots p_K$; pro každý díl existuje více plánů výroby na strojích $m_1 \dots m_M$, přičemž přestavení stroje mezi produkty stojí čas. Fitness = celkový čas výroby (*makespan*). **Kritické je kódování:**

- *Permutační:* jedinec je jen pořadí výrobků, plány dílů dourčí dekodér. Jednoduché, lze použít křížení z TSP — ale ukazuje se málo efektivní: složitou část řeší dekodér a operátory z TSP zde nejsou vhodné.
- *Přímé:* jedinec je kompletní plán; nutné specializované a složité evoluční operátory. Náročnější na návrh, ale výkonnější.

14.6 Shrnutí k zkoušce

- Batoz: bitová mapa, triviální operátory, problém s omezením kapacity. Tři strategie pro omezení: **penalizace** (volba váhy α), **oprava** (lamarckovsky/baldwinovsky), **dekodér** („přidej, pokud se vejde“ — každý genotyp je přípustný); čtvrtá možnost = vícekritériální formulace.
- TSP: fitness triviální, problém je kódování. Sousednostní a ordinální reprezentace nepoužívat; permutační + PMX/OX/CX/ER, případně maticová. Inicializace: nejbližší soused, vkládání hran. Mutace: inverze (2-opt).

- SAT: fitness = počet splněných klauzulí (samotné f nestačí), vážení klauzulí, zjemňující funkce; reprezentace bitová, reálná, klauzulová, cestová; WalkSAT jako lokální heuristika. U reálné reprezentace ($x \mapsto (1-y)^2$, $\neg x \mapsto (1+y)^2$, minimalizace) pozor: hodnoty jsou *penalty*, takže disjunkce (klauzule) je **součin** a konjunkce (formule) **součet**.
- Rozvrhování: přímé maticové kódování, tvrdá omezení řešit v operátorech, měkká ve fitness; job shop — permutace s dekodérem vs. přímé kódování.
- Obecné pravidlo: u kombinatorických úloh rozhoduje kódování a operátory respektující strukturu úlohy; hybridizace s lokální heuristikou (2-opt, WSAT) je téměř vždy výhodná.

15 Vícekriteriální optimalizace

15.1 Dominance a Pareto fronta

Místo jediné fitness máme vektor kritérií $f_1, \dots, f_k$ (pro jednoduchost vše minimalizujeme). Kritéria jsou obvykle v konfliktu (cena vs. kvalita, přesnost vs. velikost modelu), takže neexistuje jediné nejlepší řešení.

Dominance

Pro jedince x, y definujeme:

- x **slabě dominuje** y ($x \preceq y$), právě když $f_i(x) \leq f_i(y)$ pro všechna $i = 1, \dots, k$.
- x **dominuje** y ($x \prec y$), právě když $x \preceq y$ a existuje j s $f_j(x) < f_j(y)$.
- x a y jsou **neporovnatelné**, pokud ani $x \prec y$, ani $y \prec x$.

Relace $\prec$ je *ostré* (striktní) částečné uspořádání — je ireflexivní a tranzitivní, ale nikoli úplné: proto obecně existuje celá množina navzájem neporovnatelných optim.

Pareto optimalita a Pareto fronta

Řešení x^* je *Pareto-optimální*, není-li dominováno žádným přípustným řešením. Množina všech Pareto-optimálních řešení v prostoru proměnných je *Pareto množina*, její obraz v prostoru kritérií je **Pareto fronta**. *Aproximace* Pareto fronty populací je cílem vícekriteriálních EA.

Dvojitý cíl algoritmu: (a) **konvergence** — dostat se co nejbližší skutečné frontě; (b) **diverzita** — pokrýt frontu rovnoměrně v celém rozsahu. Právě proto jsou EA pro tuto úlohu ideální: pracují s populací, a mohou tedy vrátit celou aproximaci fronty v jediném běhu.

Příklad. Body $A(1, 10)$, $B(2, 7)$, $C(4, 5)$, $D(8, 2)$, $E(5, 8)$ (minimalizace obou kritérií). E je dominován C ($4 \leq 5$, $5 \leq 8$, alespoň jedna ostrá) — E tedy do fronty nepatří. Body A, B, C, D jsou navzájem neporovnatelné a tvoří Pareto frontu.

15.2 Skalarizace — jednoduchá řešení

- **Lineární skalarizace:** $f = \sum_i w_i f_i$, poté standardní jednokriteriální EA (v kontextu MOEA označované jako SOEA). Nevýhody: neumíme volit váhy; jeden běh dá jeden bod fronty; a zásadně — **nekonvexní části fronty nelze lineární skalarizací nikdy nalézt**.
- **ε -constraint:** minimalizuj f_1 s omezeními $f_i \leq \varepsilon_i$ pro $i \geq 2$. Umí najít i nekonvexní části, ale je třeba volit konstanty ε_i a řešit úlohu s omezeními.
- **Čebyševova (Chebyshev) skalarizace:** minimalizuj $\max_i \lambda_i |f_i(x) - z_i^*|$, kde z^* je referenční (ideální) bod. Na rozdíl od lineární umí dosáhnout na libovolný bod fronty včetně nekonvexních částí.

15.3 Historické algoritmy

VEGA (Schaffer, 1985) — jeden z prvních MOEA. Populace N jedinců se pro každé kritérium f_i seřadí a vybere se N/k nejlepších podle f_i ; tyto podmnožiny se sloučí, zkříží a zmutují. Nevý-

hody: obtížné udržení diverzity a konvergence ke krajním bodům optimálním pro jednotlivá kritéria („středy“ fronty se ztrácejí).

NSGA (1994) — první použití dominance přímo jako fitness. Populace se rozdělí do front: F_1 = nedominovaní jedinci z P , F_2 = nedominovaní z $P \setminus F_1$, F_3 = nedominovaní z $P \setminus (F_1 \cup F_2)$ atd. Jedinci z první fronty dostanou „fiktivní“ fitness, která se dělí *nikovým faktorem* $\sum_j sh(i, j)$, kde $sh(i, j) = 1 - (d(i, j)/d_{share})^2$ pro $d(i, j) < d_{share}$ a 0 jinak. Druhá fronta dostane fitness menší než nejhorší jedinec první fronty, opět dělenou nikovým faktorem, atd. Nevýhody: nutnost nastavit d_{share} , chybějící elitismus, výpočetní složitost $O(kN^3)$.

15.4 NSGA-II

NSGA-II (Deb a kol., 2000)

Opravuje nedostatky NSGA. Tři pilíře: **(1) rychlé nedominované třídění** do front (složitost $O(kN^2)$), **(2) crowding distance** místo parametru d_{share} , **(3) elitismus** — rodiče i potomci se spojí, setřídí a lepší polovina tvoří novou populaci.

Crowding distance

Pro každou frontu zvlášť: pro každé kritérium m seřaď jedince podle f_m , krajním jedincům přiřaď ∞ a ostatním přičti normalizovanou vzdálenost sousedů:

$$d_i = \sum_{m=1}^k \frac{f_m(i+1) - f_m(i-1)}{f_m^{\max} - f_m^{\min}}.$$

Vyjadřuje „obvod kvádry“ tvořené nejbližšími sousedy — velká hodnota znamená řídce obsazenou oblast, kterou chceme zachovat.

Crowded comparison operator

Jedinec i je lepší než j , právě když (a) i leží v *nižší* frontě ($rank_i < rank_j$), nebo (b) fronty se shodují a i má *větší* crowding distance. Žádná skalární fitness se tedy vůbec nepočítá — porovnávají se jen tato dvě čísla, typicky v binárním turnaji.

Algoritmus 10 NSGA-II (jedna generace)

- 1: $R_t \leftarrow P_t \cup Q_t$ ▷ rodiče a potomci, celkem $2N$ jedinců
- 2: $(F_1, F_2, \dots) \leftarrow$ nedominované třídění R_t
- 3: $P_{t+1} \leftarrow \emptyset$; $i \leftarrow 1$
- 4: **while** $|P_{t+1}| + |F_i| \leq N$ **do**
- 5: spočítej crowding distance ve F_i ; $P_{t+1} \leftarrow P_{t+1} \cup F_i$; $i \leftarrow i + 1$
- 6: **end while**
- 7: seřaď F_i sestupně podle crowding distance a doplň P_{t+1} na N jedinců
- 8: $Q_{t+1} \leftarrow$ turnajová selekce (crowded comparison) + křížení (SBX) + mutace

Číselný příklad crowding distance. Fronta $A(1, 10)$, $B(2, 7)$, $C(4, 5)$, $D(8, 2)$; rozsahy: $f_1 \in [1, 8]$ (rozpětí 7), $f_2 \in [2, 10]$ (rozpětí 8). Krajní body A a D dostanou ∞ (v obou kritériích jsou krajní).

$$d_B = \frac{f_1(C) - f_1(A)}{7} + \frac{f_2(A) - f_2(C)}{8} = \frac{4 - 1}{7} + \frac{10 - 5}{8} = 0,43 + 0,63 = 1,05,$$

$$d_C = \frac{f_1(D) - f_1(B)}{7} + \frac{f_2(B) - f_2(D)}{8} = \frac{8 - 2}{7} + \frac{7 - 2}{8} = 0,86 + 0,63 = 1,48.$$

Při nutnosti vybrat jednoho z B , C zvítězí C — leží v řidčeji obsazené části fronty.

NSGA-III. Pro úlohy s mnoha kritérii ($k \geq 4$, *many-objective*), kde crowding distance selhává (téměř vše je nedominované). Crowding distance je nahrazena množinou rovnoměrně rozmístěných **referenčních bodů** (jejich počet zhruba odpovídá velikosti populace, $N = \binom{p+k-1}{p}$ pro p dělení). Po nedominovaném třídění se přednostně doplňují ty referenční směry, které mají nejmeně přiřazených řešení; nemá-li směr žádné řešení, přežije jedinec s nejmenší kolmou vzdáleností od příslušného referenčního vektoru.

15.5 Další významné algoritmy

SPEA2 (Zitzler, 2001). Používá externí **archiv** nedominovaných řešení. Fitness má dvě části: *raw fitness* jedince je součet *sil* (*strength*) všech jedinců, kteří ho dominují, kde síla = počet jedinců, které daný jedinec dominuje (menší je lepší, nedominovaní mají 0); k tomu se přičte *hustota* odvozená ze vzdálenosti k σ -tému nejbližšímu sousedovi ($\sigma = \sqrt{N + \bar{N}}$). Ořezávání archivu (*truncation*) postupně odstraňuje jedince s nejbližším sousedem, což zaručuje zachování krajních bodů.

MOEA/D (Zhang & Li, 2007). Dekompoziční přístup: úloha se rozloží na μ *skalárních* podúloh s rovnoměrně rozloženými váhovými vektory $\lambda^{(1)}, \dots, \lambda^{(\mu)}$ (Čebyševova skalarizace vůči referenčnímu bodu z^*). Každý jedinec populace odpovídá jedné podúloze a spolupracuje jen se svým *sousedstvím* $B(i)$ (podúlohy s nejbližšími váhovými vektory). V každé iteraci se pro každé i vyberou dva rodiče z $B(i)$, vytvoří se potomek y (případně vylepšený lokálním prohledáváním) a y nahradí ta řešení v $B(i)$, pro jejichž skalarizovanou funkci $g(\cdot | \lambda^{(i)}, z^*)$ je lepší; průběžně se aktualizuje z^* . Výhody: nízká složitost, dobrá škálovatelnost do mnoha kritérií, přirozená rovnoměrnost pokrytí fronty.

SMS-EMOA / indikátorové algoritmy. Selektce přímo maximalizuje kvalitativní *indikátor* — typicky hypervolume (S -metriku). Algoritmus je steady-state: vytvoří se jeden potomek, spojí se s populací a vyřadí se ten jedinec (z nejhorší fronty), jehož odebrání sníží hypervolume nejmeně (nejmenší *hypervolume contribution*). Teoreticky nejlépe podložený přístup (maximalizace hypervolume implikuje Pareto-optimalitu), ale výpočet hypervolume je exponenciálně drahý v počtu kritérií.

15.6 Metriky kvality aproximace

Hypervolume (S -metrika)

Objem (v prostoru kritérií) oblasti dominované nalezenou frontou a ohraničené *referenčním bodem* r (zvoleným tak, aby byl dominován všemi řešeními):

$$HV(A, r) = \text{Vol} \left(\bigcup_{a \in A} [f_1(a), r_1] \times \dots \times [f_k(a), r_k] \right).$$

Jediná běžná metrika, která je *striktně monotónní vůči dominanci* (lepší fronta má vždy vyšší hypervolume) a nevyžaduje znalost skutečné Pareto fronty.

Číselný příklad. Fronta $\{(2, 7), (4, 5), (8, 2)\}$, referenční bod $r = (10, 10)$. Rozdělíme na svislé pruhy (setřídíme podle f_1 sestupně):

$$(8, 2) : (10 - 8)(10 - 2) = 16, \quad (4, 5) : (8 - 4)(10 - 5) = 20, \quad (2, 7) : (4 - 2)(10 - 7) = 6.$$

Celkem $HV = 42$. Přidáme-li dominovaný bod $(5, 8)$, hodnota se nezmění; přidáme-li nedominovaný bod, vzroste.

Další metriky. *GD* (generational distance) — průměrná vzdálenost nalezených bodů od skutečné fronty (měří konvergenci); *IGD* (inverted GD) — průměrná vzdálenost bodů skutečné fronty od nalezené aproximace (měří konvergenci i pokrytí; obojí vyžaduje znalost skutečné fronty); *spread/spacing* — rovnoměrnost rozložení; ε -*indikátor* — nejmenší ε , o které je nutné posunout frontu A , aby dominovala B .

15.7 Souhrnné srovnání MOEA

algoritmus	princip selekce	diverzita	poznámka
VEGA	podpopulace podle f_i	žádná	konverguje ke krajům
NSGA	fronty + fiktivní fitness	fitness sharing (d_{share})	bez elitismu, $O(kN^3)$
NSGA-II	fronty + crowding	crowding distance	elitismus, $O(kN^2)$, standard
NSGA-III	fronty + referenční body	referenční směry	many-objective
SPEA2	síla dominujících	σ -tý soused, truncation	externí archiv
MOEA/D	skalarizované podúlohy	rovnoměrné váhové vektory	sousedství, škáluje
SMS-EMOA	příspěvek k hypervolume	implicitní	teoreticky nejčistší, drahé

15.8 Shrnutí k zkoušce

- Dominance ($x \prec y$: ve všech kritériích ne horší a alespoň v jednom lepší), Pareto množina a Pareto fronta; dvojí cíl = konvergence + diverzita.
- Skalarizace: lineární (jednoduchá, ale nenajde nekonvexní části fronty), ε -constraint, Čebyševova.
- NSGA-II: nedominované třídění do front, crowding distance $d_i = \sum_m (f_m(i+1) - f_m(i-1)) / (f_m^{\max} - f_m^{\min})$, krajní body ∞ ; crowded comparison (nižší fronta, při shodě větší crowding); elitismus spojením $P_t \cup Q_t$. Umět spočítat crowding distance na příkladu.
- SPEA2 (archiv, síla dominujících, hustota přes σ -tého souseda), MOEA/D (Čebyševova skalarizace, váhové vektory, sousedství), SMS-EMOA (příspěvek k hypervolume), NSGA-III (referenční body pro many-objective).
- Hypervolume: objem dominovaný frontou vůči referenčnímu bodu; jediná běžná metrika monotónní vůči dominanci a bez znalosti skutečné fronty. Dále GD, IGD, spread, ε -indikátor.

16 Souhrnné srovnání a typické zkouškové otázky

16.1 Mapa oboru

- **Evoluční algoritmy** (populace, fitness, křížení, mutace, selekce): genetické algoritmy (binární, celočíselné, permutační, reálné), evoluční strategie, diferenciální evoluce, evoluční programování, genetické programování (stromové, lineární, kartézské, gramatická evoluce), learning classifier systems, algoritmy odhadu rozdělení (EDA).
- **Přírodou inspirované metody bez evoluce**: rojové algoritmy (PSO, ACO, ABC, firefly).
- **Nebiologicky inspirované metaheuristiky**: tabu search, scatter search, novelty search, simulované žíhání, memetické algoritmy.
- **Aplikační oblasti** s vlastními reprezentacemi a operátory: kombinatorická optimalizace, neuro-evoluce, zpětnovazební učení, vícekritériální optimalizace, AutoML/NAS.

16.2 Souhrnná srovnávací tabulka algoritmů

algoritmus	reprezentace	variace	selekce	co si zapamatovat
SGA	binární řetězec	1-bod. křížení, bit-flip	ruleta, generační	teorie schémat, BBH
ES	$\mathbb{R}^n + \sigma$	gaussovská mutace, rekombinace	náhodní rodiče, $(\mu, \lambda)/(\mu + \lambda)$	pravidlo 1/5, samo-adaptace
CMA-ES	$\mathbb{R}^n$, model $\mathcal{N}(m, \sigma^2 C)$	vzorkování z rozdělení	pořadová, μ nejlepších	evoluční cesty, rank-1/rank- μ
DE	$\mathbb{R}^n$	$\vec{x}_a + F(\vec{x}_b - \vec{x}_c)$, binom. křížení	souboj rodič-potomek	měřítko z populace
EP	doménová (automat)	jen mutace	turnaj rodič+potomci	genotyp = fenotyp
GP	syntaktický strom	výměna podstromů, mutace	turnaj	bloat, ADF
PSO	$\mathbb{R}^n +$ rychlost	polybové rovnice	žádná (jen paměť)	w, c_1, c_2 , gbest/l-best
ACO	konstrukce cesty	feromon + heuristika	implicitní	$\tau^\alpha \eta^\beta$, odpařování
SA	jedno řešení	soused	Metropolis	rozvrh chlazení
MA	libovolná	EA operátory + lokální hledání	libovolná	Lamarck vs. Baldwin
NSGA-II	libovolná	SBX + polynom. mutace	fronty + crowding	elitismus, crowding distance
NEAT	seznam hran	mutace struktury, křížení dle ID	sdílená fitness v druhu	inovační čísla, speciace

16.3 Průřezová témata — co se ptá napříč otázkami

- **Explore vs. exploatace** — jak ji řeší každý algoritmus: GA (křížení vs. selekční tlak), ES (σ a pravidlo 1/5), SA (teplota), PSO (w a topologie), ACO (q_0 , odpařování), tabu (tabu list), novelty search (jen explore).
- **Udržení diversity** — fitness sharing, crowding, speciace, ostrovní model, nenáhodné páření, restart, hypermutace.
- **Adaptace parametrů** — ladění vs. řízení; pevná, adaptivní a samoadaptivní strategie; kde se v kterém algoritmu vyskytuje.
- **Práce s omezeními** — penalizace, oprava, dekodér, uzavřené operátory, vícekritériální formulace.
- **Lamarckismus vs. baldwinismus** — memetické algoritmy, oprava jedinců, neuroevoluce s doučováním vah.
- **Teoretické zázemí** — věta o schématech a její kritika, Voseho model, Markovovy řetězce a konvergence elitistického GA, No Free Lunch, konvergence SA.

16.4 Typické zkouškové otázky

1. Popište obecné schéma evolučního algoritmu a vyjmenujte typy selekce včetně jejich vlastností. Spočtete ruletovou selekci pro danou populaci.
2. Formulujte a dokažte větu o schématech. Co je řád a definující délka? Jaké jsou slabiny věty?
3. Vysvětlete hypotézu stavebních bloků a implicitní paralelismus. Co je deceptivní úloha?
4. Popište Voseho model SGA: co jsou vektory p a s , co dělá operátor $\mathcal{F}$, co $\mathcal{M}$, jaké mají pevné body? Jak vypadá model konečné populace jako Markovův řetězec?
5. Co jsou algoritmy odhadu rozdělení (EDA)? Jaký je jejich vztah k Voseho pohledu na GA a jak se liší UMDA/PBIL od BOA?
6. Jaký je rozdíl mezi (μ, λ) a $(\mu + \lambda)$ -ES? Vysvětlete pravidlo 1/5 a samoadaptaci σ . Jak funguje CMA-ES?

7. Popište jeden krok diferenciální evoluce včetně vzorců a číselného příkladu. Jaké jsou varianty mutace?
8. Jak funguje stromové genetické programování? Co je bloat a jak se proti němu bojuje? K čemu slouží ADF?
9. Vysvětlete gramatickou evoluci a kartézské GP — jaký je genotyp a fenotyp?
10. Co je koevoluce, jaké má typy a jaké patologie? Jak se hodnotí fitness?
11. Co je otevřená evoluce, jaké má znaky a proč se umělé systémy „zasekávají“? Jak funguje novelty search a proč se u deceptivních úloh vyplatí zahodit cíl?
12. Vysvětlete iterované věžňovo dilemma, strategii TFT a vlastnosti úspěšných strategií.
13. Napište rovnice PSO a vysvětlete význam jednotlivých členů. Jaký je rozdíl mezi gbest a lbest topologií?
14. Popište ACO: pravidlo přechodu, aktualizaci feromonu, rozdíl mezi AS a ACS.
15. Vysvětlete simulované žíhání, Metropolisovo kritérium a rozvrh chlazení. Kdy konverguje?
16. Co je memetický algoritmus? Vysvětlete rozdíl mezi lamarckovským a baldwinovským učením.
17. Porovnejte michiganský a pittsburghský přístup. Jak funguje XCS a proč je fitness založena na přesnosti?
18. Popište NEAT: k čemu slouží inovační čísla a speciace? Co přidává HyperNEAT?
19. Jak se řeší TSP evolučním algoritmem? Popište PMX, OX, CX a ER.
20. Jak se v EA pracuje s omezeními (na příkladu problému batohu)?
21. Definujte dominanci a Pareto frontu. Popište NSGA-II a spočtete crowding distance na příkladu. Co je hypervolume?

16.5 Závěrečné shrnutí

- Evoluční a rojové algoritmy jsou **robustní metaheuristiky** pro úlohy, kde selhávají exaktní a gradientní metody: black-box, nediferencovatelné, diskrétní, strukturované, vícekriteriální a dynamické úlohy.
- Jejich síla není v jednom „nejlepším“ algoritmu (No Free Lunch), ale ve **schopnosti přizpůsobit reprezentaci a operátory dané doméně** a v možnosti **hybridizace** s lokálními heuristikami a doménovými znalostmi.
- Teorie (schémata, Voseho model, konvergenční věty, analýza doby běhu) poskytuje intuici a asymptotické záruky, ale praktický návrh zůstává z velké části experimentální disciplínou.